UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

Plataforma web para la gestión integral y asistencia de diseño en estudios de tatuaje

PABLO NICOLÁS CORREA MELLA

Profesor Guía: FRANCISCO ARIAS

Memoria para optar al título de
Ingeniero Civil en Computación

Curicó – Chile
mes: Mayo, año: 2026

UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

Plataforma web para la gestión integral y asistencia de diseño en estudios de tatuaje

PABLO NICOLÁS CORREA MELLA

Profesor Guía: FRANCISCO ARIAS

Profesor Informante:

Profesor Informante:

Memoria para optar al título de
Ingeniero Civil en Computación

El presente documento fue calificado con nota: _____

Curicó – Chile

mes: Mayo, año: 2026

Dedicado a ...

AGRADECIMIENTOS

Agradecimientos ...

TABLA DE CONTENIDOS

página

Dedicatoria i

Agradecimientos ii

Tabla de Contenidos iii

Índice de Figuras vi

Índice de Tablas ix

Resumen xi

1. Introducción **12**

1.1. Contexto 12

1.1.1. Descripción del problema 13

1.1.2. Propuesta de solución 15

1.2. Objetivos 17

1.2.1. Objetivo general: 17

1.2.2. Objetivos específicos: 17

1.3. Alcances 18

1.4. Plan de trabajo 19

1.4.1. Descripción de etapas del proyecto 19

1.4.2. Tareas para objetivos específicos 20

1.5. Resumen del capítulo 21

2. Marco Teórico **22**

2.1. Plataformas digitales 22

2.1.1. Aplicación móvil nativa 22

2.1.2. Progressive Web App 23

2.1.3. Página web 23

2.1.4. Elección de plataforma digital a desarrollar 23

2.2. Conceptos básicos 25

2.2.1. Conceptos globales 25
2.2.2. Conceptos de software 26
2.3. Tecnologías 26
2.3.1. Frontend 26
2.3.2. Backend 27
2.3.3. Sistema gestor de base de datos (SGBD) 27
2.3.4. Servicio de mensajería 28
2.3.5. Despliegue de aplicaciones 28
2.3.6. Herramienta para repositorio GithHub 28
2.3.7. Sistema gestor de proyectos 28
2.4. Estado del arte 29
2.5. Metodologías 32
2.5.1. Metodologías de desarrollo 32
2.5.2. Metodologías de evaluación del proyecto 34
2.6. Resumen capítulo 36

3. Metodologías **38**
3.1. Personal Extreme Programming (PXP) 38
3.1.1. Requerimientos 38
3.1.2. Planificación 38
3.1.3. Iteración 44
3.2. Evaluación 48
3.2.1. Prueba de caja negra 48
3.2.2. Prueba de caja blanca 49
3.2.3. Prueba de usabilidad 49
3.2.4. Encuesta 50
3.3. Resultados esperados 51
3.4. Resumen capítulo 52

4. Desarrollo **53**
4.1. Análisis de factibilidad 53
4.1.1. Factibilidad técnica 53
4.1.2. Factibilidad económica 54
4.2. Diseño de software 56

4.2.1. Arquitectura física 56
4.2.2. Arquitectura lógica 57
4.2.3. Modelo de datos 58
4.2.4. Mockups 59
4.3. Iteraciones 67
4.3.1. Iteración 1 67
4.3.2. Iteración 2 67
4.3.3. Iteración 3 76
4.3.4. Iteración 4 85
4.3.5. Iteración 5 95
4.3.6. Iteración 6 107
4.3.7. Iteración 7 119
4.3.8. Iteración 8 131
4.3.9. Iteración 9 140
4.3.10. Iteración 10 150

5. Pruebas y análisis de resultados **160**
5.1. Pruebas 160
5.1.1. Pruebas funcionales 161
5.1.2. Pruebas de usabilidad (SUS) 162
5.2. Resultados 163
5.2.1. Resultados de pruebas funcionales 163
5.2.2. Resultados de usabilidad (SUS) 171
5.3. Análisis de los resultados 172
5.4. Análisis de accesibilidad 174
5.5. Análisis de sostenibilidad 175
5.6. Resumen del capítulo 176

Glosario **178**

Bibliografía **180**

ÍNDICE DE FIGURAS

página

1.1. Modelo del proceso de negocio de un estudio de tatuajes (BPMN) . . 15

2.1. Fases de metodología Personal Extreme Programming (PXP) 34
2.2. Proceso de Prueba de Caja Blanca 35
2.3. Proceso de Prueba de Caja Negra 35

3.1. Trello 45
3.2. Diagrama de secuencia 46
3.3. Lista de verificación de pruebas 47
3.4. Registro de prueba de caja negra 48

4.1. Arquitectura Física 57
4.2. Arquitectura Lógica 57
4.3. Diagrama Entidad-Relación 59
4.4. Idea 1 60
4.5. Idea 2 61
4.6. Idea 3 62
4.7. Mockup Inicio Sesión 63
4.8. Mockup Dashboard 64
4.9. Mockup Dashboard Móvil 64
4.10. Mockup Previsualización 3D 65
4.11. Mockup Previsualización 3D Móvil 66
4.12. Diagrama de secuencia Inicio Sesión 69
4.13. Implementación JWT 70
4.14. Implementación Email 71
4.15. Pruebas Unitarias 72
4.16. Resultado Pruebas Unitarias 72
4.17. Vista Inicio de sesión 73
4.18. Vista Recuperar contraseña 73
4.19. Vista Resumen 74
4.20. Diagrama de secuencia Subir Boceto 79
4.21. Implementación de la subida de imágenes 80

4.22. Consulta filtrada por usuario 81
4.23. Vista Galería de bocetos 82
4.24. Resultado pruebas unitarias Iteración 3 84
4.25. Diagrama de secuencia Nivel Crítico 88
4.26. Ajuste de stock en la base de datos 89
4.27. Cálculo de los ingresos del mes 90
4.28. Vista Inventario 91
4.29. Vista Resumen con stock bajo 92
4.30. Resultado pruebas unitarias Iteración 4 94
4.31. Diagrama de secuencia Cotizar 98
4.32. Área efectiva y tiempo de trabajo 99
4.33. Consumo de un insumo 100
4.34. Descuento de stock de una cotización 102
4.35. Vista Cotizaciones 103
4.36. Costo de insumos en el proyecto 104
4.37. Resultado pruebas unitarias Iteración 5 106
4.38. Diagrama de secuencia Agendar Cita 110
4.39. Revisión del horario ocupado 112
4.40. Puntaje del cliente 113
4.41. Vista Citas 114
4.42. Vista Clientes 115
4.43. Ficha del cliente 116
4.44. Resultado pruebas unitarias Iteración 6 118
4.45. Diagrama de secuencia Recordatorio 122
4.46. Reclamo del envío 123
4.47. Citas con recordatorio pendiente 124
4.48. Vista Configuración 126
4.49. Vista Administración 127
4.50. Resultado pruebas unitarias Iteración 7 130
4.51. Diagrama de secuencia Colocar Tatuaje 133
4.52. Envolvente de la calca 135
4.53. Recorte de la vecindad de la malla 136
4.54. Vista Previsualización 3D 137
4.55. Tatuaje colocado sobre el antebrazo 138

4.56. Diagrama de secuencia Desactivar Cuenta 143
4.57. Estado de la cuenta en cada petición 144
4.58. Puertas de la baja de cuentas 145
4.59. Vista Administración, cuentas 146
4.60. Vista Cambiar contraseña 147
4.61. Resultado pruebas unitarias Iteración 9 149
4.62. Diagrama de secuencia Mi estudio 153
4.63. Métricas de uso por estudio 154
4.64. Resumen del estudio 155
4.65. Vista Administración, métricas 156
4.66. Vista Mi estudio 156
4.67. Resultado pruebas unitarias Iteración 10 158

5.1. Corrida de las pruebas funcionales 167
5.2. Ejecución de la batería completa 169
5.3. Informe de cobertura 171

ÍNDICE DE TABLAS

página

2.1. Análisis comparativo de funcionalidades por software 31

3.1. Historias de Usuario 39
3.2. Priorización y estimación de esfuerzo para cada historia de usuario . . 43
3.3. Asignación de historias de usuario a iteraciones 44

4.1. Riesgos técnicos identificados y su tratamiento 54
4.2. Costo mensual de la infraestructura en la nube 55
4.3. Resumen de costos del proyecto 56
4.4. Tareas asociadas a la Iteración 2 68
4.5. Pruebas unitarias implementadas en la Iteración 2 75
4.6. Resultado de ejecución de pruebas unitarias de la Iteración 2 76
4.7. Tareas asociadas a la Iteración 3 78
4.8. Pruebas unitarias implementadas en la Iteración 3 83
4.9. Resultado de ejecución de pruebas unitarias de la Iteración 3 84
4.10. Tareas asociadas a la Iteración 4 87
4.11. Pruebas unitarias implementadas en la Iteración 4 93
4.12. Resultado de ejecución de pruebas unitarias de la Iteración 4 94
4.13. Tareas asociadas a la Iteración 5 97
4.14. Pruebas unitarias implementadas en la Iteración 5 105
4.15. Resultado de ejecución de pruebas unitarias de la Iteración 5 106
4.16. Tareas asociadas a la Iteración 6 109
4.17. Pruebas unitarias implementadas en la Iteración 6 117
4.18. Resultado de ejecución de pruebas unitarias de la Iteración 6 118
4.19. Tareas asociadas a la Iteración 7 121
4.20. Pruebas unitarias implementadas en la Iteración 7 129
4.21. Resultado de ejecución de pruebas unitarias de la Iteración 7 130
4.22. Tareas asociadas a la Iteración 8 132
4.23. Medición de rendimiento del visor, antes y después 139
4.24. Tareas asociadas a la Iteración 9 142
4.25. Pruebas unitarias implementadas en la Iteración 9 148
4.26. Resultado de ejecución de pruebas unitarias de la Iteración 9 149

4.27. Tareas asociadas a la Iteración 10 152
4.28. Pruebas unitarias implementadas en la Iteración 10 157
4.29. Resultado de ejecución de pruebas unitarias de la Iteración 10 158

5.1. Casos de prueba funcional: autenticación y bocetos 164
5.2. Casos de prueba funcional: previsualización, inventario y cotizaciones 165
5.3. Casos de prueba funcional: agenda y pagos 166
5.4. Casos de prueba funcional: avisos y administración 167
5.5. Pruebas automatizadas por módulo 168
5.6. Cobertura de código por capa 170
5.7. Escala de interpretación del puntaje SUS 172
5.8. Resultados de la revisión de accesibilidad 174

RESUMEN

En la actualidad, el flujo de trabajo de los tatuadores presenta una alta fragmentación tecnológica, lo que hace hace que los profesionales dividan sus tareas administativas y artísticas en múltiples sistemas no integados. La falta de herramientas unificadas genera ineficiencias a la hora de trabajar, ya sea adecuando el tatuaje en una zona establecida del cuerpo, en pérdidas de tiempo en el cálculo de presupuestos y problemas gestionando las citas con los clientes.

El proyecto propone el diseño y construcción de una plataforma web integral que unifica el flujo de trabajo del estudio de tatuajes. El sistema incorpora módulos de almacenamiento en la nube para bocetos, previsualización 3D sobre modelos anatómicos, gestión de inventario con cotizaciones automatizadas, y un agendador de citas integrando una API de mensajería para los recordatorios.

Desde un análisis crítico, la solución posee factibilidad técnica, operativa y económica. Al utilizar tecnologías web y librerías de código abierto, el sistema elimina la dependencia de licencias costosas y no requiere hardware especializado. También teniendo en cuenta que se cuenta con la ayuda de un tatuador experto que ayuda en el proceso de captura de requerimientos y pruebas del sistema en cada iteración a través de su retroalimentación.

Como proyección de impacto, es un producto mínimo viable (MVP) que optimiza el modelo de negocio, con un objetivo de disminución de ausentismo de clientes, reducción de tiempo administrativo en cálculos de costos e inventario y una disminución de tiempo invertido en pruebas de diseño sobre la piel.

1. Introducción

La digitalización y el desarrollo de software han comenzado a transformar el funcionamiento de las disciplinas creativas, abriendo nuevas oportunidades de optimización para el rubro del arte corporal. El diseño e implementación de plataformas a la medida permite a los artistas modernos modernizar sus flujos de trabajo tradicionales, logrando un equilibrio eficiente entre la gestión administrativa del estudio y la ejecución de su obra.

El presente capítulo tiene como propósito describir las bases que fundamentan el trabajo realizado en esta memoria de título. Para ello, se expone inicialmente el contexto de crecimiento del rubro del tatuaje, seguido de la descripción detallada de la problemática detectada a nivel operativo y administrativo. Posteriormente, se define la propuesta de solución tecnológica, sus objetivos, los alcances que delimitan el desarrollo y, finalmente, el plan de trabajo estructurado para su ejecución.

1.1. Contexto

El presente marco teórico establece los fundamentos históricos, comerciales y socioculturales que respaldan el desarrollo de la plataforma. Para comprender la envergadura y necesidad de este proyecto, es necesario analizar la evolución del rubro del tatuaje, partiendo desde sus orígenes hasta consolidarse como una industria global, para luego conectar estas métricas en la realidad nacional y local.

El tatuaje forma parte de la historia del ser humano, siendo la evidencia más antigua el hallazgo de los restos momificados del cazador neolítico conocido como "Ötzi", datados hace más de 5.300 años. En civilizaciones antiguas, como la egipcia, romana y maya, la inserción de pigmentos bajo la epidermis cumplía funciones

estrictamente identitarias, indicando estatus o linaje, y en otros casos, marcando a prisioneros y esclavos [1]. Si bien durante gran parte de la historia occidental moderna el tatuaje estuvo estigmatizado y relegado a la criminalidad, a partir del siglo XX y XXI ha experimentado una resignificación profunda. En la actualidad, ha superado el estigma social para consolidarse como una expresión artística legítima y un medio de afirmación de la identidad y la subjetividad personal [2].

En la actualidad, la industria del arte corporal es uno de los mercados de más rápido y constante crecimiento en el mundo. Según estimaciones globales, la industria alcanzó un valor aproximado de 2.420 millones de dólares en el año 2025, y se proyecta que ascienda a 6.220 millones de dólares para el año 2035, exhibiendo una Tasa de Crecimiento Anual Compuesta (CAGR) del 9,9 % [3]. Este crecimiento exponencial está directamente impulsado por la creciente aceptación social de las modificaciones corporales, el cambio demográfico liderado por las generaciones "Millennials" y "Gen Z", y un mayor poder adquisitivo destinado a la autoexpresión [3].

Esta tendencia global también se encuentra en el panorama chileno. Un estudio estadístico realizado por la consultora GfK Adimark evidenció que el 17 % de los chilenos mayores de 15 años posee al menos un tatuaje. Dicha cifra experimenta un alza dramática al observar el segmento de adultos jóvenes (entre 25 y 34 años), donde el 38 % de la población se encuentra tatuada. En la última década, la demanda en el país ha crecido de manera exponencial, posicionando el nivel técnico y artístico del tatuaje en Chile a la par de los mercados europeos y norteamericanos[4].

En un contexto local, la ciudad de Curicó no ha quedado ajena a esta explosión de demanda. El aumento sostenido de clientes ha propiciado la apertura de nuevos estudios y el establecimiento de artistas independientes en la zona que buscan ganarse un puesto en el mercado local. Por consiguiente, el desarrollo de este proyecto cobra una relevancia para el apoyo a este dinamismo local. Al dotar a los artistas de Curicó con herramientas tecnológicas, se busca potenciar su capacidad y facilitar su consolidación en un mercado en plena expansión.

1.1.1. Descripción del problema

Actualmente el trabajo de los tatuadores se encuentra muy fragmentando por las diferentes herramientas tecnológicas que se pueden utilizar al momento de realizar

su trabajo, ya sea en el ámbito operativo como en lo artístico. El hecho de dividir sus labores crea dificultades para optimizar sus tareas de una forma limpia.

A continuación se detallan las problemáticas operativas y administrativas, las cuales fueron analizadas y planteadas por el señor Héctor Salazar Moya quien es un tatuador profesional de Curicó, que ha trabajado en el área durante varios años y en distintos estudios de tatuajes, por lo que tiene un conocimiento profundo de las dificultades que enfrentan los tatuadores de la actualidad.

En el ámbito operativo, uno de los mayores problemas es la previsualización del tatuaje en una zona concreta del cuerpo. Actualmente para visualizar como quedaría un boceto de un tatuaje se ocupa un elemento llamado "stencil", que es una guía en papel hectográfico que transfiere el diseño del papel a la piel del boceto original. El problema surge que este papel es un material que se desecha una vez utilizado, esto quiere decir que si el tatuador se equivoca en la posición que coloca el stencil tendría que repetirse de nuevo el proceso y ocupar material extra. Incluso muchas veces son los mismos clientes que cambian de opinión con respecto a la posición que les gustaría tener su tatuaje, y el hecho de que el tatuador pueda previsualizar el boceto en cualquier parte del cuerpo que considere el lugar más apropiado, le facilitaría sus decisiones artísticas y no tendría los problemas antes mencionados.

Con respecto al ámbito admnistrativo, es posible mencionar más problemáticas que viven los tatuadores para la gestión de su negocio. Para comenzar el agendamiento de las citas es uno de los principales problemas, debido a que no existe una herramienta dedicada para los tatuadores, lo que hace que la gestión de citas sea poco personalizable y haya problemas en la estructuración de tiempos del profesional. En relación a las citas, también hay problemas con los recordatorios hacia los clientes, debido a que muchos de ellos no se presentan a la hora adecuada o simplemente se olvidan de su cita el día acordado, debido a que es el tatuador el que tiene que recordar a sus clientes de forma manual el día antes de la sesión, por lo que al tener múltiples clientes esta acción se puede ver afectada por parte del tatuador.

Otro de los problemas es analizar correctamente el coste los tatuajes, si bien este precio se ve influnciado gran parte por el valor que le quiera dar el artista, analizar su costo real a partir de los materiales utilizados y el tiempo invertido en las sesiones es una labor que es dificil tenerla presente en todo momento.

Por último hay que mencionar también el problema con respecto a la gestión del inventario ya que surgen inconvenientes al momento de gastar insumos y no tener las nociones de cuanto material se va utilizando en cada sesión de trabajo y calcular también su reposición.

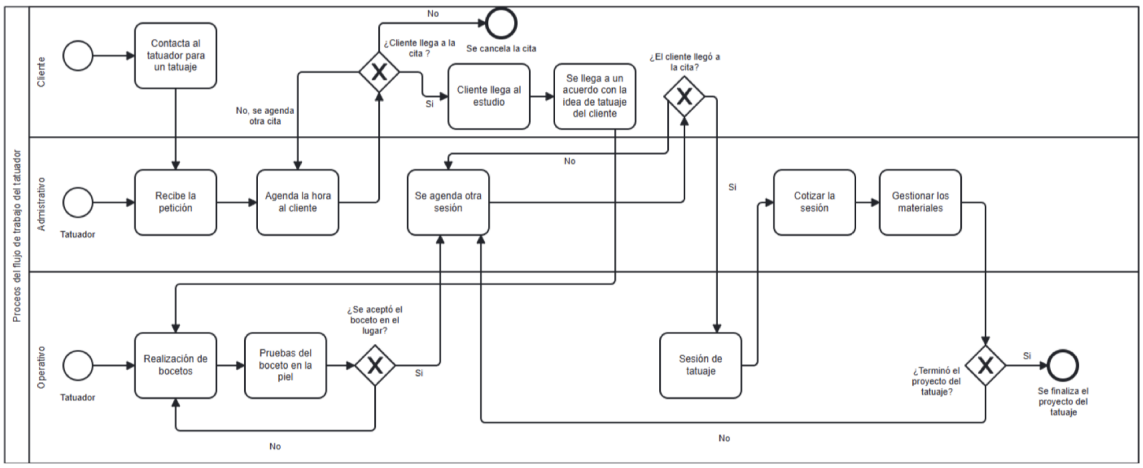


Figura 1.1: Modelo del proceso de negocio de un estudio de tatuajes (BPMN)[5].

La Figura 1.1 representa el flujo del trabajo del tatuador, incluyendo su participación con el cliente, sus labores operativas y admistrativas. Esto describe la complejidad que conlleva desarrollar un proyecto de tatuaje con múltiples iteraciones, ya que, además de ser un proceso extenso, puede involucrar múltiples retrasos por parte del cliente al no poseer un recordatorio automatizado. Por otro lado los procesos que se integran en el trabajo son desarrollados en diferentes herramientas tanto admistrativamente como operativamente, lo que retrasa aún más este flujo.

1.1.2. Propuesta de solución

La solución propuesta consiste en el diseño y construcción de una plataforma web/móvil que unifique el flujo del trabajo de los artistas dedicados al arte del tatuaje, varias de sus funciones fueron creadas a partir de los requisitos de tatuador experto. El sistema considera distintos módulos como la importación y almacenamiento en la nube para la gestión de bocetos, la previsualización 3D sobre modelos anatómicos, el agendamiento de citas con recordatorios para reducir la inasistencia y retrasos de los clientes, un gestor de cotizaciones automatizadas y gestión de inventario.

Con respecto al módulo de importación y almacenamiento en la nube, este permite subir imágenes de los bocetos a la plataforma donde serán guardadas y visibles en la galería de la aplicación. Adicionalmente, el módulo contempla la generación de un enlace público asociado a la galería del tatuador, el cual puede ser compartido con clientes u otras personas interesadas para que visualicen el portafolio de diseños disponibles sin necesidad de contar con una cuenta en la plataforma.

En cuanto al módulo de previsualización 3D sobre modelos anatómicos, este permite visualizar un modelo 3D de un cuerpo humano en la plataforma gracias a un motor de renderizado, en donde los bocetos subidos son transformados para adherirse a la superficie 3D del cuerpo y desplazarse para encontrar la posición y el ángulo que el cliente prefiera.

Sobre el módulo de agendamientos de citas con recordatorios, este permite hacer agendas personalizadas, en donde se puede crear las citas a partir de los nombres de las personas a atender, en las cuales para cada perfil se puede crear una nueva sesión de tatuaje o un nuevo proyecto, donde se pueden incluir la hora de la cita, el precio de la sesión o proyecto, junto con una foto del proceso del tatuaje. Para el problema de los recordatorios se propone implementar una API que integra los mensajes de WhatsApp[6], donde se vinculan los números para enviar avisos antes y despues de la sesión del tatuaje.

El módulo de gestión de cotizaciones automatizadas, permite calcular el precio aproximado del costo del tatuaje a partir del boceto subido a la plataforma, esto se realiza mediante un algoritmo que procesa las dimensiones del boceto, junto con otros parámetros como el grosor de los trazados y colores, ya que el costo se centra en los gastos de material como agujas, tinta y otros insumos para la elaboración del tatuaje, dejando de lado valor adicional que le quiera dar el artista de acuerdo a la complejidad del diseño.

Por último, el módulo de gestión de inventario se encarga de gestionar los materiales que posea el tatuador, ya sea agujas, tintas, insumos de higiene, entre otros, los que puede a editar a su disposición y a la vez se pueden actualizar automaticamente cada vez que se haga una cotización automatizada.

En términos de factibilidad, esta solución es viable gracias a varios factores en los que incluye factibilidad técnica, operativa y económica. En el ámbito técnico, la

solución se apoya en las tecnologías orientadas al desarrollo web, empleando librerías de código abierto establecidas para el renderizado 3D en el navegador e integraciones mediante protocolos estándar para las APIs de mensajería, lo que elimina la necesidad de desarrollar motores gráficos desde cero o adquirir licencias de software. Operativamente, se sustenta directamente con las necesidades reales de los tatuadores, en mayor medida dentro de la zona local, debido a que gran parte de los requerimientos son parte de las ideas del tatuador experto, que estuvo dispuesto a buscar mejoras en el flujo de su trabajo con la plataforma gracias a sus requerimientos, y es por ello que estuvo dispuesto a participar y validar prototipos y avances relacionados a la plataforma. Finalmente, la viabilidad económica se garantiza al basar la arquitectura en lenguajes de programación y gestores de bases de datos gratuitos, lo que reduce a cero los costos iniciales por licencias, mientras que los gastos de infraestructura se mantendrán al mínimo aprovechando entornos locales y capas gratuitas de servicios en la nube durante las fases de desarrollo y pruebas de concepto.

1.2. Objetivos

En esta sección se presentan el objetivo general y especifícos del proyecto.

1.2.1. Objetivo general:

Optimizar el flujo del trabajo artístico y administrativo de los tatuadores mediante una plataforma web que integre previsualización 3D, agendamiento personalizado de citas, cotizaciones automatizadas y gestión de inventario.

1.2.2. Objetivos específicos:

- Analizar los procesos actuales de los tatuadores, mediante entrevistas para obtener métricas a optimizar.
- Disminuir en al menos un 50 % el ausentismo y retrasos de los clientes a sus sesiones mediante un módulo de agendamientos de citas con recordatorios.
- Reducir en al menos un 40 % el tiempo empleado en el cálculo manual de costos y revisión de stock mediante un módulo de cotizaciones automatizadas y gestión de inventario.

- Reducir en al menos un 30 % el tiempo invertido en pruebas de tamaño y ubicación del diseño en el cliente mediante un módulo de previsualización 3D sobre modelos anatómicos.
- Implementar un módulo de importación y almacenamiento en la nube para la gestión de bocetos.
- Evaluar la calidad de la solución implementada a través de la ejecución de tests unitarios (integración), caja negra, caja blanca con cobertura de al menos 95 %.
- Validar el sistema mediante pruebas de aceptación con al menos un 90 % de evaluación positiva con el usuario y una evaluación de usabilidad aplicando un cuestionario SUS con un puntaje de al menos 85 puntos en escala Likert.

1.3. Alcances

En esta sección se presentan los alcances del proyecto.

- En este trabajo se espera implementar el Producto Mínimo Viable (MVP) que incluye la web funcional con importación de bocetos, mapeo 3D en navegador, calculadora de costos y gestor de citas con API de mensajería.
- No se considera implementar un módulo que permita la creación de bocetos mediante dibujo en la plataforma.
- En este trabajo no se incluye la modificación avanzada de bocetos mediante Inteligencia Artificial Generativa.
- No se contempla implementar visualización en Realidad Aumentada (AR) sobre la piel en tiempo real, limitando la visualización al entorno 3D web.
- El sistema considera la importación en formato imágenes PNG, con un tamaño limitado para garantizar un rendimiento fluido para el renderizado 3D.
- La plataforma considera modelos 3D ya integrados (cuerpo humano, brazo, pierna, torso). No se contempla la funcionalidad de subir modelos 3D por parte del usuario.
- El algoritmo de cotización de tatuaje no incluye un precio que contemple el arte del tatuaje. Solo de insumos u otros gastos objetivos.

1.4. Plan de trabajo

A continuación se presenta el plan de trabajo del proyecto.

1.4.1. Descripción de etapas del proyecto

En esta sección se presentan las etapas que posee el proyecto, tareas más importantes y la cantidad de semanas de duración que implicará realizar cada etapa.

1. Requerimientos (2 semanas)
    - Capturar requerimientos mediante reuniones con el cliente experto.
2. Planificación (9 semanas)
    - Creación y estimación de historias de usuario.
    - Definición de tecnologías.
    - Entrevista a cliente sobre métricas de objetivos específicos.
    - Diseño de la arquitectura del sistema y del modelo de la base de datos.
    - Diseño de interfaces (Mockups) centradas en la usabilidad móvil y web.
    - Analizar las consideraciones de guía de desarrollo seguro.
3. Implementación (13 semanas)
    - Configuración inicial del proyecto.
    - Implementar módulo de autenticación de usuarios.
    - Implementar módulo de importación y almacenamiento en la nube para la galería de bocetos.
    - Implementar módulo de gestión de inventario.
    - Implementar módulo de cotizaciones automatizadas conectadas al inventario.
    - Implementar módulo de agenda de citas e integración con la API de mensajería para recordatorios.
    - Implementar módulo de previsualización 3D sobre modelos anatómicos.

4. Evaluación y Validación (4 semanas)
    - Ejecutar pruebas de caja blanca (unitarias).
    - Aplicar pruebas de caja negra.
    - Realizar marcha blanca de la aplicación con el cliente experto.
    - Aplicar prueba de usabilidad (SUS) y evaluar la aceptación del usuario (UAT).
5. Conclusión (2 semanas)
    - Realizar entrevista final y comparar métricas para verificar el cumplimiento de los porcentajes de optimización planteados.
    - Realizar conclusiones y completar documento.

Quedando así un total de 30 semanas para el desarrollo del proyecto. Para mayor detalle de la planificación revisar la Carta Gantt anexo al documento.

1.4.2. Tareas para objetivos específicos

Para asegurar la trazabilidad entre el plan de trabajo y las metas del proyecto, se detallan las acciones que darán cumpliento a los objetivos específicos planteados:

- **Análisis de los procesos actuales y obtención de métricas a optimizar:**
    - *Etapa 1:* Entrevistar al cliente para levantar tiempos actuales de cotización, tiempos de prueba de diseño y tasas de inasistencia.
- **Reducir tiempos de cotización e inventario en un 40 %:**
    - *Etapa 3:* Implementar iterativamente los módulos lógicos de inventario y calculadora de costos.
    - *Etapa 5:* Comparar tiempos manuales originales versus el uso del sistema automatizado.
- **Disminuir ausentismo en un 50 %:**
    - *Etapa 3:* Desarrollar integración con API de mensajería para envío de notificaciones 24 horas antes de la cita.

- *Etapa 5:* Medir la variación en la asistencia registrada tras un mes de marcha blanca.

- **Reducir tiempos de previsualización en un 30 %:**
  - *Etapa 3:* Desarrollar el módulo de previsualización 3D en la web.
  - *Etapa 5:* Cuantificar el ahorro de tiempo al evitar imprimir y posicionar stencils físicos múltiples veces.

- **Validación de Usabilidad (SUS $\geq$ 85):**
  - *Etapa 4:* Aplicar cuestionario de Likert a los usuarios de prueba tras la fase de marcha blanca y procesar el puntaje final.

1.5. Resumen del capítulo

En este capítulo, se dió a conocer el tema del proyecto, permitiendo comprender el contexto, el cual se centra en la transformación en el mundo del tatuaje, como su popularidad a lo largo de la historia y los métodos de trabajo. También se presenta la problemática que se poseen en el y rubro y la solución propuesta a este problema, el cual es la construcción de un sistema de un sistema que permita gestionar la mayor de procesos de un tatuador para la optimización de su flujo de trabajo.

A continuación, el Capítulo 2 detallará conceptos básicos del proyecto, tecnologías a utilizarm estado del arte y metodologías, tanto de desarrollo como de evaluación.

2. Marco Teórico

El presente capítulo está orientado a describir conceptos importantes para comprender el funcionamiento de los procesos de trabajo de un estudio de tatuajes profesional. Además, se darán a conocer las tecnologías que serán parte del proyecto y las metodologías que se usarán para ejecutar y evaluar los resultados del mismo.

2.1. Plataformas digitales

Para abordar las necesidades identificadas del cliente experto, se propone el desarrollo de una plataforma digital integral que unifique las funciones operativas y administrativas del estudio de tatuajes. Para ello se evaluaron diferentes alternativas tecnológicas con el fin de seleccionar una solución que permita centralizar el flujo de trabajo, optimizar la gestión de citas, automatizar las cotizaciones y facilitar la previsualización de diseños. A continuación, se presentan las principales características, ventajas y limitaciones de cada enfoque, junto con una comparación basada en dimensiones críticas para el proyecto.

2.1.1. Aplicación móvil nativa

Una aplicación móvil nativa se desarrolla específicamente para un sistema operativo (iOS o Android) utilizando sus lenguajes y herramientas de desarrollo. Este enfoque permite aprovechar al máximo las capacidades del dispositivo, notificaciones push para recordatorios de gestión de insumos y acceso offline a la galería de bocetos [7][8]. Sin embargo, el desarrollo nativo implica crear dos aplicaciones separadas (una para iOS y otra para Android), lo que aumenta significativamente el tiempo y costo

de desarrollo, además de requerir habilidades especializadas en ambos ecosistemas [9].

2.1.2. Progressive Web App

Una Progressive Web App (PWA) es una aplicación desarrollada con tecnologías web que utiliza APIs modernas para ofrecer una experiencia similar a la de una aplicación nativa, pero se ejecuta en el navegador. Las PWAs pueden ser instaladas en el dispositivo, funcionar offline y enviar notificaciones push, lo que las hace una opción viable para este proyecto. Además, al ser multiplataforma, permite un desarrollo más rápido y económico al mantener una sola base de código para todos los usuarios. No obstante, el acceeso a funciones avanzadas del hardware puede ser limitado en ciertos navegadores, y extiste variabilidad en el nivel de soporte según el sistema opetativo [10].

2.1.3. Página web

Una aplicación web tradicional funciona completamente en línea y depende del navegador para su ejecución. Se caracteriza por su accesibilidad inmediata sin necesidad de instalación, lo que facilita su adopción por parte de los usuarios. Su principal desventaja es la falta de capacidades offline y el acceso limitado a funciones del dispositivo. Aunque es la alternativa más simple, no permite cubrir adecuadamente todas las necesidades del proyecto, especialmente en términos de previsualización y recordatorios personalizados [11].

2.1.4. Elección de plataforma digital a desarrollar

Para seleccionar el entorno digital más adecuado, se realiza un análisis comparativo considerando cinco dinmensiones clave. Estos criterios permiten evaluar la capacidad de cada plataforma para satisfacer los requerimientos funcionales del proyecto.

- **Multiplataforma:** El flujo de trabajo en un estudio de tatuaje se caracteriza por ser multidispositivo: la administración del inventario suele realizarse en un computador, mientras que el diseño y la atención dependen de tabletas o teléfonos móviles. Disponer con una solución que funcione sin adaptaciones

adicionales en teléfonos Android, iOS, computadores o tablets es esencial para garantizar la accesibilidad y usabilidad del sistema en el entorno real de trabajo.

- **Notificaciones push:** Tal como se definió en los alcances, la comunicación con el cliente final se delega a la API de WhatsApp [12], pero la plataforma requiere mantener informado al artista. El uso de notificaciones push permiten que el tatuador reciba alertas transaccionales nativas en su dispositivo (por ejemplo, cuando el sistema automático detecta bajo stock de una tinta o cuando un cliente cancela una cita), incluso si la pestaña de la aplicación se encuentra cerrada.

- **Funcionamiento offline/parcial:** Las conexiones a internet en los estudios de tatuaje pueden presentar problemas. Es una alternativa de gran ayuda para el tatuador que se pueda operar bajo condiciones de red degradadas (enfoque Offline-first). Es por ello que se requiere que si el dispositivo pierde momentáneamente la conexión, la aplicación puede seguir operando mostrando la agenda del día o permitiendo visualizar los bocetos, para luego sincronizar los cambios con la base de datos central una vez que la conectividad se restablezca.

- **Mantenimiento de código único:** Es importante considerar que el desarrollo del proyecto será realizado realizado por una sola persona. Gestionar múltiples bases de código, como ocurre con aplicaciones nativas, incrementaría significativamente la complejidad del proyecto, el tiempo de desarrollo y el riesgo de introducir errores. Por lo tanto, una solución que permita mantener una única base de código para todas las plataformas es fundamental para garantizar la viabilidad del proyecto.

- **Tiempo de desarrollo:** La limitante temporal es un factor crítico en el ciclo de vida del proyecto, el cual contempla un período de 18 semanas para la fase de implementación bajo la metodología *Personal Extreme Programming* (PXP). Es por ello que es necesario tener en cuenta elegir una tecnología que presente una curva baja de aprendizaje y de carga cognitiva.

Tomando en cuenta estas dimensiones, la opción que mejor se alinea con los requerimientos del proyecto es el desarrollo de una Progressive Web App (PWA). Esta elección permite ofrecer una experiencia de usuario fluida y accesible en múltiples

dispositivos, con capacidades offline y notificaciones push, mientras mantiene una única base de código que facilita el desarrollo y mantenimiento dentro del marco temporal establecido.

2.2. Conceptos básicos

A continuación, se presentan algunos conceptos para entender de mejor manera el proyecto.

2.2.1. Conceptos globales

A continuación, se presentan algunos conceptos para entender de mejor manera el proyecto.

- **Estudio de tatuajes:** Espacio físico o establecimiento comercial, regulado por normativas sanitarias, donde operan artistas del tatuaje. Puede funcionar bajo un modelo de artistas residentes (empleados o socios) o recibir a artistas invitados.
- **Boceto digital:** Diseño preliminar o definitivo de un tatuaje creado a través de software de ilustración en dispositivos electrónicos (como tabletas gráficas). Es el archivo base bidimensional (generalmente en formato PNG o JPG) que será transferido a la piel [13].
- **Esténcil (Transfer):** Técnica analógica utilizada en los estudios para transferir el contorno del boceto a la piel del cliente antes de comenzar a tatuar. Tradicionalmente requiere imprimir el diseño en un papel hectográfico especial y adherirlo al cuerpo mediante líquidos de transferencia [14].
- **Proyección de decal (Calcomanía):** Técnica de renderizado tridimensional que permite proyectar una textura bidimensional (el boceto) sobre la superficie curva de una malla tridimensional (el modelo anatómico), adaptándose dinámicamente a su geometría [15].

2.2.2. Conceptos de software

- Backend: Es la parte que no se ve y es esencial en un sitio, encargada de manejar la lógica y el procesamiento de datos necesarios para que todo funcione de manera correcta [16].

- Frontend: Es la parte de un programa, sitio web o dispositivo en la que un usuario puede acceder e intereactuar directamente. En el contexto de diseño web y desarrollo web, se refiere a todas las tecnologías que corren en el navegador y que se encargan de la interactividad con los usuarios [17].

2.3. Tecnologías

A continuación se presentan las principales tecnologías utilizados para el desarrollo del proyecto.

2.3.1. Frontend

Para la contrucción del frontend se utiliza React[18] como biblioteca de JavaScript[19] para construir la interfaz de usuario. React es ampliamente reconocido por su eficiencia en la creación de interfaces dinámicas y su capacidad para manejar el estado de la aplicación de manera efectiva. Además, React permite desarrollar componentes reutilizables, facilita la gestión del estado de la aplicación y ofrece una integración natural con las APIs necesarias para construir una PWA. Su popularidad, documentación extensa y curva de aprendizaje razonable lo convierten en la opción más adecuada para un desarrollo individual y con plazos acotados.

Para el manejo del entorno gráfico se utiliza Three.js[20] mediante React Three Fiber (R3F)[21]. La elección de esta librería frente a motores completos como Unity[22] radica en su ligereza; permite renderizar la anatomía 3D y proyectar los tatuajes utilizando directamente la aceleración por hardware (WebGL)[23] del navegador, además de que se integra de manera nativa con React, lo que facilita la gestión del estado y la interacción entre la interfaz de usuario y el entorno 3D. Lo que permite mantener una experiencia fluida y responsiva en una PWA, evitando la sobrecarga que implicaría un motor gráfico completo, y permitiendo un desarrollo más ágil sin obligar al usuario a instalar complementos externos.

2.3.2. Backend

Para el desarrollo del backend se utiliza Node.js [24] con Express [25] como framework para construir la API RESTful[26]. Node.js es una plataforma de JavaScript que permite ejecutar código en el servidor, lo que facilita la creación de aplicaciones web escalables. Express es un framework flexible que proporciona un conjunto robusto de características para desarrollar aplicaciones web y APIs, lo que lo convierte en una opción ideal para manejar las operaciones del backend de manera sencilla. Se justifica la elección de esta tecnología por su compatibilidad con JavaScript, lo que permite un desarrollo full-stack con un solo lenguaje, y su amplia comunidad de soporte, lo que facilita la resolución de problemas y la integración de funcionalidades adicionales.

2.3.3. Sistema gestor de base de datos (SGBD)

Para este proyecto se opta por un sistema gestor de base de datos relacional en lenguaje SQL[27], dado que el sistema debe soportar con precisión tanto operaciones gráficas tridimensionales como tareas administrativas transacionales de inventario y agenda. Tras un análisis comparativo frente a alternativas tradicionales como MySQL[28] o MariaDB[29], se selecciono PostgreSQL[30] debido a su cumplimiento de las propiedades ACID (MVCC)[31]. Esta característica garantiza que los procesos de lectura no bloqueen a los de escritura, resolviendo cualquier colisión de datos en caso de que ocurran solicitudes simultáneas para reservar un mismo bloque horario o para ir descontando insumos del inventario.

Tomando en consideración la utilización de que se trabaja con modelado 3D, es necesario persistir la configuración exacta de cómo un boceto 2D se proyecta sobre el modelo antatómico. Guardar variabes individuales como (X,Y,Z), rotación y escala para cada sesión generaría esquemas de tablas rígidos y costosos de consultar. PostgreSQL resuelve esto mediante un soporte nativo de datos JSONB (JSON Binario)[32]. JSONB descomopone el documento JSON[33] en un formato binario indexado lo que permite almacenar estructuras de datos dinámmicas, anidadas y flexibles dentro de una fila relacional. De este modo, las coordenadas espaciales calculadas por Three.js se guardan en un solo campo optimizado. Al estar indexado binariamente, el backend puede consultar, filtrar y descomponer los datos espaciales a gran velocidad, logrando que la reconstrucción del modelo 3D en el navegador sea

prácticamente instantánea cada vez que el tatuador reabra el perfil de un cliente.

2.3.4. Servicio de mensajería

Para la integración de recordatorios personalizados, se utiliza la API de WhatsApp. Esta API permite enviar mensajes automatizados a los clientes a través de WhatsApp, lo que es esencial para reducir el ausentismo y mejorar la comunicación con los clientes. La elección de esta API se justifica por la amplia adopción de WhatsApp como plataforma de mensajería, su capacidad para enviar mensajes programados y personalizados, y su compatibilidad con la arquitectura del proyecto, lo que facilita su integración con el backend desarrollado en Node.js. El envío automático se resuelve con una biblioteca que vincula el WhatsApp del propio tatuador al servidor como un dispositivo más[34], de modo que los recordatorios salen desde su número; la vía oficial de Meta queda disponible para el estudio que la tenga contratada.

2.3.5. Despliegue de aplicaciones

Como herramienta de despliegue de aplicación se utilizará Docker[35], que es una plataforma de contenedorización que permite empaquetar la aplicación y sus dependencias en un contenedor portátil. Esto facilita el despliegue en diferentes entornos sin preocuparse por las inconsistencias de configuración. Docker también simplifica la gestión de versiones y la escalabilidad, lo que es beneficioso para el desarrollo iterativo y la implementación continua del proyecto.

2.3.6. Herramienta para repositorio GithHub

Para el control de versiones y el respaldo del código fuente, se utiliza GitHub[36]. La gestión de las ramas de desarrollo y los “commits”, se realiza mediante interfaces gráficas como GitKraken[37] o GitHub Desktop[38]. Esta elección agiliza la visualización del historial del proyecto sin depender estrictamente de la interfaz de línea de comandos, minimizando errores al revertir cambios durante la fase de refactorización de PXP.

2.3.7. Sistema gestor de proyectos

Para administrar la planificación iterativa y el seguimiento de las Historias de Usuario, se emplea Trello[39]. A diferencia de herramientas más complejas como

Jira[40] orientadas a grandes equipos corporativos, Trello ofrece un sistema de tableros Kanban[41] intuitivo y directo. Esta simplicidad se alinea a la perfección con las prácticas ágiles de un desarrollador en solitario (PXP), permitiendo gestionar las listas de comprobación (checklists) de las pruebas de software de manera visual y eficiente.

2.4. Estado del arte

En cuanto al estado del arte del proyecto, existen distintos tipos de sistemas digitalizados como herramientas para los tatuadores, tanto de código libre como de pago, a continuación, se mencionan y describen algunos sistemas reconocidos y que están en relación con el proyecto a desarrollar:

- **REV23 Desktop[42]:** Es un software de gestión y punto de venta diseñado, creado de manera exclusiva para satisfacer las necesidades administrativas de los estudios de tatuajes y perforaciones. Este programa centraliza todas las operaciones del negocio en un solo lugar, permitiendo organizar calendarios de citas, automatizar recordatorios, procesar pagos y calcular las comisiones de los artistas.

- **Procreate, Clip Studio Paint, Adobe Substance 3D:** Son herramientas de creación visual digital que integran modelos tridimensionales dentro del flujo de trabajo del arte digital, permitiendo a los artistas trascender el lienzo plano tradicional. Todas ofrecen la capacidad de cargar o utilizar mallas 3D (como figuras humanas, objetos o partes específicas del cuerpo) para facilitar el diseño, ya sea interactuando con estos modelos para establecer referencias espaciales y anatómicas precisas, o pintando directamente sobre su superficie para previsualizar cómo se comportan los trazos y diseños sobre volúmenes reales. Las diferencias más notables son sus diferentes enfoques, debido a que si bien las tres aplicaciones permiten dibujar en los modelos, la facilidad no es la misma, ya que Procreate[43] es la más apta para ello, porque permite importar un modelo 3D anatómico y poder usar los pinceles 2D para pintar directamente sobre la superficie. Clip Studio Paint[44] se centraliza mucho más en las referencias anatómicas de los modelos, no de texturizado, esto quiere decir que se enfoca principalmente en modificar su complexión y articularlos

en cualquier pose. Por último Adobe Substance 3D[45] se enfoca principalmente en texturizar modelos 3D para videojuegos, cine o animación. A diferencia de Procreate aquí no se "pinta" como en un boceto sino que se ocupan otro tipos de herramientas más complejas para el diseño.

- **Fresha, Calendly, Calendario iOS:** Son softwares enfocados en la gestión del tiempo y organización de la agenda. Cumplen con el propósito de registrar fechas, eventos, reuniones o servicios, y ayudar a sus usuarios a visualizar su disponibilidad en sus horarios. Dentro de las diferencias que poseen es que Fresha[46] es una plataforma de gestión empresarial y punto de venta diseñada específicamente para la industria de la belleza y el bienestar, permite reservas de citas online, gestión de inventario y proceso de pagos. Calendly[47] por su parte es una herramienta de automatización de agendamiento enfocada en profesionales, autónomos y equipos corporativos. Es calendario que funciona como un intermediario inteligente que se configura con reglas de disponibilidad y se comparte un enlace público donde las demás personas visualizan tus horarios libres. Por último el calendarios de iOS[48] es una aplición de uso estrictamente personal, está diseñada para que el usuario organice su propia vida, anotando eventos, recordatorios o citas.

- **Tatto Studio Pro, Ink Studio AI:** Son herramientas digitales creadas exclusivamente para modernizar y optimizar la industria del tatuaje, las dos plataformas poseen la funcionalidad de la cotización de los tatuajes, pero tienen ciertas diferencias en el proceso de hacerlo. Tatto Studio Pro[49] realiza un cálculo administrativo cerrado utilizado por el estudio o el artista. lo que hace es multiplicar la tarifa por las horas de la sesión y por el costo de los insumos en donde ambas variables son ingresadas manualmente. Ink Studio AI[50] realiza una estimación algorítmica, su cálculo de precio evalúa las características visuales del diseño como "estilo full a color" o "tamaño de antebrazo" y con ello da una estimación final del tatuaje.

- **Instagram, Google Photos, Behance:** Las tres aplicaciones son repositorios visuales en la nube donde un artista puede alojar, organizar y acceder a sus galerías de bocetos desde cualquier dispositivo. Instagram[51] es una galería pública en donde el objetivo es generar la mayor interacción en likes y compar-

tidos en tus publicaciones, donde en este caso los bocetos publicados servirian como publicidad para que el usuario envie un mensaje directo y reserve una cita. Google Photos[52] es una galería utilitaria y personal, su objetivo solo es ser un respaldo en la nube y la organización admnistrativa de los bocetos. Behance[53] es una galaría de exhibición estática y de alto perfil. Los bocetos se presentan como "proyectos" y permite mostrar flujos de trabajo completos.

Tabla 2.1: Análisis comparativo de funcionalidades por software

Herramienta / Software	Previsualización 3D	Galería	Gestor de Inventario	Gestión de Citas	Cotización	Recordatorios
REV23 Desktop	×	×	✓	✓	✓ (Comisiones)	✓
Procreate / HiPasint / Affinity	✓	✓	×	×	×	×
Fresha	×	×	✓	✓	×	×
Calendly / Calendario iOS	×	×	×	✓	×	✓
Tattoo Studio Pro	×	×	✓	×	✓ (Manual)	×
Ink Studio AI	×	×	×	×	✓ (Algorítmica)	×
Instagram / Google Photos / Behance	×	✓	×	×	×	×
Mi Sistema	✓	✓	✓	✓	✓ (Automatizada)	✓

El trabajo relacionado demuestra que existen múltiples herramientas que cumplen con los módulos propuestos para el sistema. Sin embargo se deja en evidencia el problema fundamental, que es la división de trabajo del tatuador ya que no hay una herramienta o un sistema que poseea una mayor integración de módulos en un sólo lugar. El único de los softwares que se acerca a ser un espacio completo para el flujo del tatuador sería Rev23 Desktop que tiene como módulos el gestor de inventario, gestión de citas, cotizaciones y recordatorios. Aún así carece de la previsualización 3D y galería como módulos para ser un sistema completo según lo propuesto.

Las demás herramientas mencionadas en el Tabla 2.1 no cumplen con ser un software integral para el trabajo de un tatuador, ya que se limitarían a tener uno o dos módulos como funciones en su programa.

En resumen, no existe actualmente un software o herramienta unificado que considere todas las funcionalidades propuestas para ser considerado un sistema que optimice el flujo del trabajo del tatuador.

2.5. Metodologías

En este capítulo se presentan las metodologías de desarrollo y de evaluación para el proyecto.

2.5.1. Metodologías de desarrollo

El proceso de implementación de la plataforma web involucra sólo un desarrollador, por lo que se analizan las metodologías de desarrollo de software más adecuadas para este contexto, destacando dos opciones principales: Personal Software Process (PSP)[54] y Personal Extreme Programming (PXP)[55]. Ambas metodologías comparten fases clave como la planificación, diseño, implementación y pruebas. Sin embargo, PSP se enfoca en una planificación estricta y alta documentación, lo cual le resta agilidad al proyecto.

Por el contrario, PXP reduce la sobrecarga de documentación y adopta un subconjunto de prácticas ágiles de Extreme Programming (XP)[56] adaptadas para un solo desarrollador. Para este proyecto en particular, el desarrollo del módulo de previsualización 3D y el algoritmo de cotización requiere de retroalimentación temprana para asegurar su usabilidad práctica. Al basarse en iteraciones cortas, PXP proporciona la flexibilidad necesaria para construir el software de forma incremental, permitiendo que los usuarios finales (los artistas del tatuaje) interactúen con prototipos funcionales y validen que el sistema realmente se adapta a su flujo de trabajo antes de avanzar a la siguiente fase. Por lo tanto, se ha optado por PXP como la metodología más adecuada, garantizando agilidad, calidad de código y satisfacción del cliente.

Para el desarrollo de la propuesta, se siguen las distintas fases que contempla esta metodología, adaptadas a las necesidades del proyecto:

- **Requerimientos:** En esta etapa se capturan los requisitos del proyecto mediante reuniones e interacción directa con un artista del tatuaje que asume el rol de cliente experto, traduciendo sus necesidades operativas en historias de usuario.
- **Planificación:** Esta fase comprende la estimación de las historias de usuario, la elección de las tecnologías a utilizar (como librerías gráficas para el motor 3D

y gestores de bases de datos relacionales), el diseño de interfaces y la definición de la arquitectura web.

- **Iteración:** Se define el inicio de cada ciclo de desarrollo, el cual establece una duración estimada de entre 1 y 3 semanas, enfocándose en entregar un módulo funcional (ej. Gestor de inventario o Integración de mensajería).
- **Diseño:** Comprende el diseño lógico de la base de datos, los módulos del backend y las funciones del frontend correspondientes a las historias de la iteración en curso.
- **Implementación:** Es la fase de codificación pura de las tareas. Para dar por finalizada esta etapa, el código debe compilar sin errores y superar las pruebas unitarias; de lo contrario, se debe refactorizar aplicando los principios de simplicidad de PXP.
- **Sistema de Pruebas y Validación:** Comprende el testeo integral para asegurar que las funcionalidades cumplen con los requisitos iniciales. Es en este punto donde el incremento del software es presentado al cliente (tatuador) para validar empíricamente que la solución tecnológica resuelve el problema planteado.
- **Retrospectiva:** Marca el final de cada iteración. Se realiza un análisis del desempeño del desarrollo, se integran las recomendaciones o ajustes detectados por los usuarios, y se planifica el inicio de la siguiente iteración o el cierre definitivo del proyecto.

La Figura 2.1 muestra una representación de las fases que posee la metodología de desarrollo Personal Extreme Programming (PXP):

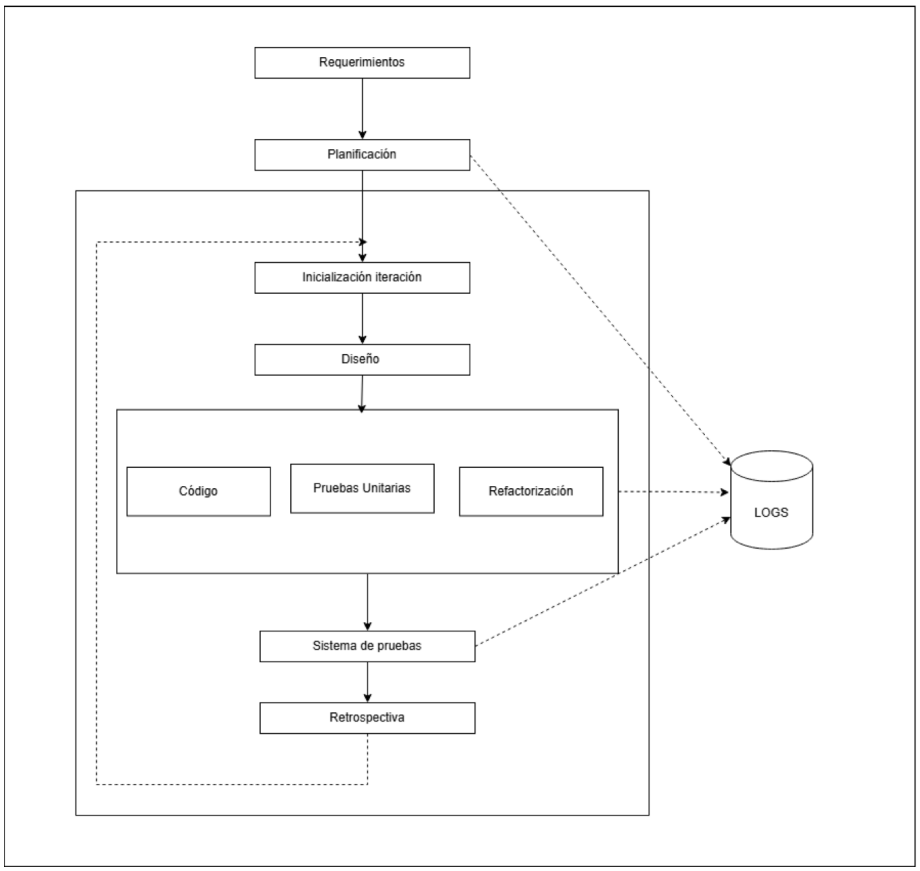


Figura 2.1: Fases de metodología Personal Extreme Programming (PXP).

2.5.2. Metodologías de evaluación del proyecto

En esta sección se describen las metodologías de evaluación que se implementarán para validar la calidad, usabilidad y efectividad del sistema desarrollado.

Evaluación de Calidad

Para asegurar que la plataforma esté libre de fallos críticos antes de su despliegue, se implementa una estrategia de pruebas a nivel de código y de sistema:

- **Pruebas de Caja Blanca (Pruebas Unitarias y de Integración):** Estas pruebas se enfocan en verificar la lógica interna del código fuente, evaluando que los algoritmos críticos (como el motor de cálculo de cotizaciones y la lógica de proyección 3D) funcionen correctamente desde su interior. Se definen tests con el objetivo de alcanzar una cobertura de código de al menos un 95 %.

Para dar un ejemplo de las pruebas unitarias se considera realizar pruebas para funciones del código del programa, por ejemplo en el área de cotizaciones se ingresan valores a los insumos para verificar que los datos finales concuerden con los cálculos esperados en todo momento. A continuación, la Figura 2.2 ilustra el proceso de la prueba.

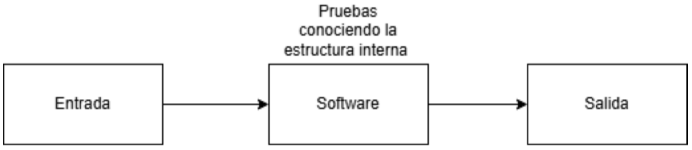


Figura 2.2: Proceso de Prueba de Caja Blanca.

- **Pruebas de Caja Negra:** Corresponden a una forma de evaluar y validar el funcionamiento de los módulos del sistema sin necesidad de conocer su proceso interno. En estas pruebas, se definen entradas específicas (por ejemplo, subir un boceto y asignar horas) y se comparan con las salidas esperadas en la interfaz de usuario, garantizando que los módulos operen correctamente de extremo a extremo. A continuación, la Figura 2.3 ilustra el proceso de la prueba.

Figura 2.3: Proceso de Prueba de Caja Negra.

Evaluación de Usabilidad y Aceptación

Dado que el sistema está enfocado en usuarios finales (tatuadores y clientes del estudio), es fundamental validar que la interfaz sea intuitiva y el flujo de trabajo sea el adecuado.

- **System Usability Scale (SUS)[57]:** Para evaluar la usabilidad empírica del sistema se considera aplicar el cuestionario estándar SUS. Este instrumento consiste en 10 afirmaciones evaluadas mediante una escala de Likert (desde "Totalmente en desacuerdo" hasta "Totalmente de acuerdo"). A partir de las

respuestas, se calcula un puntaje global. Para este proyecto, se ha establecido como métrica de éxito alcanzar un puntaje mínimo de 85 puntos, lo que clasificaría a la plataforma dentro de un rango de excelencia en términos de usabilidad.

- **Pruebas de Aceptación del Usuario (UAT)[58]:** Se llevarán a cabo sesiones de prueba controladas donde el cliente experto y otros tatuadores invitados interactuan libremente con la versión final del sistema. Al finalizar, se registra su nivel de satisfacción, teniendo como meta alcanzar al menos un 90 % de evaluación positiva respecto a la funcionalidad de la plataforma.

Evaluación de Validación

Para complementar las métricas de software y validar el impacto del sistema en el negocio, se considera un enfoque comparativo (Pre-test y Post-test) mediante entrevistas y análisis de registros:

- **Entrevistas:** Se realiza entrevistas al inicio del proyecto para cuantificar las bases actuales del artista experto respecto a el tiempo que invierten en ajustar bocetos en papel, el tiempo que dedican a calcular costos manualmente, y su tasa de inasistencia semanal promedio.

- **Contraste de Resultados:** Una vez implementado el MVP y tras un periodo de prueba real, se vuelven a medir estos parámetros operando con el sistema. El objetivo es validar estadísticamente el logro en la reducción de los tiempos de previsualización en un 30 % los tiempos de previsualización (módulo 3D), en un 40 % los tiempos de cotización (módulo de inventario) y en un 50 % el ausentismo (módulo de agendamiento con WhatsApp).

2.6. Resumen capítulo

En este capítulo se explicaron los conceptos básicos necesarios para entender el proyecto, se describieron las tecnologías a utilizar, evaluando alternativas y justificando la elección de cada una. También se revisó el estado del arte, presentando aplicaciones que ofrecen funcionalidades similares a la solución propuesta. Finalmen-

te, detallaron las metodologías de desarrollo y evaluación, justificando la elección de cada una.

En el siguiente capítulo se abordará la aplicación de estas metodologías, junto con las adaptiones necesarias para el proyecto y los resultados esperados de los objetivos específicos.

3. Metodologías

El presente capítulo está orientado a describir la manera en que se aplicará las metodologías propuestas en el capítulo anterior y los resultados esperados de los objeticos específicos.

3.1. Personal Extreme Programming (PXP)

Como se mencionó en el Capítulo 2.4.1, la metodología PXP adopta las mejores prácticas de las metodologías Personal Software Process (PSP) y Extreme Programming (XP). Se basa en un desarrollo personal pero a la vez sigue manteniendo las fases claves como Requerimientos, Planififcación e Iteración.

3.1.1. Requerimientos

Durante esta fase, se realiza una entrevista entre el desarrollador y el cliente para capturar los requerimientos iniciales del sistema. Estos requerimientos se transforman en la fase siguiente en historias de usuario, lo que permite enfocar el desarrollo en la experiencia del usuario.

3.1.2. Planificación

Durante esta fase, la actividad central consiste en la creación, priorización y organización de las historias de usuario (HU), las cuales se dividen y asignan a iteraciones específicas. En esta fase también se toman las decisiones de diseño de la aplicación, tales como las configuraciones base de las tecnologías a utilizar, arquitecturas de software y mockups, que permitirán el flujo de las distitnas vistas que tendrá la aplicación y su interacción con los usuarios.

Historias de usuario

A continuación, la Tabla 3.1 ilustra las historias de usuario obtenidas a partir de la entrevista inicial con el cliente.

Tabla 3.1: Historias de Usuario

HU	**Actor(es)**	**Descripción**
HU01	Tatuador	quiero registrarme en la plataforma ingresando mis datos, para tener un espacio de trabajo integral.
HU02	Tatuador	quiero iniciar y cerrar sesión mediante mis credenciales, para proteger la privacidad de mi negocio.
HU03	Tatuador	quiero recuperar mi contraseña en caso de olvido, para no perder el acceso a mi agenda e inventario.
HU04	Tatuador	quiero importar imágenes a la plataforma, para almacenarlas en la nube.
HU05	Tatuador	quiero ver todos mis diseños en una galería, para acceder a ellas de manera expedita.
HU06	Tatuador	quiero eliminar un boceto antiguo de la galería, para mantener mi espacio ordenado y liberar almacenamiento.
HU07	Tatuador	quiero añadir etiquetas a mis bocetos, para filtrarlos y encontrarlos ágilmente frente al cliente.
HU08	Tatuador	quiero elegir entre distintos modelos anatómicos 3D ya integrados, para utilizarlos como lienzo de previsualización.
HU09	Tatuador	quiero proyectar un boceto 2D (PNG) sobre el modelo 3D, para evaluar cómo se adapta a la curvatura de la piel.
HU10	Tatuador	quiero escalar, mover y rotar el diseño sobre el modelo 3D, para definir la posición exacta.
HU11	Tatuador	quiero registrar un material con su cantidad y costo unitario, para mantener un control inicial de mi stock.

Continúa en la siguiente página...

Tabla 3.1 – Continuación de la página anterior

HU	Actor(es)	Descripción
HU12	Tatuador	quiero modificar el stock manual o borrar insumos descontinuados, para mantener la realidad operativa de mi estudio.
HU13	Tatuador	quiero visualizar una advertencia cuando un insumo llegue a un nivel crítico, para comprar reposiciones a tiempo.
HU14	Tatuador	quiero que el sistema calcule el costo de los insumos basándose en el tamaño del boceto y variables de trazado, para obtener un presupuesto automatizado.
HU15	Tatuador	quiero agregar o quitar materiales específicos sugeridos por el algoritmo, para refinar el cálculo final de la cotización.
HU16	Tatuador	quiero que al dar por finalizada una sesión cotizada los materiales se descuenten solos, para automatizar el control de stock.
HU17	Tatuador	quiero agendar a un cliente especificando fecha, hora, número de contacto y precio del proyecto, para organizar mis tiempos.
HU18	Tatuador	quiero ver mis citas en un formato de calendario mensual y semanal, para organizar visualmente mi jornada.
HU19	Tatuador	quiero poder crear distintos proyectos para clientes específicos junto con sus sesiones.
HU20	Tatuador	quiero poder subir una foto de cada sesión para ver los progresos del proyecto.
HU21	Tatuador	quiero cambiar la fecha de una cita o cancelarla, para mantener la agenda actualizada frente a imprevistos.
HU22	Tatuador	quiero que el sistema envíe automáticamente un mensaje de WhatsApp 24 horas antes de la sesión, para evitar olvidos o retrasos.

Continúa en la siguiente página...

Tabla 3.1 – Continuación de la página anterior

HU	Actor(es)	Descripción
HU23	Tatuador	quiero que al cancelar o cambiar una hora el sistema avise al cliente por WhatsApp, para mantener una comunicación profesional.
HU24	Tatuador	quiero ver el historial de todas las sesiones pasadas y futuras asociadas a un cliente, para hacer seguimiento de piezas grandes.
HU25	Tatuador	quiero ver un panel de control al iniciar sesión con las citas programadas y alertas de stock bajo, para organizar mi jornada de un vistazo.
HU26	Tatuador	quiero visualizar una suma básica de los ingresos generados en el mes actual, para evaluar la rentabilidad del estudio.
HU27	Tatuador	quiero definir mis días y horas de trabajo, para que el calendario bloquee automáticamente mis días libres.
HU28	Tatuador	quiero poder editar el texto base del mensaje que se envía por WhatsApp, para darle recomendaciones del trato del tatuajes posteriormente al trabajo realizado.
HU29	Tatuador	quiero tener el resgistro de todos mis clientes atendidos con su información.
HU30	Tatuador	quiero poder visualizar un marcardor de todos mis clientes y asignarles puntaje para saber su responsabilidad de llegada y poder filtrarlos de acuerdo a ello.
HU31	Administrador	quiero registrar nuevos estudios de tatuaje en la plataforma indicando nombre, correo y plan contratado, para incorporar a nuevos clientes del sistema.
HU32	Administrador	quiero activar o desactivar una cuenta de estudio, para gestionar el acceso a la plataforma ante impagos o solicitudes de baja.

Continúa en la siguiente página...

Tabla 3.1 – Continuación de la página anterior

HU	Actor(es)	Descripción
HU33	Administrador	quiero agregar tatuadores adicionales dentro de un estudio existente, para dar soporte a estudios con más de un profesional.
HU34	Administrador	quiero eliminar o desactivar la cuenta de un tatuador específico dentro de un estudio, para mantener actualizada la nómina de usuarios activos.
HU35	Administrador	quiero visualizar un listado de todos los estudios y tatuadores registrados, para monitorear el uso general de la plataforma.
HU36	Administrador	quiero ver métricas globales de uso por estudio (número de sesiones, citas agendadas, bocetos almacenados), para evaluar la adopción de la plataforma.
HU37	Administrador	quiero gestionar los modelos anatómicos 3D disponibles en la plataforma (agregar, actualizar o eliminar), para mantener actualizado el catálogo compartido entre estudios.
HU38	Administrador	quiero restablecer la contraseña de cualquier usuario registrado, para brindar soporte ante bloqueos de acceso.

Priorización y estimación de historias de usuario

En PXP, luego de realizar la creación de historias de usuario, se deben priorizar y estimar en base al esfuerzo que toma implementar cada una. Para priorizar se utiliza una escala que va desde el valor más bajo correspondiente a “Baja” hasta “Crítica” siendo el valor más alto, categorizándolas en base a su impacto sobre los objetivos específicos del proyecto.

Por el lado de la estimación, si bien existen distintos métodos, se utilizará Planning Poker[59], el cual se elige debido a que emplea la escala de la serie de Fibonacci[60] para estimar cada historia, en el que entre más bajo sea el número otorgado, representará un menor esfuerzo técnico y cognitivo. A continuación, la Tabla 3.2 ilustra el resultado de los métodos mencionados aplicados al catálogo del sistema.

Tabla 3.2: Priorización y estimación de esfuerzo para cada historia de usuario

HU	**Priorización**	**Estimación de esfuerzo**
HU01	Crítica	3
HU02	Crítica	2
HU03	Media	3
HU04	Crítica	3
HU05	Alta	3
HU06	Baja	2
HU07	Media	3
HU08	Crítica	5
HU09	Crítica	8
HU10	Crítica	8
HU11	Alta	3
HU12	Media	2
HU13	Baja	2
HU14	Crítica	5
HU15	Media	3
HU16	Alta	5
HU17	Crítica	5
HU18	Alta	3
HU19	Media	3
HU20	Alta	3
HU21	Alta	5
HU22	Crítica	5
HU23	Alta	3
HU24	Media	3
HU25	Media	3
HU26	Baja	2
HU27	Media	2
HU28	Baja	2
HU29	Media	3

Continúa en la siguiente página...

Tabla 3.2 – Continuación de la página anterior

HU	Priorización	Estimación de esfuerzo
HU30	Media	3
HU31	Crítica	3
HU32	Crítica	2
HU33	Alta	3
HU34	Media	2
HU35	Alta	3
HU36	Media	5
HU37	Media	3
HU38	Baja	2

Planificación de historias de usuario

A continuación, la Tabla 3.3 ilustra las iteraciones y las historias de usuario asociadas a cada una de ellas, distribuidas estratégicamente a lo largo de la fase de implementación del proyecto.

Tabla 3.3: Asignación de historias de usuario a iteraciones

Iteración	Historias de usuario
2	HU01, HU02, HU03
3	HU04, HU05, HU06, HU07
4	HU11, HU12, HU13, HU25, HU26
5	HU14, HU15, HU16, HU27
6	HU17, HU18, HU19, HU20, HU24, HU30
7	HU21, HU22, HU23, HU28, HU29, HU31
8	HU08, HU09, HU10
9	HU32, HU33, HU34, HU38
10	HU35, HU36, HU37

3.1.3. Iteración

La fase de iteración es la de mayor duración en la metodología, y está dividida en varias etapas: Inicio de Iteración, Diseño, Implementación, Sistema de Pruebas y

Retrospectiva. A continuación, se describe la implementación de cada una de estas etapas.

Inicio de iteración

En esta etapa, se crean las tareas específicas para cada historia de usuario correspondiente a la iteración actual. Para llevar un control de la tareas y su estado de avance, se utiliza Trello. A continuación, la Figura 3.1 muestra un ejemplo de cómo se organizan las tareas en el tablero de Trello, con columnas que representan los distintos estados del desarrollo (Por hacer, En progreso, En revisión, Completado) y tarjetas que detallan cada tarea específica asociada a las historias de usuario asignadas a la iteración.

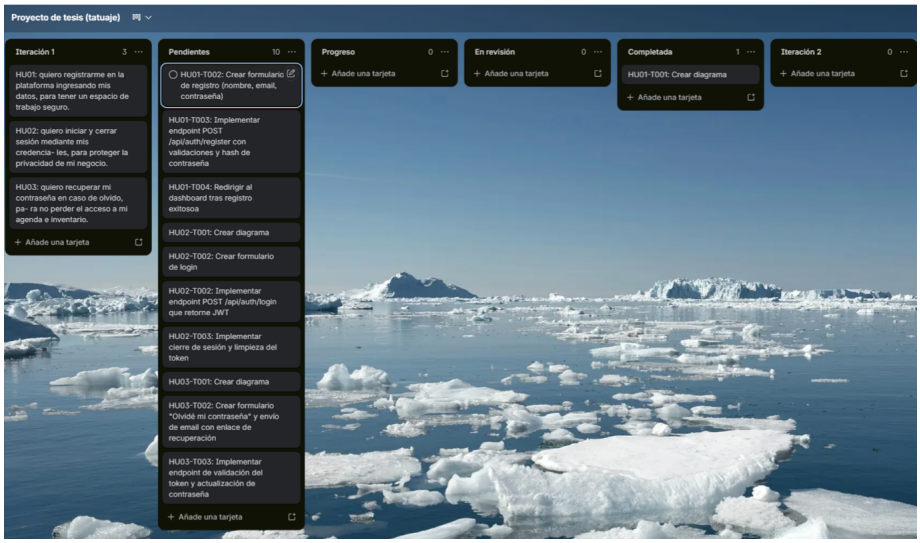


Figura 3.1: Extracto de la tabla dee registro de tareas por iteración.

Diseño

En esta etapa, se crean los diagramas de sencuencia de las historias de usuario asociadas a la iteración. A continuación, la Figura 3.2 representa un ejemplo de diagrama de secuencia para el inicio de sesión.

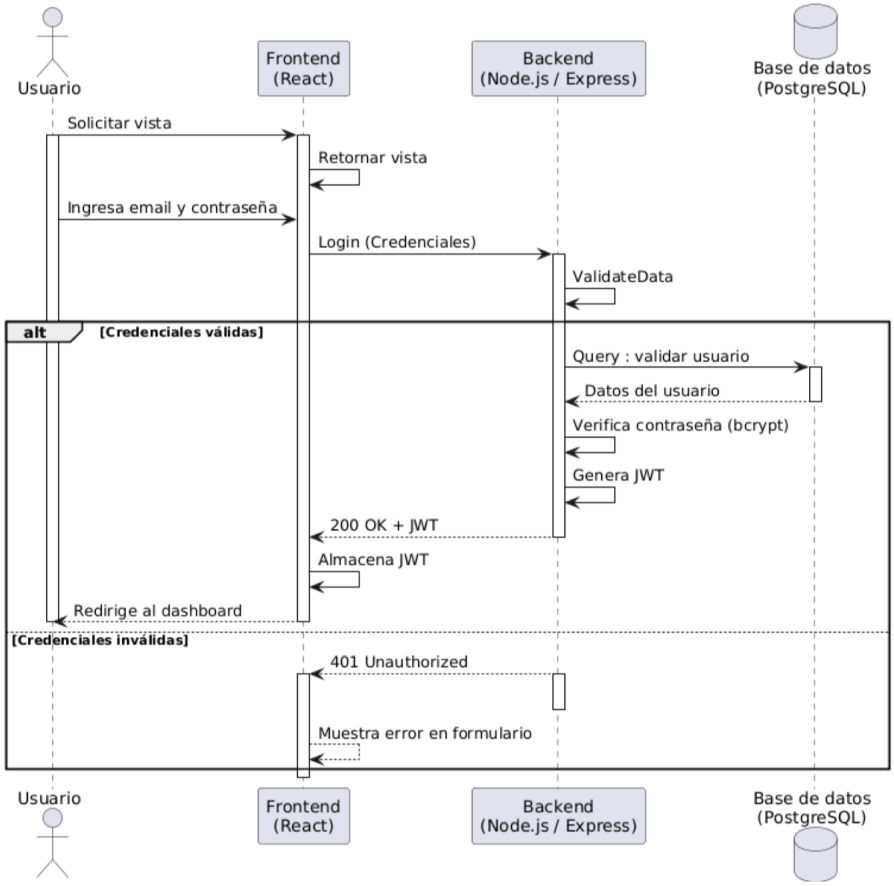


Figura 3.2: Ejemplo de diagrama de secuencia para el inicio de sesión.

Las tareas T0001 de cada iteración corresponden a estos diagramas. Para no extender la memoria con flujos que repiten el mismo recorrido —vista, interfaz de programación, base de datos y respuesta—, en el capítulo de Desarrollo se reproduce el diagrama más representativo de cada iteración.

Implementación

En esta etapa se lleva a cabo el desarrollo de código, pruebas unitarias y refactorización. La implementación sigue los diagramas de secuencia para guiar el flujo de las funcionalidades y los mockups propuestos. Aunque la metodología PXP sugiere un enfoque de Test-Driven Development (TDD)[61], en el que primero se crean las pruebas, luego el código, y finalmente se realiza la refactorización este caso se ajustaría el proceso: primero se desarrollará el código, luego se completarán las pruebas

unitarias, y se refactorizará en caso de que alguna prueba falle.

Sistema de pruebas

En esta etapa se crean y ejecutan las pruebas correspondientes a todas las tareas definidas en el Inicio de Iteración. Si alguna de las pruebas falla, se regresa a la etapa de Implementación para realizar la refactorización necesaria y asegurar que el código cumpla con los requisitos definidos en las historias de usuario. Para llevar el control de las pruebas se utiliza una lista de verifiación en el mismo Trello a cada tarea, en donde se marcan las pruebas que han sido superadas y las que aún están pendientes o han fallado.

A continuación la Figura 3.3 muestra un ejemplo de una lista de verificación para una iteración en particular.

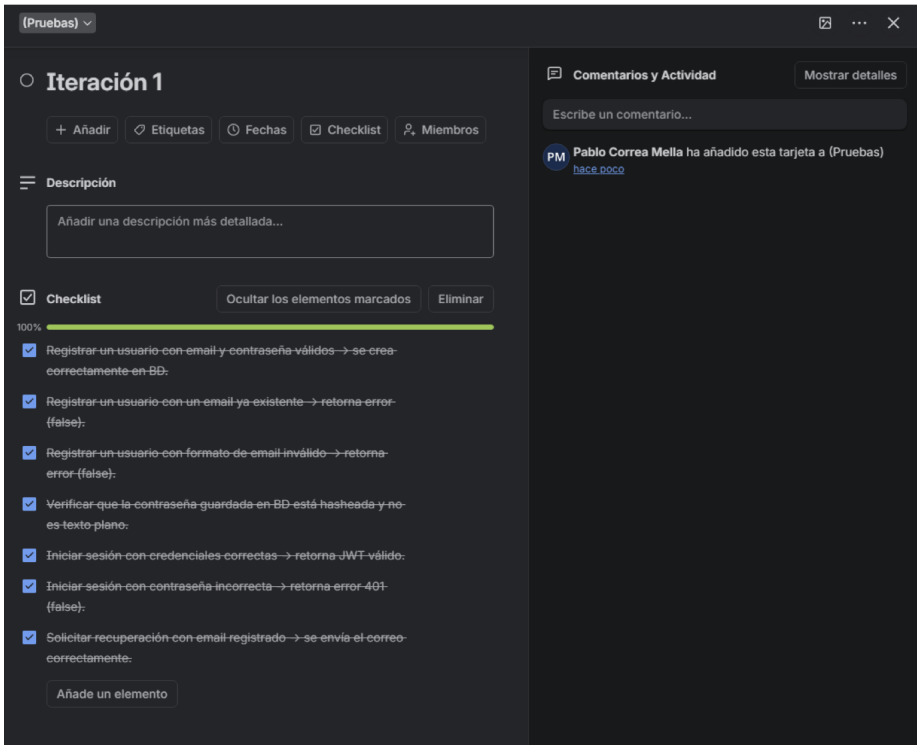


Figura 3.3: Ejemplo de lista de verificación de pruebas para una iteración.

Retrospectiva

En esta etapa se realiza un análisis del desempeño del desarrollo, se integran las recomendaciones o ajustes detectados por el usuario experto, y se planifica el inicio

de la siguiente iteración o el cierre definitivo del proyecto. Para llevar un control de las recomendaciones y ajustes, se utiliza una sección específica en Trello donde se documentan los comentarios recibidos durante la fase de pruebas, así como las acciones a tomar para mejorar el sistema en la siguiente iteración.

3.2. Evaluación

Como se mencionó en el Capítulo 2.5.2 para eevaluar el cumplimiento el objetivo general y los objetivos específicos, se implementarán distintas metodologías de evaluación, las cuales se describen a continuación.

3.2.1. Prueba de caja negra

Las pruebas de caja negra se enfocan en evaluar el funcionamiento de los módulos del sistema sin necesidad de conocer su proceso interno. En estas pruebas, se definen entradas específicas (por ejemplo, subir un boceto y asignar horas) y se comparan con las salidas esperadas en la interfaz de usuario, garantizando que los módulos operen correctamente de extremo a extremo. Para llevar un control de las pruebas de caja negra, se registran en una hoja de cálculo donde se documentan los casos de prueba definidos, las entradas utilizadas, las salidas esperadas y los resultados obtenidos, permitiendo así un seguimiento detallado del proceso de validación. A continuación, la Figura 3.4 muestra un ejemplo de cómo se registra una prueba de caja negra en la hoja de cálculo.

Registro de Pruebas de Caja Negra – Plataforma Web Estudio de Tatuaje							
ID	Módulo (HU)	Descripción	Entradas	Salidas	Resultado esperado	Resultado obtenido	Estado
AUTENTICACIÓN							
1	HU01	Registro de usuario	Nombre, correo y contraseña válidos	Usuario creado exitosamente	Redirigir al dashboard		
2	HU01	Registro con correo duplicado	Correo ya registrado en el sistema	Mensaje de error visible	Mostrar 'el correo ya está en uso'		
3	HU01	Registro con contraseña débil	Contraseña de menos de 6 caracteres	Mensaje de validación	Mostrar 'contraseña muy corta'		
4	HU02	Inicio de sesión exitoso	Correo y contraseña correctos	Sesión iniciada	Acceso al panel principal		
5	HU02	Inicio de sesión fallido	Contraseña incorrecta	Mensaje de error	Mostrar 'credenciales inválidas'		
6	HU02	Cierre de sesión	Usuario autenticado presiona 'Cerrar sesión'	Sesión finalizada	Redirigir a pantalla de login		
7	HU03	Recuperación de contraseña	Correo registrado ingresado en formulario	Correo de recuperación enviado	Mostrar 'revisa tu correo'		
8	HU03	Recuperación con correo inexistente	Correo no registrado en el sistema	Mensaje de error	Mostrar 'correo no encontrado'		

Figura 3.4: Ejemplo de registro de prueba de caja negra en una hoja de cálculo.

3.2.2. Prueba de caja blanca

Las pruebas de caja blanca se enfocan en verificar la lógica interna del código fuente, evaluando que los algoritmos críticos (como el motor de cálculo de cotizaciones y la lógica de proyección 3D) funcionen correctamente desde su interior. Se definen tests con el objetivo de alcanzar una cobertura de código de al menos un 95 %. Para llevar un control de las pruebas de caja blanca, se utilizan herramientas de análisis de cobertura que generan reportes detallados sobre qué partes del código han sido ejecutadas durante las pruebas, permitiendo identificar áreas no cubiertas y asegurar que se cumpla con el objetivo establecido.

3.2.3. Prueba de usabilidad

La prueba de usabilidad SUS permite que el usuario final evalúe la aplicación mediante un cuestionario de 10 preguntas, que serán respondidas en una escala de Likert, con el objetivo de obtener una puntuación global que refleje la usabilidad del sistema. Para este proyecto, se ha establecido como métrica de éxito alcanzar un puntaje mínimo de 85 puntos, lo que indicaría que la plataforma es altamente usable y cumple con las expectativas del cliente experto. A continuación, se detallan las preguntas:

1. Creo que me gustaría usar esta plataforma con frecuencia.

2. Encontré la plataforma innecesariamente compleja.

3. Creo que la plataforma es fácil de usar.

4. Creo que necesitaría la ayuda de una persona con conocimientos técnicos para usar esta plataforma.

5. Creo que las funciones de esta plataforma están bien integradas.

6. Creo que hay demasiada inconsistencia en esta plataforma.

7. Imagino que la mayoría de las personas aprenderían a usar esta plataforma muy rápidamente.

8. Creo que esta plataforma es muy difícil de usar.

9. Creo que me sentiría muy seguro usando esta plataforma.

10. Necesitaría aprender muchas cosas antes de poder usar esta plataforma.

Para obtener el grado de aceptación de la aplicación se debe realizar el siguiente cálculo:

- Sumar las puntuaciones de las preguntas impares (1, 3, 5, 7, 9) y restar 5 al resultado.
- Sumar las puntuaciones de las preguntas pares (2, 4, 6, 8, 10) y restar el resultado a 25.
- Sumar los resultados de ambos cálculos y multiplicar por 2.5 para obtener la puntuación final en una escala de 0 a 100.

El puntaje final, que varía entre 0 y 100, permite clasificar la usabilidad de la plataforma como Inaceptable (0-49), Mediocre (50-68) o Aceptable (69-100).

3.2.4. Encuesta

Este método de evaluación, como se detalla en el Capítulo 2.5.2, permite medir objetivos específicos comparando el estado inicial y final de las métricas evaluadas. Las preguntas de la encuesta están diseñadas para obtener respuestas númericas, lo que facilita la comparación de ambos estasdos. A continuación, se presentan algunas

de las preguntas, en este caso las que se utilizan para determinar el estado inicial de los objetivos específicos.

1. ¿Cuánto tiempo promedio dedicas a ajustar o posicionar un boceto en papel para que se adapte a la curvatura de la piel? (en minutos)

2. ¿Cuánto tiempo promedio dedicas a calcular el costo de un tatuaje manualmente? (en minutos)

3. ¿Cuál es tu tasa de inasistencia semanal promedio? (en porcentaje)

4. ¿Cuánto tiempo promedio dedicas a gestionar tu agenda y enviar recordatorios a tus clientes? (en minutos)

5. ¿Cuánto tiempo promedio dedicas a organizar tu inventario y calcular el costo de los insumos? (en minutos)

3.3. Resultados esperados

A continuación, se detallan los resultados esperados para cada uno de los objetivos específicos planteados en el Capítulo 1.2.2:

- **Reducción del ausentismo (Objetivo Específico 2):**
  - Resultado esperado: Se espera lograr una reducción del 50 % en la tasa de inasistencia semanal al implementar el módulo de agendamiento con recordatorios automáticos por WhatsApp, en comparación con la tasa actual sin este sistema.
  - Pruebas para lograr resultado esperado: Prueba de que el funcionamiento del módulo de agendamiento y recordatorios automáticos por WhatsApp

realice el envío correcto de los mensajes 24 horas antes de la sesión programada, y que el cliente reciba estos mensajes sin fallos.

- **Reducción del tiempo de cotización (Objetivo Específico 3):**
  - Resultado esperado: Se espera lograr una reducción del 40 % en los tiempos de cotización al utilizar el módulo de inventario y cálculo automatizado, en comparación con el método manual de cálculo.
  - Pruebas para lograr resultado esperado: Prueba de que el funcionamiento del módulo de inventario y cálculo automatizado permita calcular los costos de manera precisa y rápida, en comparación con el método manual.
- **Reducción del tiempo de previsualización (Objetivo Específico 4):**
  - Resultado esperado:Se espera lograr una reducción del 30 % en los tiempos de previsualización al utilizar el módulo de proyección 3D, en comparación con el método tradicional de ajuste manual en papel.
  - Pruebas para lograr resultado esperado: Prueba de que el funcionamiento del módulo de proyección 3D permita ajustar y posicionar los bocetos de manera rápida y precisa sobre el modelo anatómico, en comparación con el método manual en papel.

3.4. Resumen capítulo

En este capítulo se presentó detallademente la forma en que se aplicará la metodología PXP en el proyecto. También, se explican las metodologías de evaluación, mediante pruebas de software y, pruebas cualititativas y cuantitativas, en donde se agregan ejemplos de estas. Finalmente, se mencionan los resultados esperados, los cuales responden a los objetivos específicos planteados en el Capítulo 1.2.2.

En el siguiente capítulo se detallará la implementación de la metodología de desarrollo, en donde se explicará el trabajo realizado en las iteraciones, siguiendo las etapas de inicio de iteración, diseño, implementación, sistema de pruebas y retrospectiva.

4. Desarrollo

El presente capítulo está orientado a presentar la implementación de la metodología de desarrollo.

4.1. Análisis de factibilidad

Antes de diseñar la solución se evalúa si el proyecto puede llevarse a cabo con los medios disponibles. El análisis se hace en dos dimensiones: la técnica, que pregunta si existen la tecnología y el conocimiento para construirlo, y la económica, que pregunta cuánto cuesta construirlo y mantenerlo funcionando.

4.1.1. Factibilidad técnica

Todo el software empleado es gratuito o de código abierto, de modo que no hay licencias que adquirir: el entorno de desarrollo, el motor de base de datos, la contenedorización y las bibliotecas del lenguaje se distribuyen bajo licencias libres, y el control de versiones usa el plan gratuito de GitHub.

La plataforma se entrega como una aplicación web progresiva, de modo que el tatuador solo necesita un navegador actualizado con soporte de WebGL, que es lo que traen hoy los teléfonos y computadores de uso corriente, y una conexión a internet. No hay que instalar nada ni comprar equipamiento.

Respecto del conocimiento necesario, el proyecto se apoya en tecnologías ampliamente documentadas y de uso extendido en la industria, y en los contenidos de la carrera. La excepción es la previsualización tridimensional, que exige geometría

computacional y manejo de mallas, un área que el desarrollo abordó con documentación y experimentación directa sobre los modelos.

La Tabla 4.1 reúne los riesgos técnicos que se identificaron al evaluar la factibilidad, junto con la decisión que se tomó para controlarlos. Los cuatro se materializaron durante el desarrollo, y el capítulo documenta en cada iteración cómo se resolvieron.

Tabla 4.1: Riesgos técnicos identificados y su tratamiento

Riesgo	**Tratamiento**
El visor 3D puede no rendir en equipos modestos	Se trabaja sobre modelos optimizados y se recorta la malla alrededor del tatuaje en lugar de recorrerla entera, lo que permitió sostener el arrastre a 60 fotogramas por segundo
Los servicios gratuitos imponen límites y pueden cambiar	Se elige una arquitectura desacoplada, donde cada pieza puede migrar por separado, y el almacenamiento de archivos pasa por una capa con controladores intercambiables
El envío de mensajes al cliente depende de un servicio externo	Cada mensaje se arma una sola vez y puede salir por tres vías: la biblioteca que vincula el WhatsApp del tatuador, que automatiza el envío sin costo por mensaje; la plantilla de la API oficial, si el estudio la contrata; y un enlace manual de respaldo
El desarrollo lo realiza una sola persona	Se adopta PXP, una metodología pensada para desarrolladores individuales, y una sola base de código para todos los dispositivos

Con el equipamiento disponible, herramientas sin costo de licencia, tecnologías documentadas y riesgos acotados, el proyecto es técnicamente factible.

4.1.2. Factibilidad económica

El proyecto se desarrolla en el contexto de una memoria de título, por lo que no hubo desembolso por el trabajo de desarrollo.

La operación, en cambio, sí tiene un costo real, y la decisión de diseño fue dejarlo en cero. Cada pieza de la plataforma se publica en el plan gratuito de un servicio distinto, como resume la Tabla 4.2. Ninguno de ellos exige tarjeta de crédito.

Tabla 4.2: Costo mensual de la infraestructura en la nube

Pieza	**Servicio**	**Plan gratuito**	**Costo**
Frontend	Cloudflare Pages	Tráfico sin límite	$0
Backend	Render	750 horas al mes y 512 MB; se duerme tras 15 minutos sin uso	$0
Base de datos	Neon	0,5 GB, sin vencimiento	$0
Imágenes	Cloudinary	25 créditos al mes	$0
Correo	Brevo	Unos 300 correos al día	$0
Repositorio	GitHub	Repositorios privados sin límite	$0

Estos planes tienen una contrapartida que conviene dejar escrita: el backend se suspende tras unos minutos sin uso, de modo que la primera petición después de una pausa demora cerca de un minuto en responder. La plataforma lo advierte en pantalla mientras el servidor despierta. Es el precio de no pagar alojamiento, y para un estudio que consulta la agenda unas pocas veces al día resulta aceptable; si dejara de serlo, el plan de pago de Render resuelve exactamente ese punto sin cambiar el código.

El envío de los recordatorios tampoco agrega costo: la biblioteca que vincula el WhatsApp del tatuador al servidor los manda desde su propio número, de modo que la automatización no depende de contratar un servicio de mensajería. La vía oficial de Meta, que cobra por cada mensaje de plantilla entregado, queda como alternativa configurable para el estudio que la prefiera. La Tabla 4.3 resume el panorama completo.

Tabla 4.3: Resumen de costos del proyecto

Concepto	**Costo**
Licencias de software	$0
Equipamiento, ya disponible	$0
Infraestructura en la nube, mensual	$0
Mensajes al cliente, mensual	$0

La conclusión es que el proyecto es económicamente factible, y además lo es para quien lo va a usar: un estudio pequeño puede adoptar la plataforma sin pagar una suscripción mensual ni comprar equipamiento, que es precisamente la barrera que dejó fuera a las herramientas comerciales revisadas en el estado del arte.

4.2. Diseño de software

A continuación, se presentan elementos claves en el diseño de software, tales como arquitecturas, modelos de datos y mockups.

4.2.1. Arquitectura física

La arquitectura física implementada corresponde a un modelo cliente-servidor. Durante la fase de desarrollo y pruebas, todos los componentes del sistema se alojan en un entorno local, utilizando contenedores Docker, lo que garantiza la portabilidad del entorno y elimina dependencias de configuración entre máquinas.

Como se ilustra en la Figura 4.1, el flujo de comunicación comienza cuando el dispositivo cliente realiza una petición a través de Internet hacia el servidor de Frontend, donde se sirve la aplicación React. Las interacciones dinámicas del usuario generan peticiones HTTP/HTTPS hacia el servidor Backend desarrollado en Node.js, el cual se comunica con dos servicios externos: el Servidor de Base de Datos PostgreSQL, encargado de la persistencia de datos, y la API de Mensajería de WhatsApp, responsable del envío de recordatorios automáticos a los clientes.

Esta separación en contenedores independientes facilita una eventual migración hacia servicios en la nube especializados.

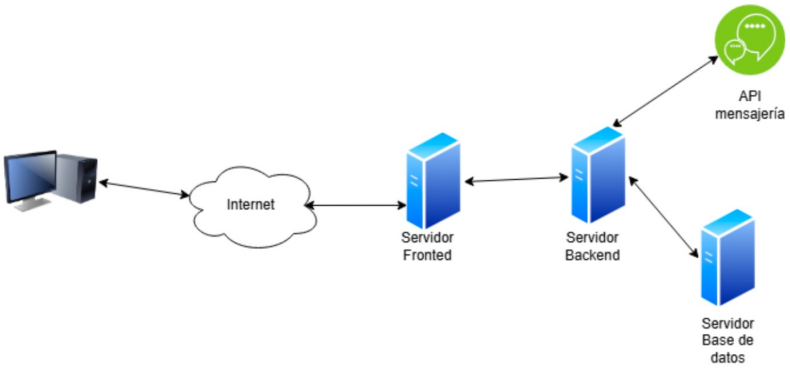


Figura 4.1: Arquitectura Física del Sistema.

4.2.2. Arquitectura lógica

La arquitectura lógica propuesta sigue el patrón de tres capas.

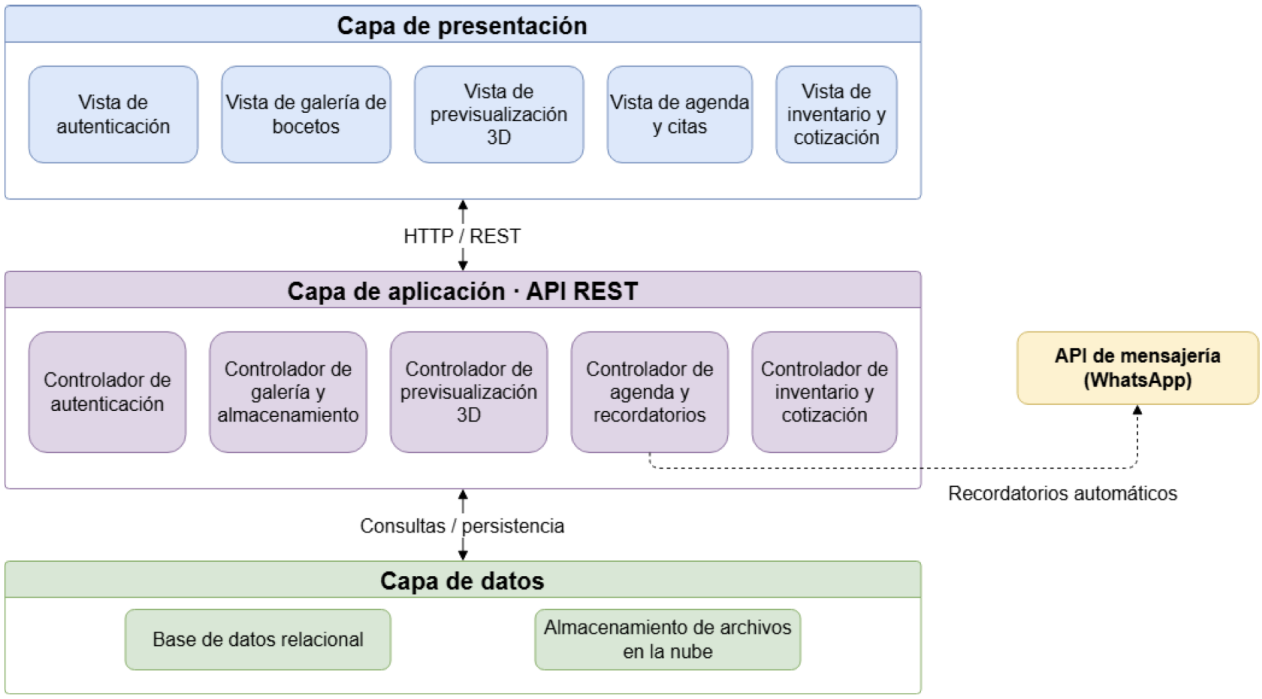


Figura 4.2: Arquitectura Lógica del Sistema.

En la Figura 4.2 se ilustra la arquitectura lógica propuesta sigue el patrón de tres capas, ampliamente utilizado en el desarrollo de aplicaciones web modernas. Este patrón organiza el sistema en niveles de responsabilidad bien definidos, donde

cada capa solo se comunica con la capa inmediatamente adyacente, lo que favorece la separación de responsabilidades, la mantenibilidad del código y la escalabilidad del sistema.

La capa de presentación corresponde a la interfaz de usuario ejecutada en el navegador del cliente. Esta capa es responsable de renderizar las vistas y gestionar la interacción del usuario, organizándose en módulos que corresponden a las funcionalidades principales del sistema: autenticación, galería de bocetos, previsualización 3D, agenda de citas, e inventario y cotización.

La capa de aplicación centraliza la lógica de negocio del sistema, expuesta mediante una API REST que recibe las solicitudes HTTP provenientes de la capa de presentación. Esta capa contiene los controladores de cada módulo, encargados de validar los datos de entrada, ejecutar las reglas de negocio como el algoritmo de cotización automática o el envío de recordatorios y coordinar las operaciones sobre la capa de datos.

La capa de datos es responsable de la persistencia del sistema. Se divide en dos componentes: la base de datos relacional, que almacena de forma estructurada los registros de usuarios, citas, inventario y cotizaciones, y el almacenamiento de archivos en la nube, dedicado a los bocetos en formato de imagen y los modelos anatómicos tridimensionales.

Adicionalmente, la capa de aplicación se integra con un servicio externo de mensajería para el envío automatizado de recordatorios a los clientes, manteniéndose esta integración desacoplada del resto del sistema mediante su propio controlador.

4.2.3. Modelo de datos

Como se mencionó en el Capítulo 2.3.3, el modelo de datos se basa en una base de datos relacional, utilizando PostgreSQL. A continuación, la Figura 4.3 muestra el diagrama entidad-relación del modelo de datos propuesto para el sistema.

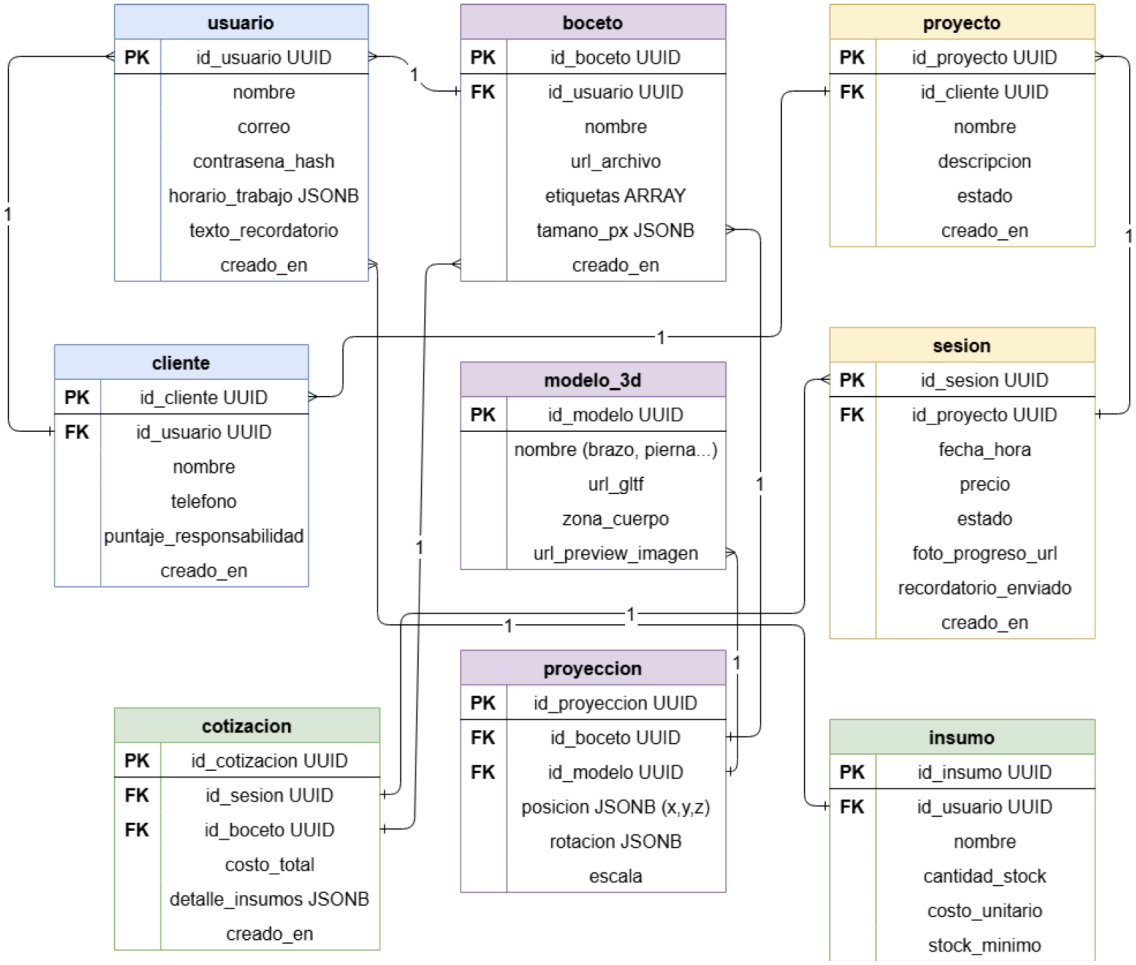


Figura 4.3: Diagrama Entidad-Relación del modelo de datos del sistema.

4.2.4. Mockups

En esta sección se presentan algunos de los mockups propuestos durante el proceso de desarrollo del sitema, los cuales fueron diseñados en base a los requerimientos funcionales y validos por la contrapartee. Estos mockups tienen como objetivo guiar el desarrollo de las principales interfaces del sistema y asegurando que cumplan con las expectativas del cliente experto.

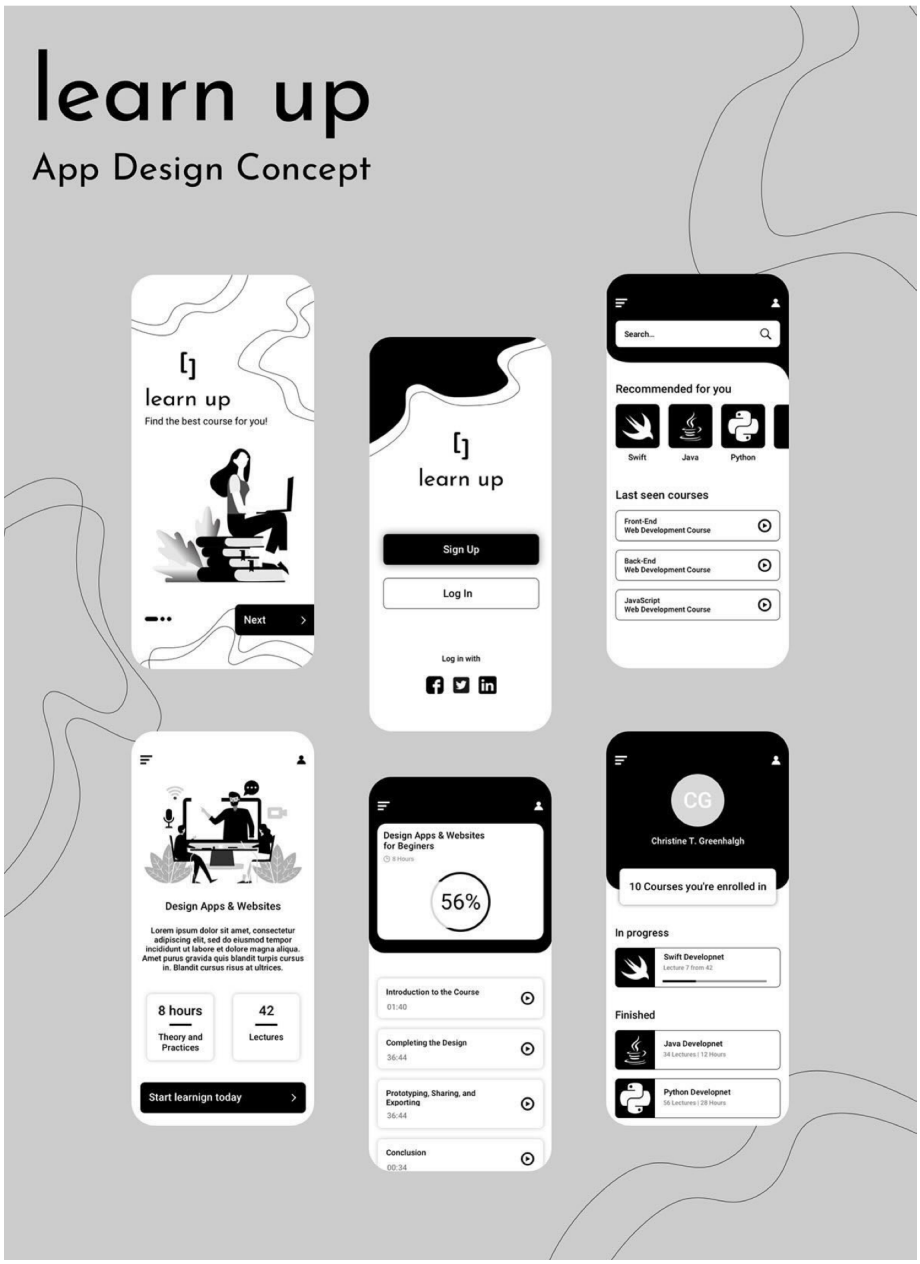


Figura 4.4: Idea 1

Figura 4.5: Idea 2

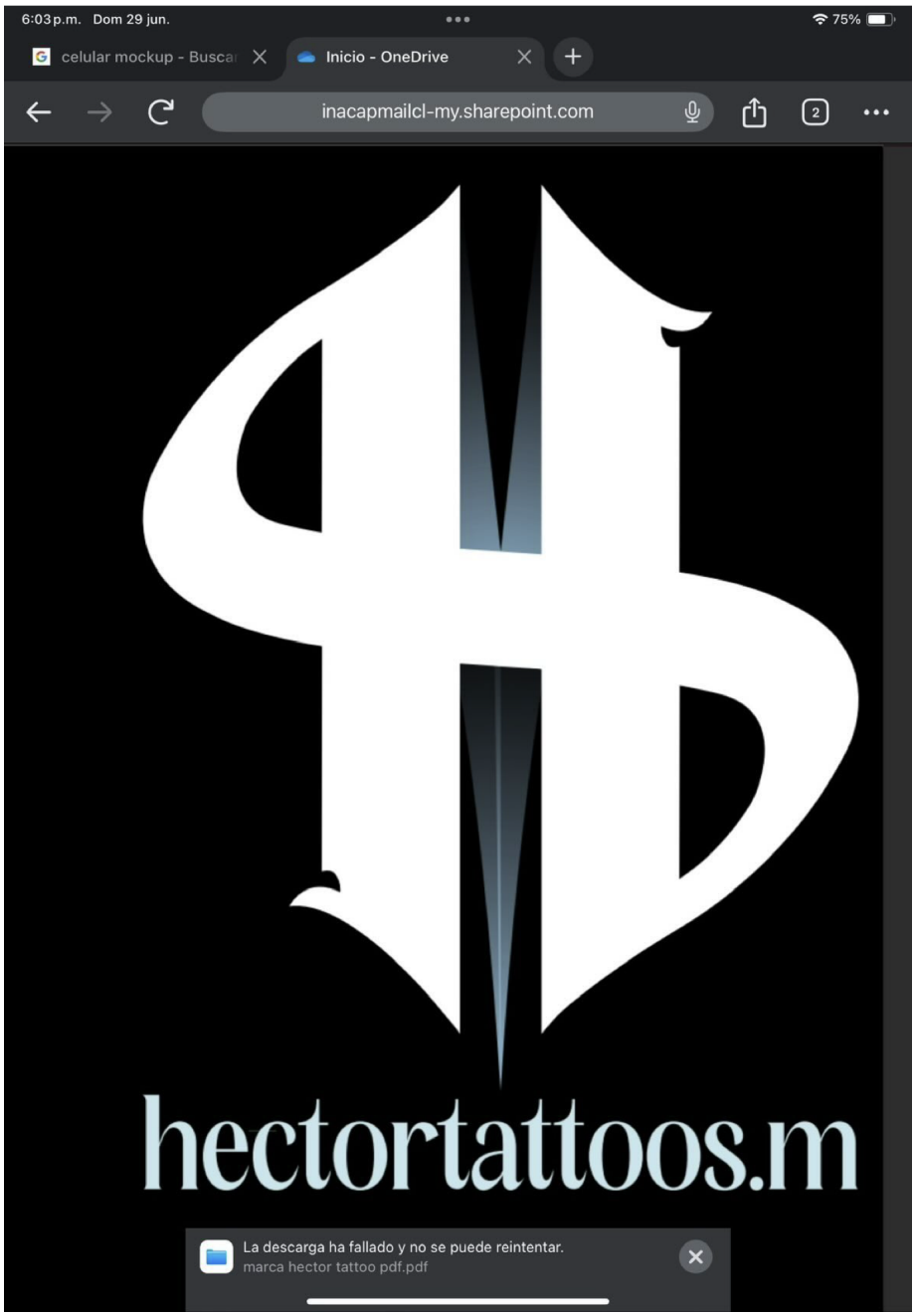


Figura 4.6: Idea 3

En las Figuras 4.4, 4.5 y 4.6 se presentan tres ideas de diseño para la interfaz de usuario del sistema, estas ideas fueron diseñadas por el usuario experto y buscan ser una guía visual para la elaboración de los mockups finales y la estética que se busca conseguir en la aplicación.

Figura 4.7: Mockup Inicio Sesión.

La figura 4.7 muestra la primera vista del usuario al ingresar a la aplicación, en el cual podrá iniciar sesión o redirigirse a la vista de “Olvidé Mi Contraseña”.

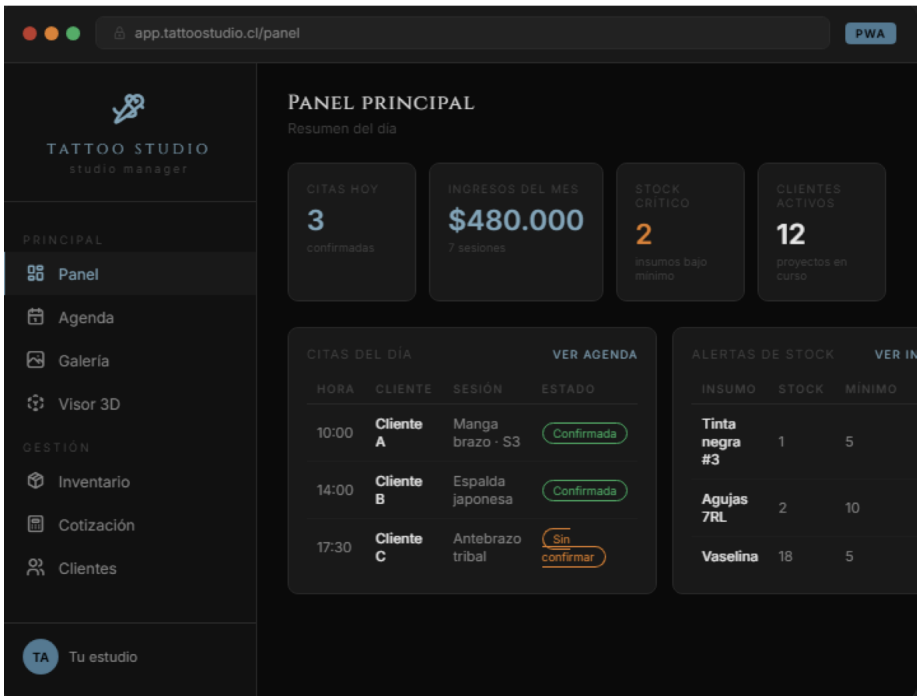


Figura 4.8: Mockup Dashboard.

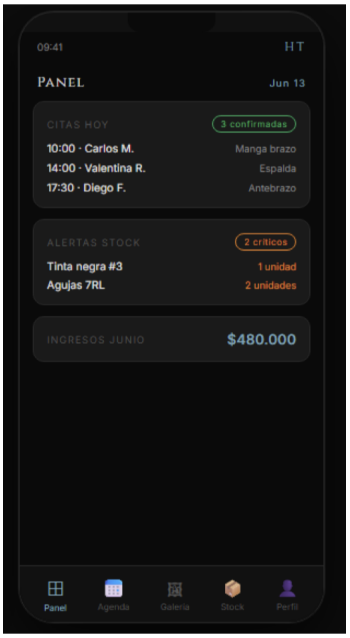


Figura 4.9: Mockup Dashboard Móvil.

La Figura 4.8 y 4.9 muestran el diseño propuesto para el dashboard del sistema,

tanto web como móvil, en el cual se muestran todos los componentes principales con lo que puede interactuar el usuario.

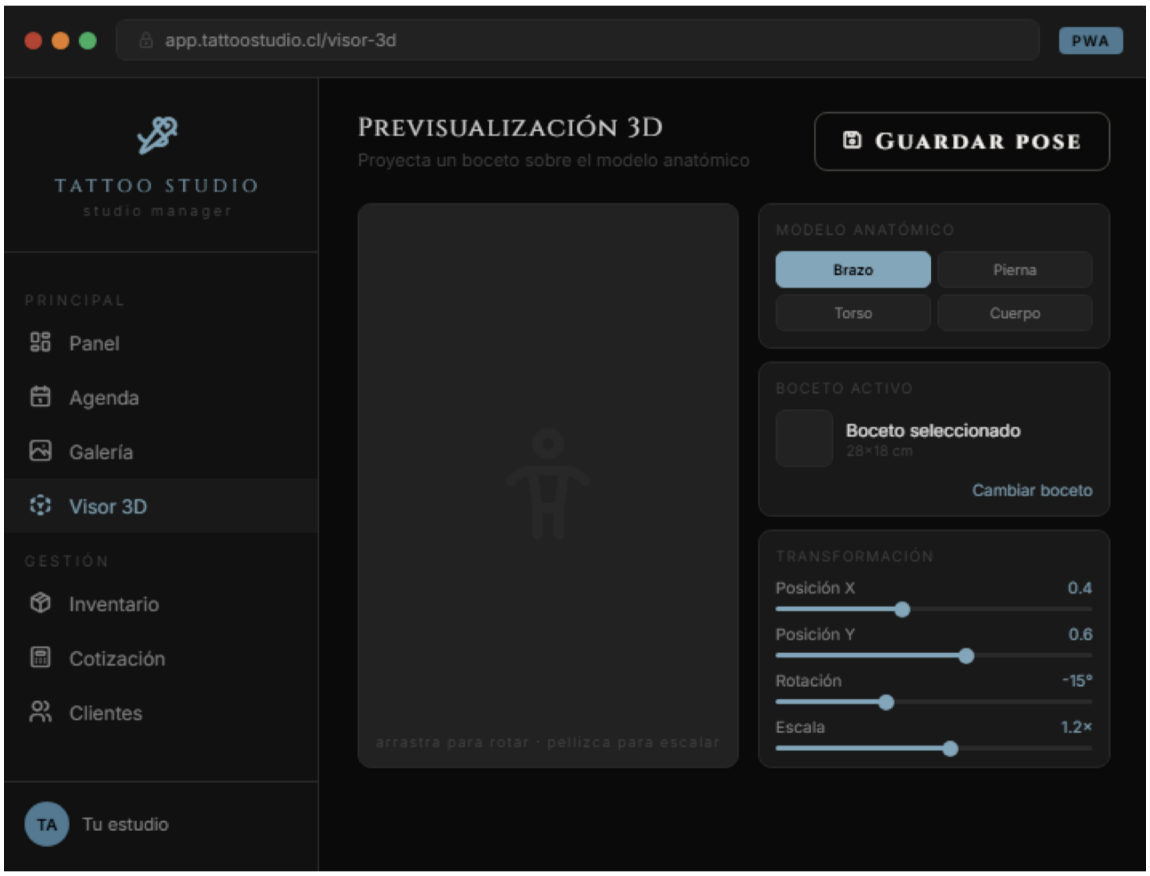


Figura 4.10: Mockup Previsualización 3D.

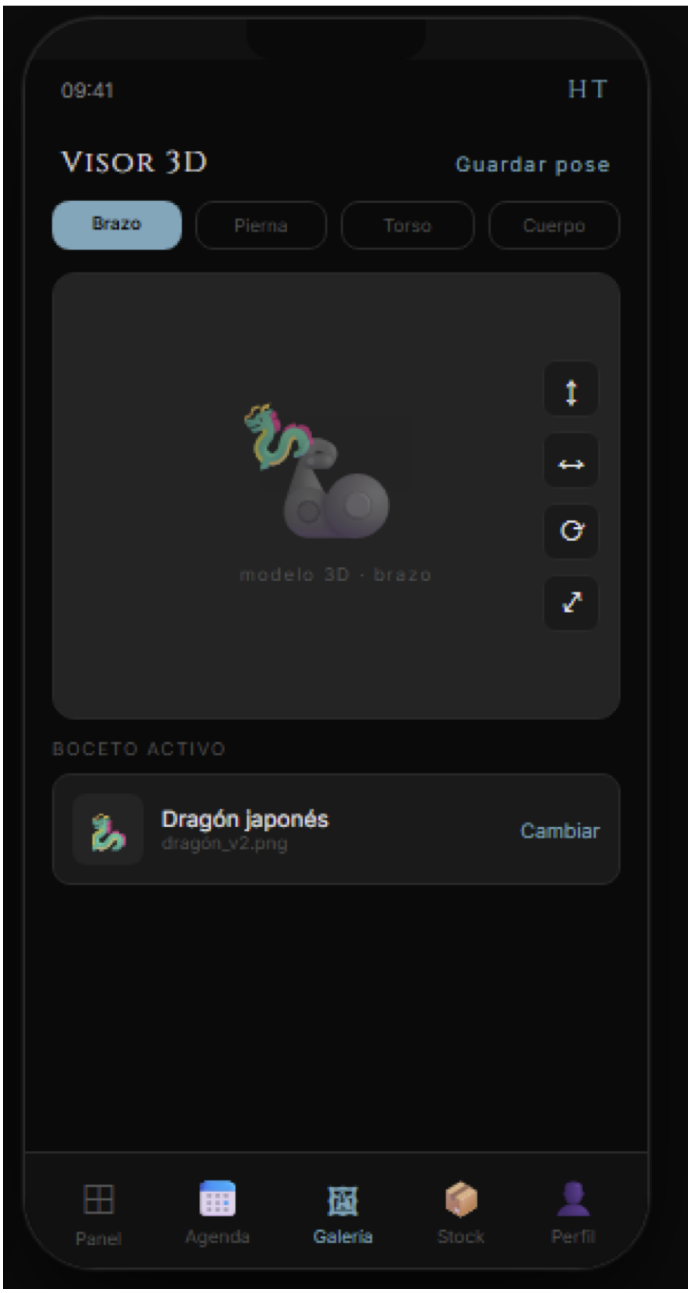


Figura 4.11: Mockup Previsualización 3D Móvil.

Por último, la Figura 4.10 y 4.11 muestran el diseño propuesto para la vista de previsualización 3D, tanto web como móvil, en el cual se muestra la vista del modelo anatómico con el boceto proyectado sobre este, junto con las herramientas de ajuste para escalar, mover y rotar el diseño sobre el modelo 3D.

4.3. Iteraciones

A continuación, se detallan como se abordaron las iteraciones descritas en el Capítulo 3.1.2.

4.3.1. Iteración 1

En esta iteración, no existen historias de usuario asignadas, por lo que se enfocó en la planificación y diseño de la arquitectura del sistema, el modelo de datos y los mockups. Además, se establecieron las bases para el desarrollo de las siguientes iteraciones, definiendo los estándares de codificación, las herramientas a utilizar y la estructura general del proyecto.

4.3.2. Iteración 2

En esta iteración se comienza con la implementación de las historias de usuario HU01, HU02 y HU03, las cuales corresponden a la funcionalidad de autenticación. A continuación, se detalla la implementación siguiendo la metodología de desarrollo PXP.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 8 puntos según la Tabla 3.2.

- **HU01:** como tatuador quiero registrarme en la plataforma ingresando mis datos, para tener un espacio de trabajo integral.
- **HU02:** como tatuador quiero iniciar y cerrar sesión mediante mis credenciales, para proteger la privacidad de mi negocio.
- **HU03:** como tatuador quiero recuperar mi contraseña en caso de olvido, para no perder el acceso a mi agenda e inventario.

Las tres historias operan sobre el mismo módulo: el registro crea la cuenta, el inicio de sesión la abre y la recuperación devuelve el acceso cuando se pierde. Por eso se desarrollaron en conjunto, compartiendo el modelo de usuario, el mecanismo de sesión y las vistas de entrada a la plataforma.

Inicio de Iteración

En esta etapa, se crean y son registradas las tareas acordes a las historias de usuario, donde la Tabla 4.4 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.4: Tareas asociadas a la Iteración 2

Historia	**Tarea**	**Descripción**
HU001	T0001	Crear diagrama de secuencia
HU001	T0002	Crear ruta para inicio de sesión
HU001	T0003	Crear vista para inicio de sesión
HU002	T0001	Crear diagrama de secuencia
HU002	T0002	Crear dashboard inicial
HU002	T0003	Crear vista para cerrar sesión
HU003	T0001	Crear diagrama de secuencia
HU003	T0002	Crear ruta para olvidar contraseña
HU003	T0003	Crear ruta para cambiar contraseña
HU003	T0004	Crear vista para olvidar contraseña
HU003	T0005	Crear vista para cambiar contraseña
HU001, HU002, HU003	T0009	Creación y testeo de pruebas unitarias
HU001, HU002, HU003	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa, se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. A continuación, la Figura 4.12 ilustra el flujo del inicio de sesión, que es el que establece el patrón de autenticación que siguen las demás funcionalidades de la plataforma.

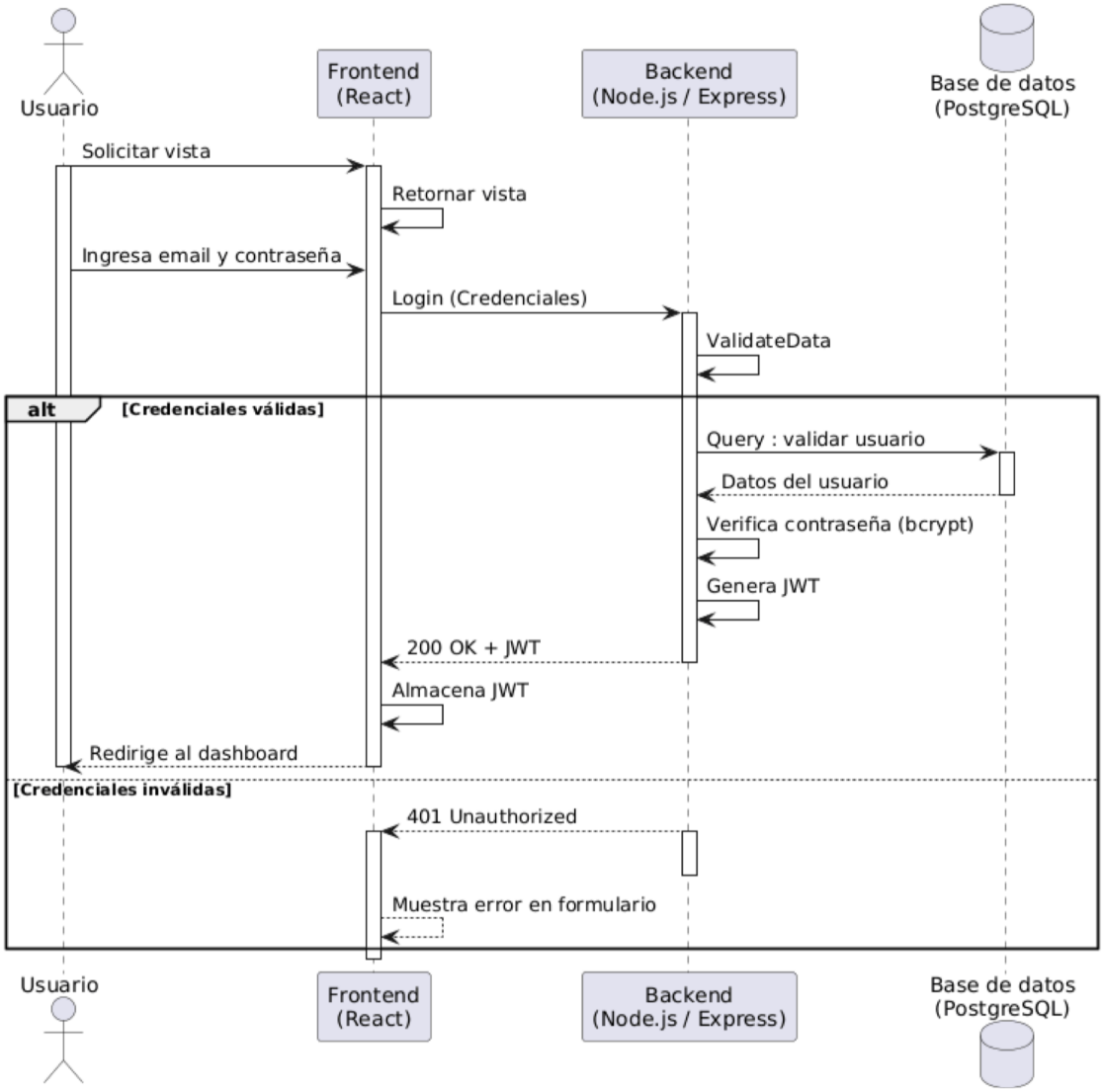


Figura 4.12: Diagrama de secuencia HU01: Login.

Implementación

En esta etapa, se desarrolla tanto el backend como el frontend de las historias de usuario asignadas a la iteración. Se crean las rutas necesarias en el backend para manejar las solicitudes de inicio de sesión, cierre de sesión y recuperación de contraseña, mientras que en el frontend se desarrollan las vistas correspondientes para que el usuario pueda interactuar con estas funcionalidades. En el caso del inicio de sesión, se implemente JSON Web Tokens (JWT) para la autenticación y autorización

de usuarios, asegurando acceso seguro a rutas protegidas del sistema. (Figura 4.13)

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const userModel = require('../models/userModel');
const passwordResetModel = require('../models/passwordResetModel');
const emailService = require('../services/emailService');

const FRONTEND_URL = process.env.FRONTEND_URL || 'http://localhost:5173';

const JWT_SECRET = process.env.JWT_SECRET || 'dev_secret_changeme';
const TOKEN_TTL = '7d';

function signToken(user) {
  return jwt.sign({ sub: user.id, email: user.email }, JWT_SECRET, {
    expiresIn: TOKEN_TTL,
  });
}
```

Figura 4.13: Implementación de autenticación mediante JSON Web Tokens (JWT).

Otro modulo que se implementó fue el servicio de envío de correos electrónicos para la funcionalidad de recuperación de contraseña, utilizando una librería de Node.js (Nodemailer) para enviar correos a través de un servicio SMTP, lo que permite a los usuarios recibir un enlace para restablecer su contraseña de manera segura como se puede ver en la Figura 4.14.

Proyecto > backend > src > services > emailService.js > ...

```
1   const nodemailer = require('nodemailer');
2
3   let transporter = null;
4
5   async function getTransporter() {
6     if (transporter) return transporter;
7
8     if (process.env.SMTP_HOST) {
9       transporter = nodemailer.createTransport({
10        host: process.env.SMTP_HOST,
11        port: Number(process.env.SMTP_PORT) || 587,
12        secure: Number(process.env.SMTP_PORT) === 465,
13        auth: {
14          user: process.env.SMTP_USER,
15          pass: process.env.SMTP_PASS,
16        },
17        tls: { rejectUnauthorized: false },
18      });
19
20      await transporter.verify();
21      console.log('✓ Conexión SMTP verificada');
22    } else {
23      const test = await nodemailer.createTestAccount();
24      transporter = nodemailer.createTransport({
25        host: 'smtp.ethereal.email',
26        port: 587,
27        auth: { user: test.user, pass: test.pass },
28      });
29      console.log('📧 SMTP no configurado — usando Ethereal (pruebas)');
30    }
```

Figura 4.14: Implementación del servicio de envío de correos electrónicos para recuperación de contraseña.

Al finalizar el desarrollo de las funcionalidades, se crean las pruebas unitarias correspondientes para cada una de las tareas definidas en el Inicio de Iteración, utilizando un framework de pruebas como Jest para asegurar que el código cumpla con los requisitos definidos en las historias de usuario [62]. La Figura 4.15 muestra la estrucutura de la prueba, donde se describe el test, usando supertest para simular las peticiones HTTP y verificar las respuestas del servidor. Los elementos claves para este ejemplo son post, que corresponde a la ruta a interactuar, send, que corresponde a los datos enviados en la petición, expect, que corresponde a las aserciones para validar el resultado esperado y end, que corresponde a condiciones.

```
describe('POST /api/auth/login (HU02)', () => {
  const password = TEST_PASSWORD;
  let user;

  beforeAll(async () => {
    user = {
      id: '11111111-1111-1111-1111-111111111111',
      name: TEST_NAME,
      email: TEST_EMAIL,
      password_hash: await bcrypt.hash(password, 10),
    };
  });

  beforeEach(() => {
    jest.clearAllMocks();
  });

  test('credenciales válidas devuelven 200 con token y usuario', (done) => {
    userModel.findByEmail.mockResolvedValue(user);

    request(app)
      .post('/api/auth/login')
      .send({ email: user.email, password })
      .expect('Content-Type', /json/)
      .expect(200)
      .expect((res) => {
        if (!res.body.token) throw new Error('No se recibió el token');
        if (res.body.user.email !== user.email) throw new Error('Correo incorrecto');
        if ('password_hash' in res.body.user) throw new Error('Se filtró el hash');
      })
```

Figura 4.15: Estructura de pruebas unitarias.

La Figura 4.16 muestra los resultado de la ejecución de las pruebas unitarias.

```
PS C:\Users\pablo\Downloads\tesis\Proyecto\backend> npm test

> backend@1.0.0 test
> jest --runInBand

 PASS  tests/auth.login.test.js
  POST /api/auth/login (HU02)
    √ credenciales válidas devuelven 200 con token y usuario (132 ms)
    √ contraseña incorrecta devuelve 401 (85 ms)
    √ correo inexistente devuelve 401 (5 ms)
    √ faltan campos obligatorios devuelve 400 (5 ms)

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        1.107 s, estimated 2 s
Ran all test suites.
PS C:\Users\pablo\Downloads\tesis\Proyecto\backend>
```

Figura 4.16: Resultado de la ejecución de pruebas unitarias.

Resultado del desarrollo

Como resultado de esta iteración, la plataforma cuenta con un acceso propio para

cada tatuador: puede crear su cuenta, entrar con sus credenciales, cerrar la sesión y pedir un enlace por correo cuando olvida su contraseña. Las Figuras 4.17, 4.18 y 4.19 presentan las vistas desarrolladas durante esta etapa.

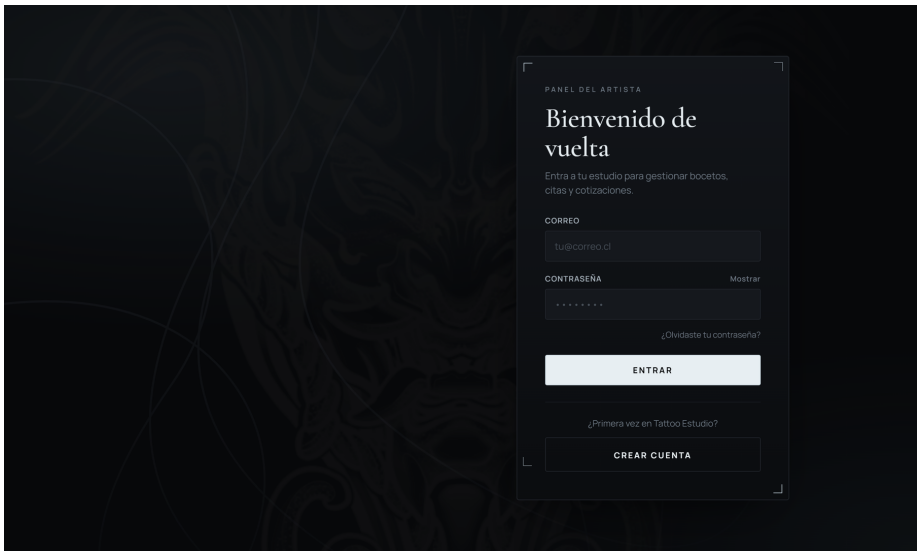


Figura 4.17: Vista "Inicio de sesión", con el acceso a la recuperación de contraseña y a la creación de una cuenta nueva.

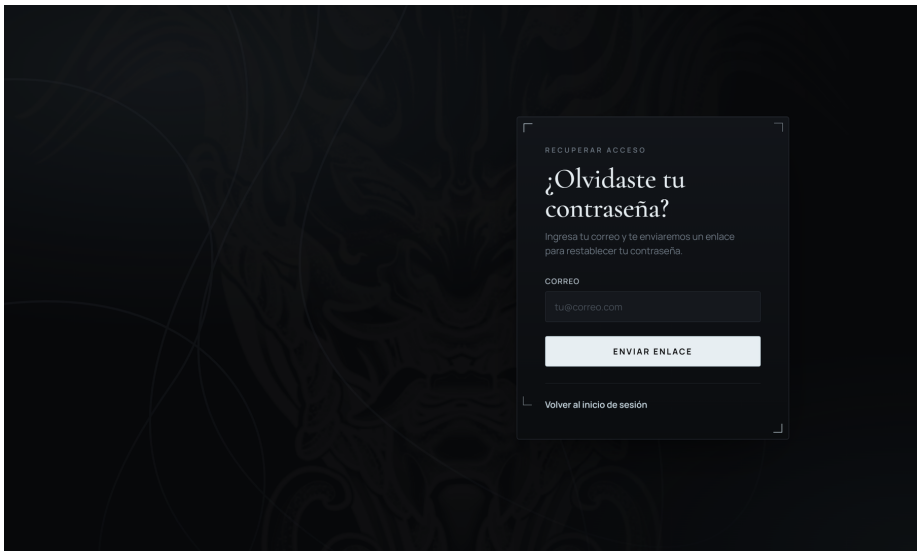


Figura 4.18: Vista "Recuperar contraseña": el tatuador indica su correo y recibe el enlace para definir una nueva.

La Figura 4.19 muestra la pantalla a la que llega el tatuador una vez autenticado,

que en esta iteración se dejó montada con la navegación de la plataforma y se fue completando con datos reales en las iteraciones siguientes.

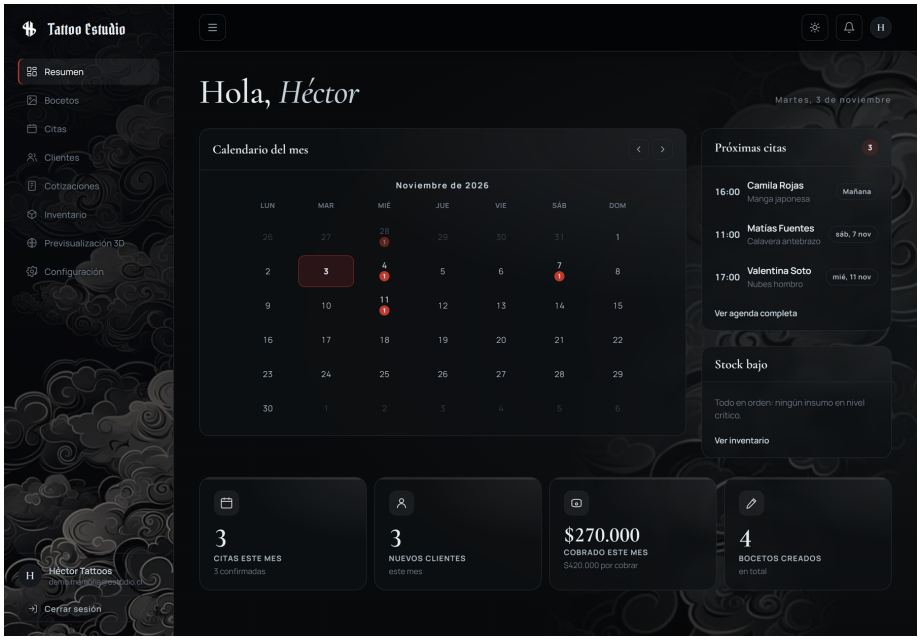


Figura 4.19: Vista “Resumen”, primera pantalla tras iniciar sesión.

Sistema de pruebas

Después de la implementación, se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Ingresar con credenciales válidas.
- Ingresar a una ruta que no existe.
- Ingresar con un usuario que no existe.
- Ingresar con una contraseña incorrecta.
- Solicitar recuperación de contraseña con un correo registrado.
- Solicitar recuperación de contraseña con un correo no registrado.
- Solicitar recuperación de contraseña con un correo con formato inválido.
- Solicitar recuperación de contraseña sin ingresar un correo.

- Cerrar sesión correctamente.

Pruebas unitarias

Junto con los casos anteriores, las historias de esta iteración quedaron cubiertas por pruebas automatizadas sobre las rutas de autenticación y sobre el servicio de correo, usando Jest con Supertest. Las capas que tocan la base de datos se reemplazan por dobles de prueba, de modo que cada caso evalúa el controlador de forma aislada. La Tabla 4.5 detalla las pruebas implementadas; incluye también los casos que se agregaron después sobre estas mismas rutas, como el bloqueo de cuentas desactivadas y el cambio de la contraseña temporal entregada por el administrador. La columna "Tests" indica cuántos casos de prueba independientes verifican cada función.

Tabla 4.5: Pruebas unitarias implementadas en la Iteración 2

ID	**Función evaluada**	**Objetivo**	**Tests**
PU-01	`POST /login`	Verificar el ingreso con credenciales válidas y el rechazo de contraseña incorrecta, correo inexistente, campos faltantes o cuenta desactivada.	8
PU-02	`POST /register`	Validar que cada cuenta nueva queda como su propio estudio y que un correo repetido devuelve 409.	2
PU-03	`POST /change-password`	Verificar el cambio con la contraseña actual, el rechazo sin sesión y el aviso de contraseña temporal en el login.	7
PU-04	`parseAddress()`	Validar la interpretación del remitente del correo de recuperación.	4
PU-05	`sendPasswordReset()`, `cleanApiKey()`	Comprobar que el correo sale con remitente, destinatario, asunto y enlace, y que una clave de servicio mal copiada se rechaza explicando cuál usar.	7

En total, durante esta iteración se ejecutaron 28 pruebas unitarias distribuidas en 2 archivos de prueba, enfocadas en el control de acceso y en el envío del correo de recuperación.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto del inicio de sesión, del registro y de la recuperación de contraseña. Se comprobó en particular que un intento con credenciales equivocadas no revela si el correo existe ni si la cuenta está desactivada, y que el enlace de recuperación queda anulado una vez usado. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.6 y muestra la Figura 4.16.

Tabla 4.6: Resultado de ejecución de pruebas unitarias de la Iteración 2

Archivos evaluados	**Tests ejecutados**	**Resultado**
2	28	28 aprobados

Retrospectiva

Al finalizar la iteración, se revisa si las estimaciones de tiempo para las historias de usuario fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. En este caso, las estimaciones fueron acertadas, y se completaron todas las funcioanlidades planeadas sin pendientes.

4.3.3. Iteración 3

En esta iteración se implementan las historias de usuario HU04, HU05, HU06 y HU07, que en conjunto forman la galería de bocetos: el módulo con el que el tatuador sube sus diseños a la plataforma, los consulta, los clasifica con etiquetas y elimina los que ya no necesita. Mientras la iteración anterior resolvió el acceso al sistema, esta incorpora el primer contenido propio del usuario y, con él, dos problemas nuevos: el manejo de archivos binarios y el aislamiento de la información entre cuentas.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 11 puntos según la Tabla 3.2.

- **HU04:** como tatuador quiero importar imágenes a la plataforma, para almacenarlas en la nube.
- **HU05:** como tatuador quiero ver todos mis diseños en una galería, para acceder a ellos de manera expedita.
- **HU06:** como tatuador quiero eliminar un boceto antiguo de la galería, para mantener mi espacio ordenado y liberar almacenamiento.
- **HU07:** como tatuador quiero añadir etiquetas a mis bocetos, para filtrarlos y encontrarlos ágilmente frente al cliente.

Las cuatro historias operan sobre un mismo recurso del sistema, el boceto, por lo que se desarrollaron en conjunto: comparten el modelo de datos, la ruta de la API y la vista que las presenta al usuario.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.7 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.7: Tareas asociadas a la Iteración 3

Historia	**Tarea**	**Descripción**
HU04	T0001	Crear diagrama de secuencia
HU04	T0002	Crear la capa de almacenamiento con drivers intercambiables
HU04	T0003	Crear ruta de subida con validación de formato y tamaño
HU04	T0004	Crear formulario de subida en la vista de galería
HU05	T0001	Crear diagrama de secuencia
HU05	T0002	Crear tabla de bocetos y consulta filtrada por usuario
HU05	T0003	Crear ruta de listado
HU05	T0004	Crear vista de galería en grilla
HU06	T0001	Crear diagrama de secuencia
HU06	T0002	Crear ruta de eliminación con borrado del archivo asociado
HU06	T0003	Agregar la confirmación de borrado en la vista
HU07	T0001	Agregar etiquetas y estado al modelo de datos
HU07	T0002	Crear ruta de edición de metadatos
HU07	T0003	Agregar los filtros por estado y etiqueta en la vista
HU04, HU05, HU06, HU07	T0009	Creación y testeo de pruebas unitarias
HU04, HU05, HU06, HU07	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario

asociadas a la iteración. La Figura 4.20 ilustra la subida de un boceto, que es el flujo más completo de la galería. La consulta filtrada y la eliminación de un diseño siguen el mismo recorrido: la petición pasa primero por el middleware de autenticación, que entrega el identificador del tatuador dueño de la sesión, y ese identificador es el que después filtra cada consulta a la base de datos.

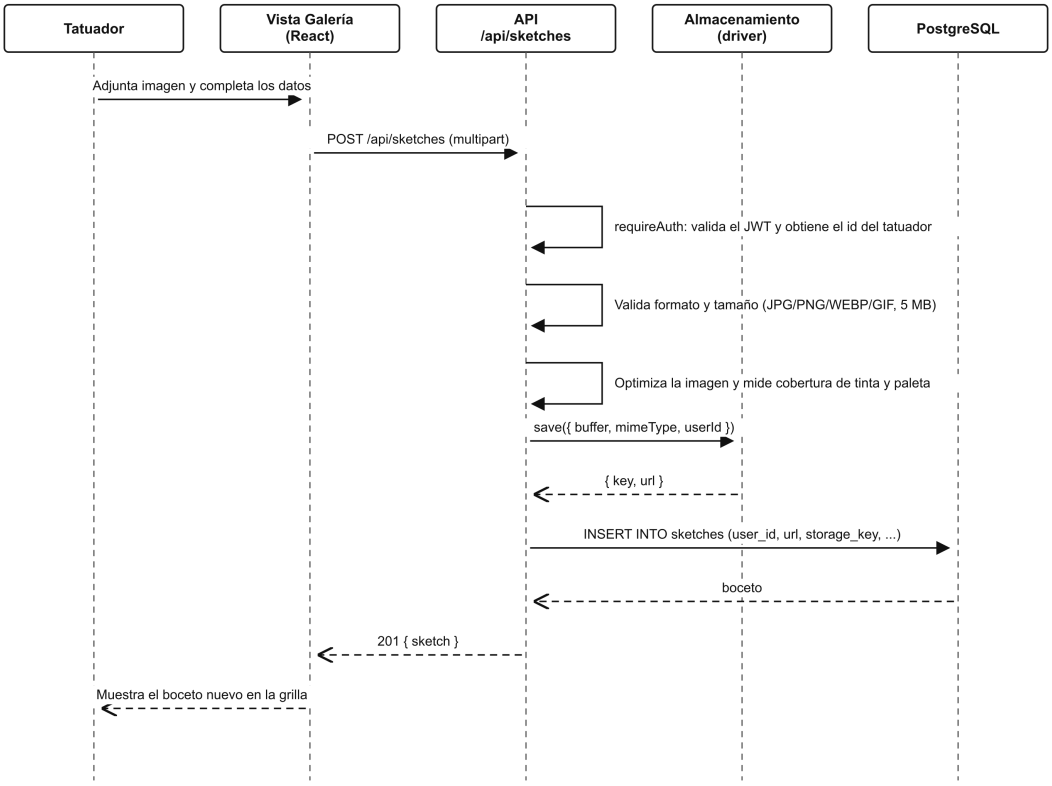


Figura 4.20: Diagrama de secuencia HU04: subida de un boceto.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. En el backend se levanta el recurso `/api/sketches` con sus cuatro operaciones (listar, subir, editar metadatos y eliminar), todas protegidas por el mismo middleware de autenticación de la iteración anterior.

La subida de archivos se resuelve con Multer, una librería de Node.js que interpreta las peticiones *multipart* y deja la imagen disponible para el controlador. Se configuró para recibir el archivo en memoria y no en disco, con un límite de 5 MB y

una lista blanca de formatos (JPG, PNG, WEBP y GIF), de modo que una imagen rechazada nunca llega a ocupar espacio. La Figura 4.21 muestra esa configuración.

backend/src/middleware/imageUpload.js

```
const multer = require('multer');

const MAX_FILE_BYTES = Number(process.env.MAX_UPLOAD_BYTES) || 5 * 1024 * 1024;
const ALLOWED_MIME = ['image/jpeg', 'image/png', 'image/webp', 'image/gif'];

// El archivo se recibe en memoria: nunca toca el disco antes de que el
// driver decida dónde guardarlo (disco local o nube).
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: MAX_FILE_BYTES, files: 1 },
  fileFilter: (req, file, cb) => {
    if (!ALLOWED_MIME.includes(file.mimetype)) {
      return cb(new Error('Formato no permitido. Usa JPG, PNG, WEBP o GIF.'));
    }
    cb(null, true);
  },
});
```

Figura 4.21: Configuración de la recepción de imágenes en memoria.

Una decisión de diseño relevante de esta iteración fue no guardar las imágenes en la base de datos ni escribirlas directamente a disco desde el controlador, sino concentrar todo el acceso a archivos en una capa de almacenamiento con drivers intercambiables. Esa capa expone tres operaciones (guardar, leer y eliminar) y decide con una variable de entorno si trabaja contra el disco local, usado en desarrollo, o contra un servicio en la nube, usado en producción. La base de datos guarda únicamente los metadatos del boceto junto con el driver y la clave con que se almacenó el archivo, lo que permite que un diseño subido antes de migrar a la nube se siga eliminando correctamente del disco. Esta separación no estaba contemplada en el planteamiento inicial y surgió durante el desarrollo, al advertir que la plataforma debía poder desplegarse en la nube sin reescribir el módulo de bocetos.

Aprovechando que la imagen ya está en memoria, al momento de subirla se mide también la fracción del lienzo cubierta por tinta y las familias de color predominantes del diseño. Ambos valores se guardan junto al boceto y son los que más adelante alimentan el cálculo de costos de la HU14, de manera que una medición que de otro modo habría que repetir en cada cotización se realiza una sola vez.

Respecto del aislamiento de la información, cada consulta incorpora el identifi-

cador del tatuador obtenido del token, tanto en el listado como en la edición y el borrado, según se observa en la Figura 4.22. Un boceto que pertenece a otra cuenta responde 404 y no 403: de esa forma el sistema no revela que el recurso existe. Los filtros de la HU07 se resuelven en la misma consulta, agregando condiciones por estado y por etiqueta sobre el arreglo de etiquetas del boceto.

backend/src/models/sketchModel.js

```
async function listByUser(userId, { status, tag } = {}) {
  const params = [userId];
  let sql = `SELECT ${PUBLIC_COLUMNS} FROM sketches WHERE user_id = $1`;

  if (status) {
    params.push(status);
    sql += ` AND status = $${params.length}`;
  }
  if (tag) {
    params.push([tag]);
    sql += ` AND tags @> $${params.length}`;
  }

  sql += ' ORDER BY created_at DESC';

  const { rows } = await pool.query(sql, params);
  return rows;
}
```

Figura 4.22: Consulta de bocetos filtrada por el usuario del token y por los filtros de la HU07.

El controlador también contempla el caso en que el archivo se guarda correctamente pero la inserción en la base de datos falla: ante ese escenario elimina el archivo recién subido, evitando que queden imágenes huérfanas ocupando almacenamiento.

En el frontend se desarrolló la vista de galería, que presenta los bocetos en una grilla ordenada del más reciente al más antiguo, permite subir un diseño arrastrándolo sobre el formulario, cambiar su estado desde la misma tarjeta y filtrar por estado y por etiqueta. Los filtros no se resuelven en el navegador sino que se envían a la API, de modo que la vista siempre muestra el resultado de la consulta y no un subconjunto de lo ya descargado.

Resultado del desarrollo

Como resultado de esta iteración, el tatuador cuenta con un repositorio propio de diseños dentro de la plataforma, con sus imágenes almacenadas fuera de la base

de datos y accesibles desde cualquier dispositivo. La Figura 4.23 presenta la vista desarrollada durante esta etapa. En ella se distinguen el formulario de importación a la izquierda, los filtros por estado y por etiqueta sobre la grilla, y las tarjetas de cada diseño con su estado, su zona del cuerpo, sus etiquetas y el tamaño del archivo.

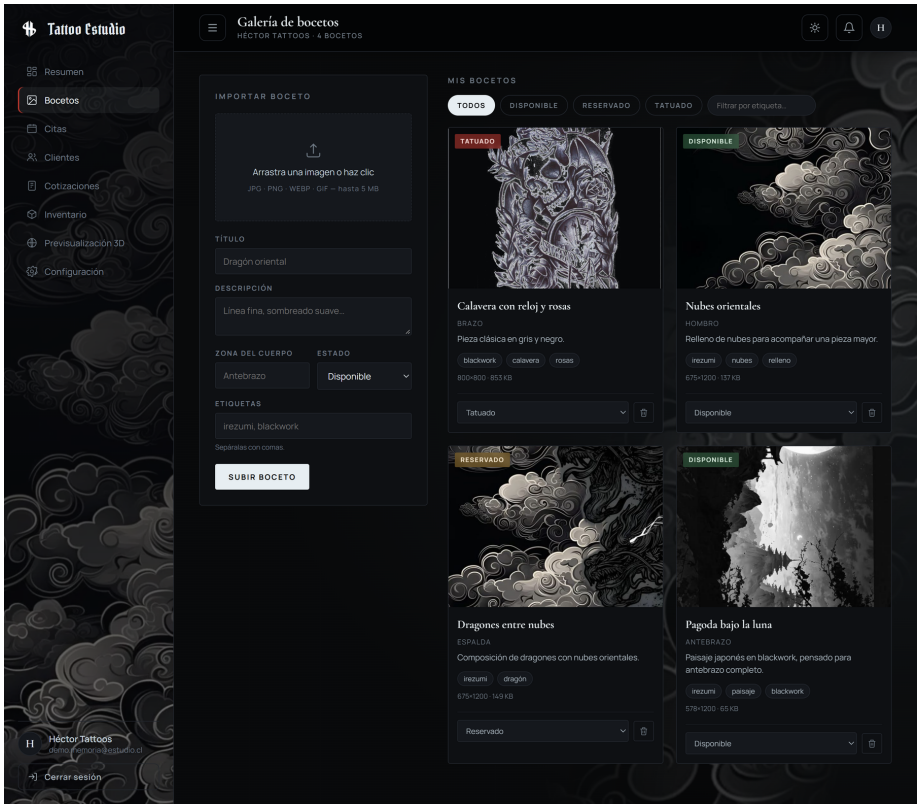


Figura 4.23: Vista “Galería de bocetos”.

Sistema de pruebas

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, usando Jest junto con Supertest para simular las peticiones HTTP y verificar las respuestas del servidor [62]. Las capas que tocan la base de datos y el almacenamiento se reemplazan por dobles de prueba, de manera que cada caso evalúa el comportamiento del controlador de forma aislada y sin depender de una base de datos levantada. La Tabla 4.8 detalla las pruebas implementadas.

Tabla 4.8: Pruebas unitarias implementadas en la Iteración 3

ID	**Función evaluada**	**Objetivo**	**Tests**
PU-06	`requireAuth`	Verificar que listar y subir bocetos sin token devuelven 401.	2
PU-07	`list()`	Validar que solo se devuelven los bocetos del usuario del token y que los filtros de estado y etiqueta llegan a la consulta, rechazando estados inválidos.	3
PU-08	`create()`	Verificar la subida completa, las validaciones de archivo y título, y la limpieza del archivo cuando falla la inserción.	5
PU-09	`update()`	Validar la edición de metadatos del boceto propio y el rechazo de un boceto ajeno o de un estado inválido.	3
PU-10	`remove()`	Verificar el borrado de la fila y del archivo con el driver que lo subió, y el rechazo de bocetos ajenos o de identificadores mal formados.	3

En total, durante esta iteración se ejecutaron 16 pruebas unitarias en un archivo de prueba, enfocadas en el control de acceso al recurso, las validaciones de la subida y la propiedad de los bocetos.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto de las operaciones implementadas. Se comprobó especialmente que un boceto perteneciente a otra cuenta no puede ser consultado, editado ni eliminado, y que el archivo almacenado se elimina junto con su registro. El resultado general de ejecución fue satisfactorio, obteniendo aprobación completa de los casos evaluados, como ilustran la Tabla 4.9

y la Figura 4.24.

Tabla 4.9: Resultado de ejecución de pruebas unitarias de la Iteración 3

Archivos evaluados	**Tests ejecutados**	**Resultado**
1	16	16 aprobados

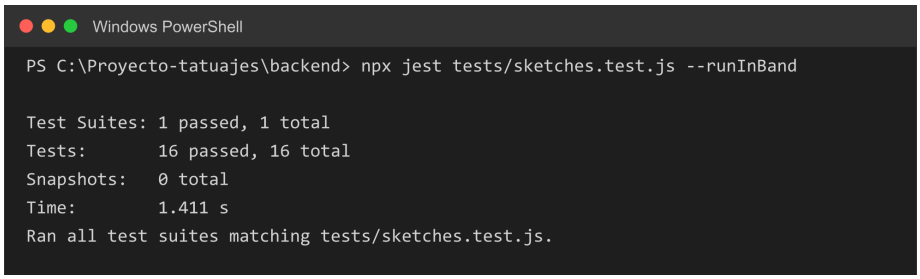


Figura 4.24: Resultado de la ejecución de las pruebas unitarias de la galería de bocetos.

Además de las pruebas automatizadas, se llevó a cabo una fase de pruebas manuales sobre la interfaz, evaluando los siguientes casos:

- Subir un boceto con título, zona del cuerpo y etiquetas.
- Subir un archivo que no es una imagen.
- Subir una imagen que supera el límite de tamaño.
- Subir un boceto sin completar el título.
- Filtrar la galería por estado y por etiqueta.
- Cambiar el estado de un boceto desde la galería.
- Eliminar un boceto y verificar que el archivo deja de estar disponible.
- Consultar la galería con una cuenta distinta y comprobar que no aparecen bocetos ajenos.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Las cuatro historias se completaron dentro de lo estimado y el trabajo resultó menor al previsto en la HU07, ya que las etiquetas se resolvieron con el mismo mecanismo de filtros del listado en lugar de requerir una estructura aparte. En contrapartida, la capa de almacenamiento con drivers intercambiables tomó más tiempo del considerado para la HU04, pero esa decisión evitó rehacer el módulo al momento del despliegue y dejó preparada la carga de las fotografías de sesión de iteraciones posteriores. Como aprendizaje del proceso, se incorporó la práctica de medir las propiedades de la imagen en el momento de subirla, anticipando necesidades de historias futuras sin agregar trabajo a la iteración.

4.3.4. Iteración 4

En esta iteración se implementan las historias de usuario HU11, HU12, HU13, HU25 y HU26. Las tres primeras levantan el inventario de insumos, que es lo que el tatuador gasta al trabajar; las dos últimas arman el panel de control con el que entra a la plataforma. Van juntas porque el panel es, en buena parte, la cara visible del inventario: ahí es donde el artista se entera de que algo se está acabando sin tener que ir a buscarlo.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 12 puntos según la Tabla 3.2.

- **HU11:** como tatuador quiero registrar un material con su cantidad y costo unitario, para mantener un control inicial de mi stock.
- **HU12:** como tatuador quiero modificar el stock manual o borrar insumos descontinuados, para mantener la realidad operativa de mi estudio.
- **HU13:** como tatuador quiero visualizar una advertencia cuando un insumo llegue a un nivel crítico, para comprar reposiciones a tiempo.
- **HU25:** como tatuador quiero ver un panel de control al iniciar sesión con las

citas programadas y alertas de stock bajo, para organizar mi jornada de un vistazo.

- **HU26:** como tatuador quiero visualizar una suma básica de los ingresos generados en el mes actual, para evaluar la rentabilidad del estudio.

El costo unitario que pide la HU11 no se usa todavía en esta iteración: es el dato con el que la Iteración 5 calcula el costo de insumos de una cotización, de modo que el inventario se diseñó desde el principio pensando en ese uso posterior.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.10 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.10: Tareas asociadas a la Iteración 4

Historia	**Tarea**	**Descripción**
HU11	T0001	Crear diagrama de secuencia
HU11	T0002	Crear la tabla de insumos y la ruta de registro
HU11	T0003	Crear el formulario de insumo en la vista de inventario
HU12	T0001	Crear diagrama de secuencia
HU12	T0002	Crear la ruta de ajuste de stock
HU12	T0003	Crear las rutas de edición y eliminación
HU12	T0004	Agregar los botones de ajuste en la vista
HU13	T0001	Crear diagrama de secuencia
HU13	T0002	Marcar el nivel crítico en el listado y en el resumen
HU13	T0003	Crear el aviso de nivel crítico para la campanita
HU25	T0001	Crear diagrama de secuencia
HU25	T0002	Crear el panel de stock bajo en el resumen
HU25	T0003	Mostrar el calendario del mes y las próximas citas
HU26	T0001	Crear la ruta de ingresos del mes
HU26	T0002	Mostrar lo cobrado y lo por cobrar en el resumen
HU11–HU13, HU25, HU26	T0009	Creación y testeo de pruebas unitarias
HU11–HU13, HU25, HU26	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario

asociadas a la iteración. La Figura 4.25 ilustra el descuento de stock que dispara la advertencia de nivel crítico, que es el flujo de la iteración en el que intervienen más partes: la vista, la base de datos y el servicio de avisos. El registro de un insumo y la carga del panel de control siguen el recorrido habitual de la plataforma.

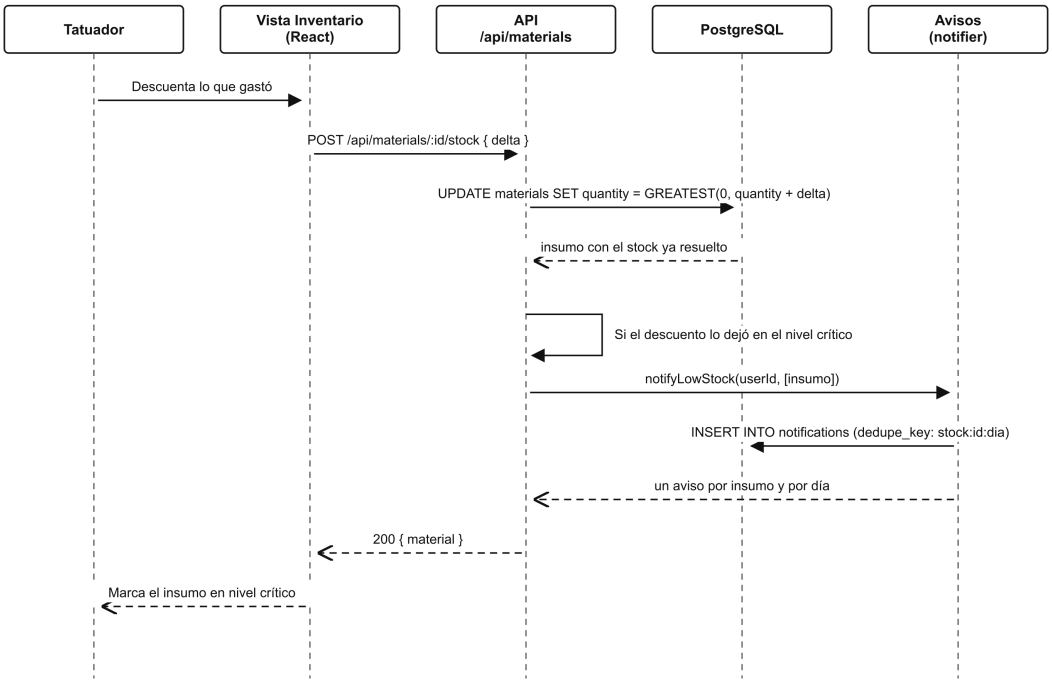


Figura 4.25: Diagrama de secuencia HU12 y HU13: descuento de stock y advertencia de nivel crítico.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. En el backend se levanta el recurso `/api/materials` con el registro, la consulta filtrada, la edición, la eliminación y el ajuste de stock, con las mismas reglas de propiedad de las iteraciones anteriores: todo filtra por el identificador del tatuador y un insumo ajeno responde 404.

Un insumo se define por la unidad en que se cuenta y por dos cantidades enteras: el stock actual y el nivel crítico. Esa segunda cantidad es la que convierte el inventario en una herramienta de aviso y no solo en una lista: un insumo está en nivel crítico cuando su stock es menor o igual a su mínimo, y el listado devuelve esos insumos primero, de modo que lo urgente aparece arriba sin que el artista tenga que buscarlo.

El ajuste de stock no se calcula en el servidor para después guardarlo, sino que se resuelve dentro de la misma sentencia en la base de datos, como muestra la Figura 4.26. Así dos descuentos seguidos no se pisan —cada uno parte del valor que dejó el anterior— y el stock nunca queda bajo cero aunque se descuente más de lo que había.

backend/src/models/materialModel.js

```
// Suma `delta` al stock (negativo para descontar consumo). Se resuelve en
// la base con GREATEST para que dos ajustes seguidos no se pisen y el
// stock nunca quede bajo cero.
async function adjustStock(id, userId, delta) {
  const { rows } = await pool.query(
    `UPDATE materials
        SET quantity = GREATEST(0, quantity + $3), updated_at = now()
      WHERE id = $1 AND user_id = $2
      RETURNING ${COLUMNS}`,
    [id, userId, delta]
  );
  return rows[0] ?? null;
}
```

Figura 4.26: Ajuste de stock resuelto en la base de datos.

La advertencia de la HU13 se emite cuando un descuento deja al insumo en su nivel crítico, nunca al reponerlo. Para que el artista no reciba un aviso por cada descuento mientras gasta lo poco que le queda, cada insumo avisa como mucho una vez por día, usando una clave que combina el insumo con la fecha en hora de Chile; al día siguiente, si sigue sin reponer, se le vuelve a recordar. El aviso queda en la campanita de la aplicación y, si el artista tiene las notificaciones activadas, también llega al teléfono.

En cuanto al panel de control, la HU26 pide una suma de ingresos del mes, y la decisión de diseño está en qué fecha se considera. Lo cobrado se acumula por la fecha en que la sesión se marcó pagada, no por la fecha de la cita: si un cliente paga en noviembre una sesión de octubre, ese dinero entra en noviembre, que es cuando efectivamente ingresó. Lo pendiente, en cambio, se cuenta por la fecha de la cita, porque responde a otra pregunta: cuánto falta por cobrar de lo que está agendado este mes. La Figura 4.27 muestra cómo se obtienen ambas cifras en una sola consulta.

backend/src/models/sessionModel.js

```
async function incomeByUser(userId, { from, to }) {
  const { rows } = await pool.query(
    `SELECT
       COALESCE(SUM(price) FILTER (
         WHERE paid AND status <> 'cancelada' AND paid_at >= $2 AND paid_at < $3
       ), 0)::bigint AS collected,
       COALESCE(SUM(price) FILTER (
         WHERE NOT paid AND status IN ('agendada', 'completada')
           AND starts_at >= $2 AND starts_at < $3
       ), 0)::bigint AS pending
       FROM project_sessions
      WHERE user_id = $1`,
    [userId, from, to]
  );
}
```

Figura 4.27: Cobrado y pendiente del mes, resueltos en una sola consulta.

En el frontend se desarrollaron dos vistas. La de inventario presenta los insumos con su stock, su nivel crítico, su costo unitario y su regla de consumo, con botones para sumar o descontar unidades sin salir de la lista, filtros por categoría y un atajo para ver solo lo que está en nivel crítico. La de resumen reúne el calendario del mes con las citas marcadas, las próximas citas, el panel de stock bajo y las cifras del mes.

Resultado del desarrollo

Como resultado de esta iteración, el tatuador tiene el control de sus insumos dentro de la misma plataforma donde lleva su agenda, y una pantalla de entrada que le dice en qué está su mes sin que tenga que entrar a buscarlo. Las Figuras 4.28 y 4.29 presentan las vistas desarrolladas durante esta etapa.

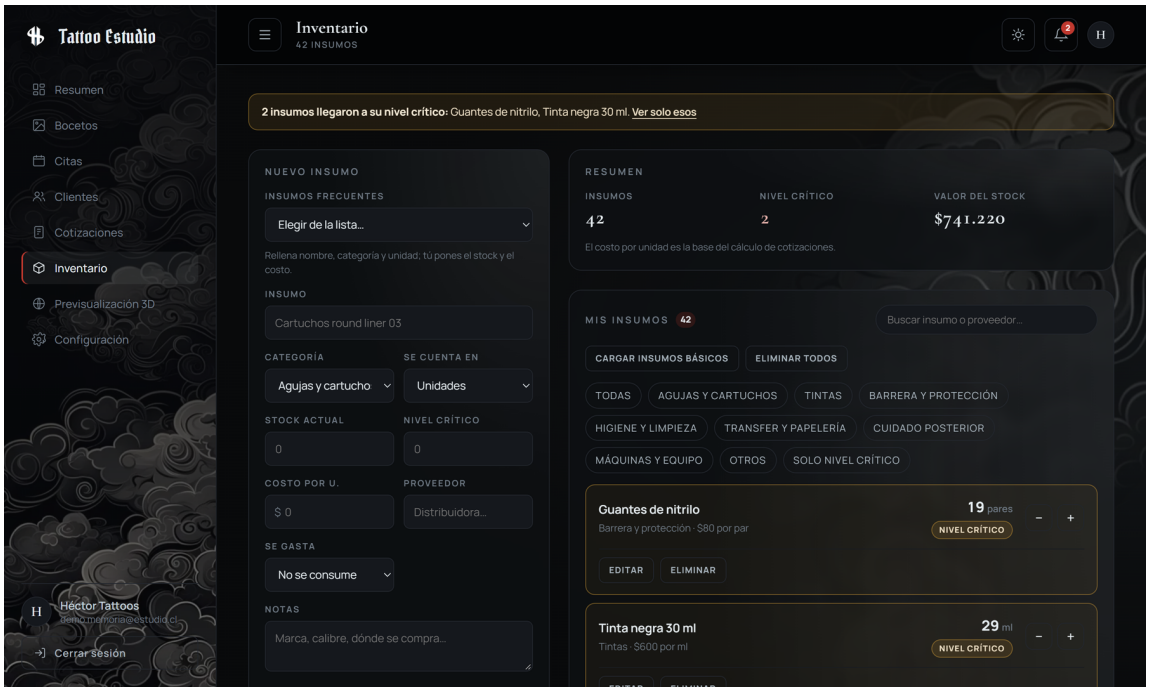


Figura 4.28: Vista “Inventario”, con la advertencia de los insumos en nivel crítico, el resumen del stock y los insumos críticos primero en la lista.

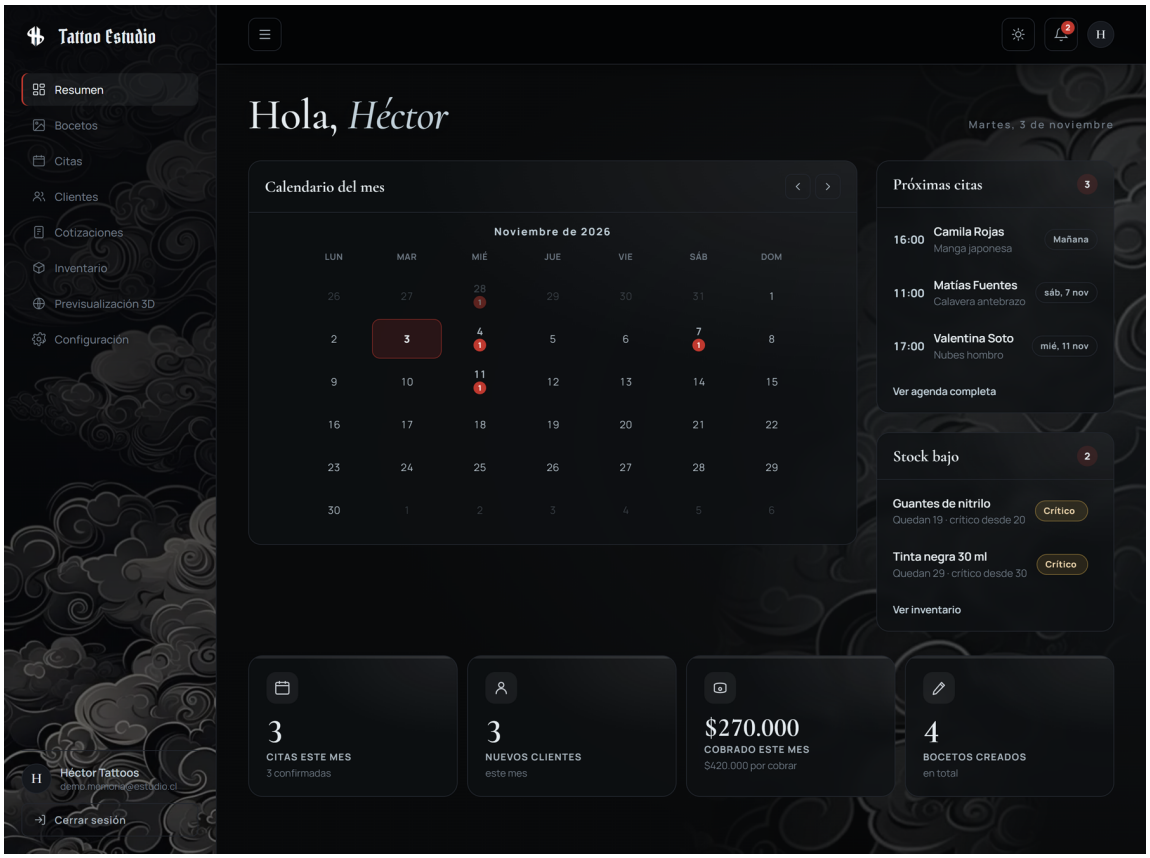


Figura 4.29: Vista "Resumen": calendario del mes, próximas citas, panel de stock bajo y las cifras de lo cobrado y lo por cobrar.

Sistema de pruebas

Después de la implementación se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Registrar un insumo con unidad, stock, nivel crítico y costo.
- Registrar un insumo sin nombre o con una cantidad con decimales.
- Descontar stock hasta llegar al nivel crítico.
- Descontar más stock del que hay disponible.
- Reponer stock de un insumo que estaba en nivel crítico.
- Filtrar el inventario por categoría y ver solo los insumos críticos.

Tabla 4.11: Pruebas unitarias implementadas en la Iteración 4

ID	Función evaluada	Objetivo	Tests
PU-11	`POST /materials`, `GET /materials`	Validar el registro de un insumo, el rechazo de categorías, unidades o cantidades inválidas, y los filtros del listado.	6
PU-12	`POST /materials/ :id/stock`	Comprobar el ajuste con un delta y el rechazo de uno nulo o no numérico.	2
PU-13	`DELETE /materials/:id`	Verificar la eliminación y el rechazo de un insumo ajeno o de un identificador mal formado.	3
PU-14	`POST /materials/ defaults`	Validar la carga del catálogo básico sin duplicar lo existente y el vaciado con confirmación explícita.	6
PU-15	`ensure StarterKit()`	Comprobar que el inventario inicial se entrega una sola vez y que respeta un inventario ya armado.	5
PU-16	`notifyLow Stock()`	Verificar que avisa al llegar al mínimo y que no repite el aviso el mismo día de Chile.	5
PU-17	`GET /sessions/ income`	Comprobar la suma de lo cobrado y lo pendiente del artista del token.	2

- Eliminar un insumo descontinuado.
- Entrar al panel de control y comprobar las citas del mes, el stock bajo y lo cobrado.
- Marcar una sesión como pagada y ver el monto reflejado en el mes.

Pruebas unitarias

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, con Jest y Supertest y con la base de datos reemplazada por dobles de prueba. La Tabla 4.11 detalla las pruebas implementadas.

En total, durante esta iteración se ejecutaron 29 pruebas unitarias repartidas en cuatro archivos de prueba. Los dos propios del inventario reúnen 22 de ellas; las siete restantes acompañan a las pruebas de avisos y de agenda, porque evalúan funciones que esas historias comparten.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto del registro, del ajuste de stock y de la advertencia de nivel crítico. Se comprobó especialmente que el stock no queda negativo por descontar de más, que estar justo en el mínimo ya cuenta como nivel crítico, y que un segundo descuento el mismo día no genera un aviso repetido. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.12 y muestra la Figura 4.30.

Tabla 4.12: Resultado de ejecución de pruebas unitarias de la Iteración 4

Archivos evaluados	Tests ejecutados	Resultado
4	29	29 aprobados

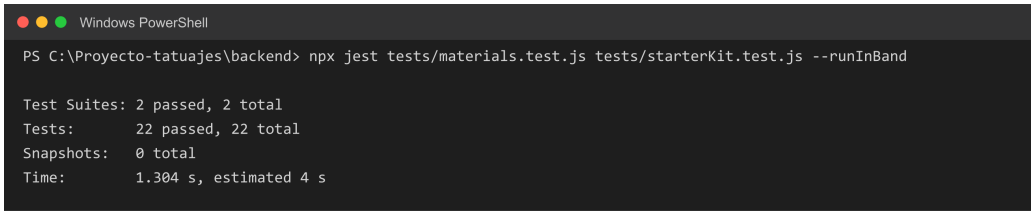


Figura 4.30: Ejecución de los dos archivos de prueba propios del inventario, que reúnen 22 de las 29 pruebas de la iteración.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Las cinco historias se completaron dentro de lo estimado. La HU25 resultó más barata de lo previsto porque reutilizó la consulta de citas que ya existía de la agenda, y lo que costó más fue decidir la regla de la HU26: la primera versión sumaba por la fecha de la cita y mostraba como ingreso de un mes dinero que aún no se había cobrado, así que se agregó la fecha de pago a cada sesión para poder separar ambas cosas. Como aprendizaje,

quedó claro que conviene fijar desde el principio qué fecha representa cada cifra que se muestra al artista, porque corregirlo después obliga a tocar el modelo de datos.

4.3.5. Iteración 5

En esta iteración se implementan las historias de usuario HU14, HU15 y HU16, que son el cálculo automatizado del costo de insumos de un tatuaje, el ajuste manual de ese cálculo y el descuento del inventario al dar por terminada una sesión. Las tres se apoyan en lo construido antes: la cobertura de tinta y los colores que se midieron del boceto en la Iteración 3, y el inventario con costos unitarios y reglas de consumo de la Iteración 4. La HU27, asignada también a esta iteración, se descartó; la decisión se explica más abajo y se retoma en la retrospectiva.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 15 puntos según la Tabla 3.2.

- **HU14:** como tatuador quiero que el sistema calcule el costo de los insumos basándose en el tamaño del boceto y variables de trazado, para obtener un presupuesto automatizado.
- **HU15:** como tatuador quiero agregar o quitar materiales específicos sugeridos por el algoritmo, para refinar el cálculo final de la cotización.
- **HU16:** como tatuador quiero que al dar por finalizada una sesión cotizada los materiales se descuenten solos, para automatizar el control de stock.
- **HU27:** como tatuador quiero definir mis días y horas de trabajo, para que el calendario bloquee automáticamente mis días libres.

La HU27 se revisó con el tatuador antes de programarla y se decidió no implementarla, de modo que el esfuerzo efectivo de la iteración fue de 13 puntos. La razón es que no describe cómo trabaja el estudio: el tatuador no tiene días libres fijos, atiende fuera de su horario habitual cuando al cliente le acomoda, y un calendario que bloqueara esos horarios le habría impedido agendar citas que sí quería tomar. La necesidad concreta detrás de la historia, que es no comprometer dos veces la misma hora, se resolvió en la Iteración 6 con una comprobación de solapamiento entre citas,

que actúa sobre las horas realmente ocupadas en lugar de sobre un horario declarado de antemano.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.13 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.13: Tareas asociadas a la Iteración 5

Historia	**Tarea**	**Descripción**
HU14	T0001	Crear diagrama de secuencia
HU14	T0002	Crear las tablas de cotización y de su detalle
HU14	T0003	Crear el motor de cálculo con las variables de trazado
HU14	T0004	Repartir el área entintada entre las tintas del diseño
HU14	T0005	Crear la ruta de estimación que calcula sin guardar
HU14	T0006	Crear la vista de cotizaciones con el cálculo en vivo
HU15	T0001	Crear diagrama de secuencia
HU15	T0002	Crear las rutas de guardado y de ajuste del detalle
HU15	T0003	Permitir agregar, corregir y quitar líneas en la vista
HU16	T0001	Crear diagrama de secuencia
HU16	T0002	Crear el descuento de stock dentro de una transacción
HU16	T0003	Enganchar el descuento al cierre de la sesión
HU27	T0001	Descartada antes de programarla (ver Retrospectiva)
HU14–HU16	T0009	Creación y testeo de pruebas unitarias
HU14–HU16	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. La Figura 4.31 ilustra el cálculo de una cotización, donde además de la base de datos interviene el motor de cálculo. El ajuste manual del detalle y el descuento de inventario al cerrar una sesión se describen en la implementación, con el código que los resuelve.

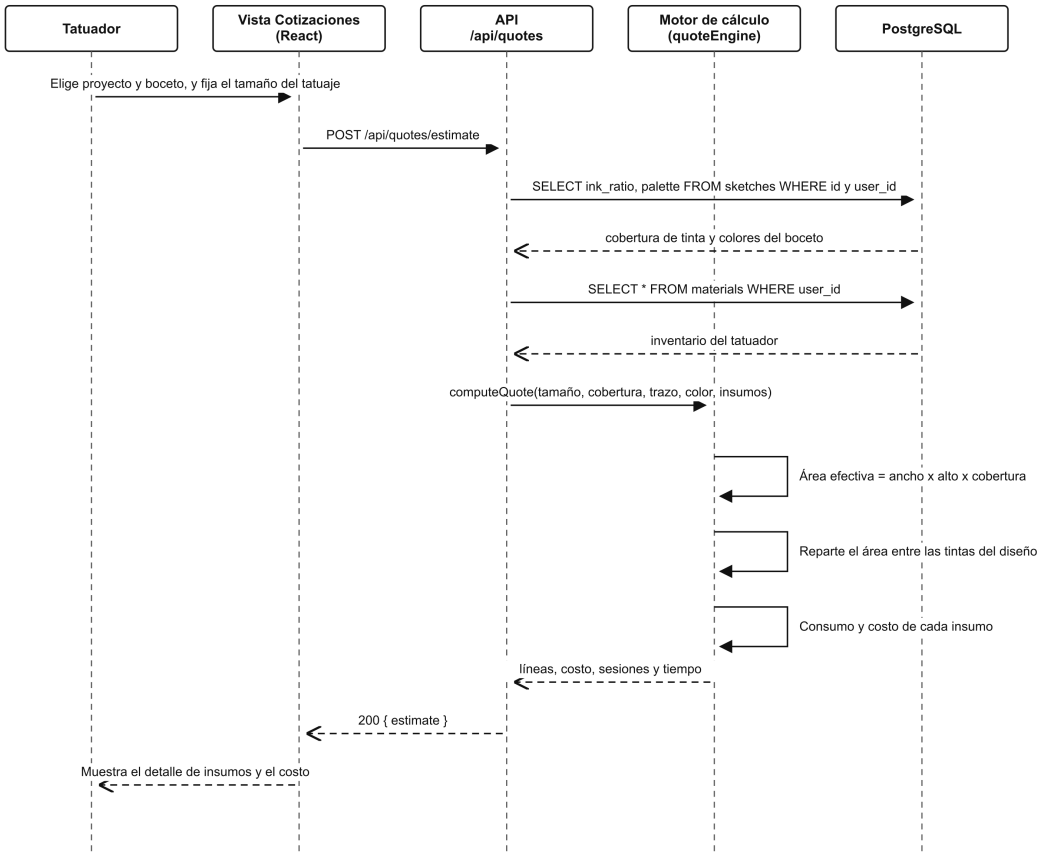


Figura 4.31: Diagrama de secuencia HU14: cálculo del costo de insumos de un tatuaje.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. En el backend se levanta el recurso `/api/quotes` con el cálculo sin guardar, el guardado, el ajuste, la eliminación y el descuento de stock, con las mismas reglas de propiedad de las iteraciones anteriores.

Lo primero que hubo que acotar es qué calcula la plataforma. La cotización entrega el **costo objetivo de los insumos** que gasta un tatuaje, no el precio que

el artista cobra por su trabajo: ese precio depende de su experiencia, de su estilo y de lo que decida cobrar, y sigue viviendo en el proyecto como un dato que él escribe. Separar ambas cosas fue lo que permitió automatizar la primera sin pretender automatizar la segunda.

El cálculo parte del área que de verdad se va a entintar, como muestra la Figura 4.32. El rectángulo del boceto por sí solo miente: un lettering fino y un diseño macizo del mismo tamaño gastan cosas muy distintas. Por eso el área del rectángulo se pondera por la cobertura de tinta que se midió del archivo al subirlo, y sobre esa área efectiva se calcula todo lo demás. Las horas de aguja salen de dividir esa área por un rendimiento en centímetros cuadrados por hora, calibrado con los tiempos que maneja el oficio y corregido por las variables de trazado.

backend/src/services/quoteEngine.js

```
// El rectángulo del boceto miente: un lettering fino y un blackwork
// macizo de 10x10 cm gastan cosas muy distintas. Por eso el área se
// pondera por la cobertura de tinta medida sobre los píxeles del PNG.
const boundingArea = widthCm * heightCm;
const effectiveArea = round3(boundingArea * inkRatio);

// Horas de aguja del proyecto completo, corregidas por trazo y color.
const workHours = (effectiveArea / CM2_POR_HORA) * strokeFactor.time * colorFactor.time;

// Las sesiones se deducen del trabajo que pide la pieza; el artista solo
// las fija si ya las acordó con el cliente.
const sessions = Math.min(
  MAX_SESIONES,
  Math.max(1, sessionsCount ?? Math.ceil(workHours / HORAS_POR_SESION))
);

// La preparación de cada cita es lo que explica que una pieza chica
// nunca sea tan rápida como sugeriría su área.
const rawMinutes = workHours * 60 + MINUTOS_PREPARACION * sessions;
```

Figura 4.32: Área efectiva, horas de aguja y sesiones deducidas del trabajo que pide la pieza.

Las variables de trazado que pide la HU14 son el grosor del trazo y el modo de color, y mueven el material y el tiempo en sentidos distintos. Un trazo grueso come más tinta pero avanza más rápido por centímetro cuadrado; uno fino es de detalle y ocurre al revés. En la misma pieza de prueba de 12 por 18 centímetros, el trazo fino gasta 1,489 ml de tinta en 2 horas 9 minutos de aguja, el medio 1,861 ml en 1 hora 43 minutos y el grueso 2,419 ml en 1 hora 28 minutos.

Un segundo punto de diseño fue cómo se cobran las tintas. La primera versión cotizaba todas las tintas del inventario por el área completa del diseño, de modo que un tatuaje en negro salía pagando también el rojo y el amarillo, y agregar tintas al inventario encarecía cualquier cotización. La regla que corrigió el problema es que el área entintada se **reparte** entre las tintas y nunca se multiplica por cuántas haya guardadas: los colores que se midieron del boceto se emparejan con las tintas que el artista declaró, y cada una recibe solo la parte del diseño que lleva su color. Un color del diseño que ninguna tinta del inventario cubre no desaparece del cálculo ni se reparte entre las demás: se cotiza con una tinta de referencia y se muestra marcado como que hay que comprarla, porque el tatuaje la va a necesitar igual.

El consumo de cada insumo se resuelve según la base que declaró en el inventario, como muestra la Figura 4.33. Lo que escala con el área lleva además un piso de una unidad por sesión, que no es un redondeo de conveniencia: un cartucho y la tinta servida en la copita son estériles y se botan al terminar, no se guardan para el cliente siguiente, así que una pieza chica no puede cotizarse en centésimas de cartucho.

backend/src/services/quoteEngine.js

```
function consumption(material, { effectiveArea, sessionsCount, hours,
                                 materialFactor, share }) {
  const rate = Number(material.consumption_rate);
  if (!rate || rate <= 0) return 0;

  switch (material.consumption_basis) {
    // Tinta y agujas: lo único que escala con el tamaño del tatuaje.
    // `share` es la parte del diseño que lleva el color de esta tinta.
    // El piso no es un redondeo: un cartucho y la tinta servida son
    // estériles y se botan al terminar la sesión.
    case 'area': {
      const area = effectiveArea * share;
      return round3(Math.max(sessionsCount, rate * area * materialFactor));
    }
    // Guantes, film, papel transfer: una vez por sesión.
    case 'sesion':
      return round3(rate * sessionsCount);
    // Gasas, alcohol, jabón: acompañan el tiempo que dure el trabajo.
    case 'hora':
      return round3(rate * hours);
    default:
      return 0;
  }
}
```

Figura 4.33: Consumo de cada insumo según la base declarada en el inventario.

La HU15 se resolvió guardando el detalle como una copia congelada. Cada línea de la cotización conserva el nombre, la unidad y el costo del insumo tal como estaban al cotizar, de modo que si mañana sube el precio de la tinta la cotización que ya se le pasó al cliente no cambia sola. El artista puede corregir cantidades, quitar lo que no va a usar y agregar insumos que el algoritmo no consideró, pero el costo total lo recalcula siempre el servidor a partir de las líneas: una cifra enviada desde el navegador se ignora.

La HU16 descuenta el inventario cuando la sesión se marca como completada, y su exigencia principal es no descontar dos veces. Volver a guardar una sesión ya cerrada, un segundo clic o dos pestañas abiertas no pueden vaciar el stock dos veces por el mismo trabajo. Como muestra la Figura 4.34, el descuento ocurre dentro de una transacción que empieza sellando la cotización con un UPDATE condicionado a que ese sello siga vacío: si otra ejecución llegó antes, ese UPDATE no afecta ninguna fila, la transacción se corta y el inventario queda intacto. Si el descuento falla por cualquier motivo, la sesión igual queda cerrada y el artista puede ajustar el stock a mano, porque perder el registro de la cita por un problema del inventario sería peor que el problema mismo.

backend/src/models/quoteModel.js

```
// El sello `consumed_at` se pone con un UPDATE condicionado a que siga
// nulo: si la sesión se cierra dos veces, el segundo UPDATE no afecta
// ninguna fila y el stock no se descuenta de nuevo.
const { rows: marked } = await client.query(
  `UPDATE quotes SET consumed_at = now(), updated_at = now()
     WHERE id = $1 AND user_id = $2 AND consumed_at IS NULL
     RETURNING ${FIELDS}`,
  [id, userId]
);
if (!marked[0]) { await client.query('ROLLBACK'); return null; }

// GREATEST(0, ...) igual que el ajuste manual: el inventario nunca
// queda negativo. Solo se tocan insumos vivos y del mismo dueño.
const { rows: affected } = await client.query(
  `UPDATE materials m
      SET quantity = GREATEST(0, m.quantity - c.used), updated_at = now()
     FROM (SELECT material_id, round(sum(quantity))::int AS used
             FROM quote_items
            WHERE quote_id = $1 AND material_id IS NOT NULL
            GROUP BY material_id) c
    WHERE m.id = c.material_id AND m.user_id = $2
    RETURNING m.id, m.name, m.quantity, m.min_quantity, c.used`,
  [id, userId]
);
```

Figura 4.34: Descuento del inventario en una transacción, con el sello que lo hace irrepetible.

En el frontend se desarrolló la vista de cotizaciones, que estima en vivo contra el servidor mientras el artista ajusta los campos y recién guarda cuando él lo decide. La página se abre desde el proyecto, con el boceto y el tamaño ya precargados, y el cálculo no se hace nunca en el navegador: la pantalla solo muestra lo que el servidor devuelve.

Resultado del desarrollo

Como resultado de esta iteración, el tatuador puede saber cuánto material le cuesta un trabajo antes de cotizarlo, ajustar ese cálculo con lo que él sabe de su propia forma de trabajar, y dejar que el inventario se descuente solo cuando la sesión termina. Las Figuras 4.35 y 4.36 presentan las vistas desarrolladas durante esta etapa. En el ejemplo, una pieza de 12 por 18 centímetros con un 20 % de cobertura de tinta da 43 centímetros cuadrados efectivos, 2 horas 15 minutos de trabajo en una sesión y un costo de insumos de $9.983.

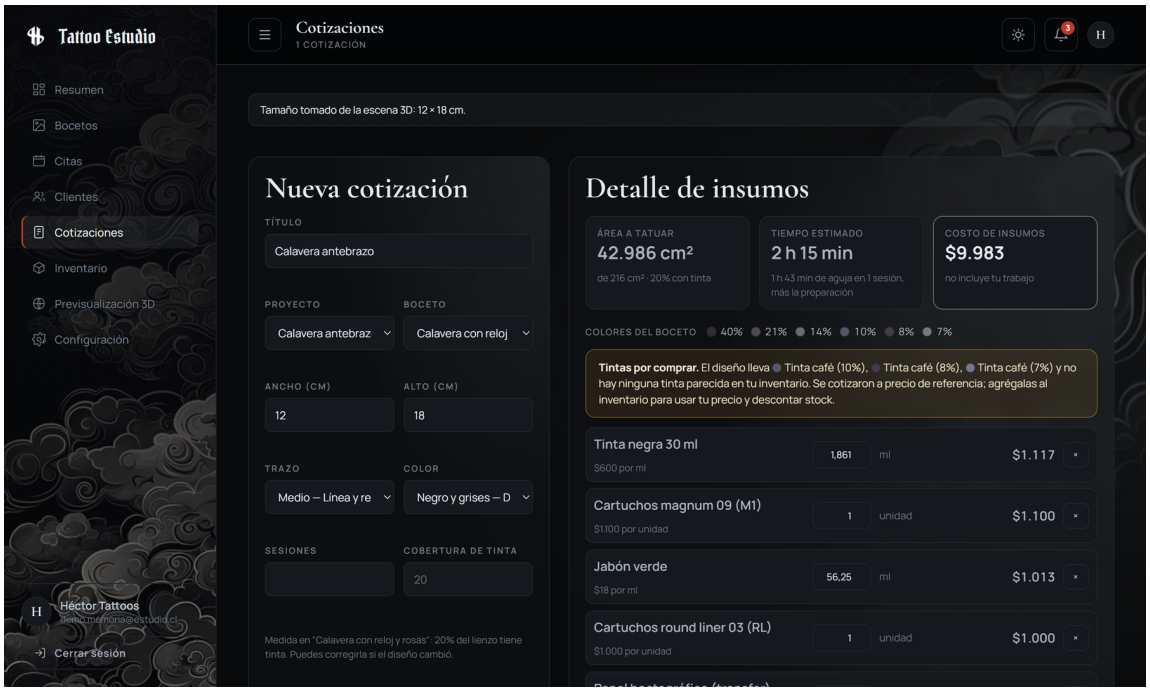


Figura 4.35: Vista “Cotizaciones”: parámetros del tatuaje a la izquierda y, a la derecha, el área a tatuar, el tiempo estimado, el costo de insumos, los colores del boceto y el detalle editable.

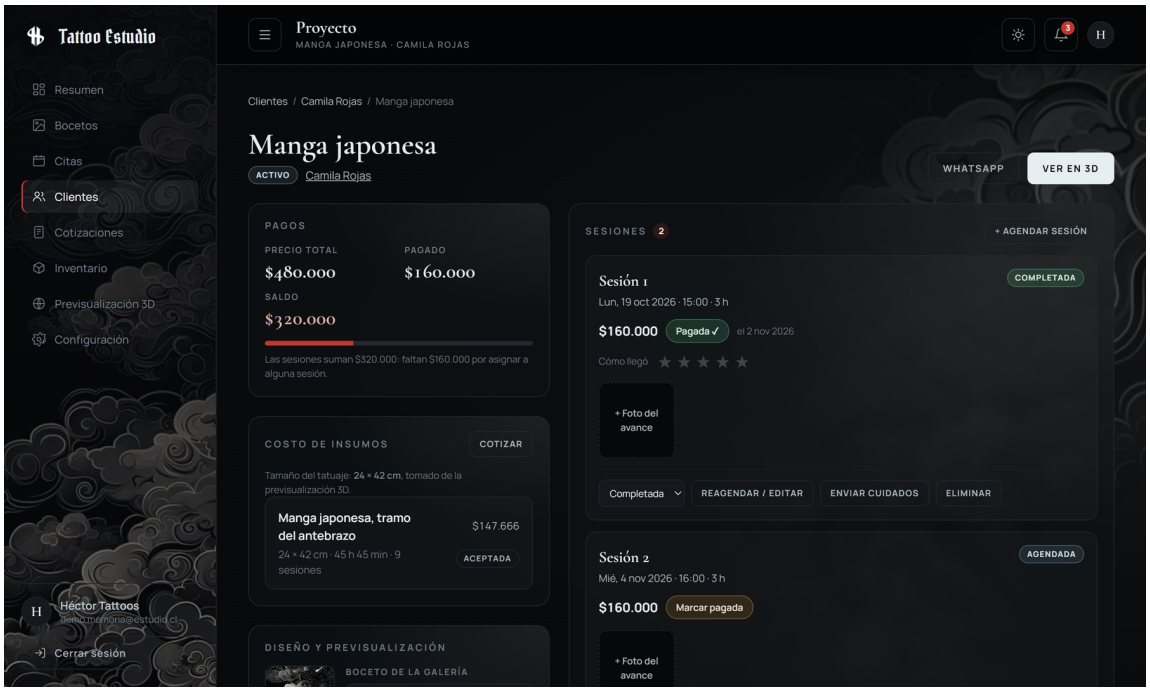


Figura 4.36: Vista “Proyecto”: el costo de insumos vive en su propio panel, separado de lo que el artista cobra por su trabajo.

Sistema de pruebas

Después de la implementación se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Cotizar un boceto medido y comprobar el área efectiva que resulta.
- Cotizar la misma pieza con trazo fino, medio y grueso.
- Cotizar un diseño a color y revisar el reparto entre las tintas.
- Cotizar un diseño con un color que no está en el inventario.
- Quitar una línea y corregir la cantidad de otra antes de guardar.
- Guardar la cotización y volver a abrirla desde el proyecto.
- Intentar modificar una cotización que ya descontó el stock.
- Marcar como completada una sesión cotizada y revisar el inventario.

- Volver a guardar esa misma sesión y comprobar que el stock no baja de nuevo.

Pruebas unitarias

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, con Jest y Supertest y con la base de datos reemplazada por dobles de prueba. La Tabla 4.14 detalla las pruebas implementadas.

Tabla 4.14: Pruebas unitarias implementadas en la Iteración 5

ID	Función evaluada	Objetivo	Tests
PU-18	`computeQuote()`	Validar el área efectiva, las bases de consumo, las variables de trazado y el tiempo.	25
PU-19	`computeQuote()`	Verificar que el área entintada se reparte entre las tintas y nunca se multiplica.	17
PU-20	`matchInk()`	Validar el emparejamiento de un color con la tinta más parecida y los grises.	11
PU-21	`allocate Palette()`	Comprobar que las partes del área suman 1, con paleta medida y por modo de color.	14
PU-22	`POST /quotes/ estimate`	Verificar el cálculo sin guardar y la cobertura tomada del boceto.	7
PU-23	`POST /quotes`	Validar el guardado y el rechazo de referencias ajenas o sin token.	8
PU-24	`PATCH /quotes/:id`	Comprobar el reemplazo de las líneas y el bloqueo de una cotización consumida.	6
PU-25	`POST /quotes/ :id/consume`	Verificar el descuento de stock y que no se repite la segunda vez.	2

En total, durante esta iteración se ejecutaron 90 pruebas unitarias repartidas en tres archivos de prueba. La mayoría se concentra en el motor de cálculo y en el emparejamiento de color, que son la parte del sistema donde un error no se nota a simple vista: una cotización equivocada igual se ve bien en pantalla.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto del cálculo, del ajuste manual y del descuento de inventario. Se comprobó especialmente que las partes del área repartidas entre las tintas suman siempre uno, de modo que agregar tintas al inventario no encarece un diseño en negro; que repartir la misma pieza en más citas no agranda el tatuaje ni su costo; y que descontar dos veces la misma cotización deja el inventario intacto la segunda vez. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.15 y muestra la Figura 4.37.

Tabla 4.15: Resultado de ejecución de pruebas unitarias de la Iteración 5

Archivos evaluados	**Tests ejecutados**	**Resultado**
3	90	90 aprobados

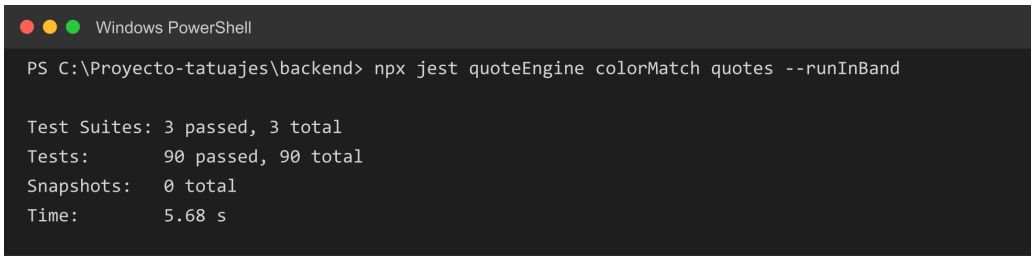


Figura 4.37: Ejecución de los tres archivos de prueba de la cotización automatizada.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Las tres historias implementadas se completaron, pero la HU14 tomó bastante más de lo estimado: calcular el costo era la parte fácil, y lo que costó fue el reparto de las tintas por color, que hubo que rehacer después de descubrir que un tatuaje en negro salía cotizando todas las

tintas del inventario. La HU16, en cambio, resultó más barata de lo previsto porque el descuento se apoyó en el ajuste de stock que ya existía de la Iteración 4. La HU27 se descartó antes de programarla, lo que liberó sus dos puntos de esfuerzo.

Quedaron dos aprendizajes. El primero es que un error de cálculo no se delata solo: la pantalla muestra una cifra con el mismo aspecto esté bien o mal, y fueron las pruebas sobre el motor las que dejaron en evidencia el problema del reparto, no el uso de la aplicación. El segundo es que conviene revisar con el usuario si una historia describe su trabajo real antes de programarla; la HU27 estaba escrita desde una idea razonable de cómo debería funcionar un calendario, pero no desde cómo este estudio toma sus citas.

4.3.6. Iteración 6

En esta iteración se implementan las historias de usuario HU17, HU18, HU19, HU20, HU24 y HU30, que en conjunto forman la agenda del estudio: a quién se atiende, qué se le está tatuando, cuándo, cómo quedó y qué tan cumplidor resultó. Es la parte de la plataforma que reemplaza directamente al cuaderno y a las conversaciones de Instagram con las que el tatuador llevaba sus citas hasta ahora.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 20 puntos según la Tabla 3.2.

- **HU17:** como tatuador quiero agendar a un cliente especificando fecha, hora, número de contacto y precio del proyecto, para organizar mis tiempos.
- **HU18:** como tatuador quiero ver mis citas en un formato de calendario mensual y semanal, para organizar visualmente mi jornada.
- **HU19:** como tatuador quiero poder crear distintos proyectos para clientes específicos junto con sus sesiones.
- **HU20:** como tatuador quiero poder subir una foto de cada sesión para ver los progresos del proyecto.
- **HU24:** como tatuador quiero ver el historial de todas las sesiones pasadas y futuras asociadas a un cliente, para hacer seguimiento de piezas grandes.

- **HU30:** como tatuador quiero poder visualizar un marcador de todos mis clientes y asignarles puntaje para saber su responsabilidad de llegada y poder filtrarlos de acuerdo a ello.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.16 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.16: Tareas asociadas a la Iteración 6

Historia	**Tarea**	**Descripción**
HU17	T0001	Crear diagrama de secuencia
HU17	T0002	Crear las tablas de cliente, proyecto y sesión
HU17	T0003	Crear las rutas de cliente y de sesión
HU17	T0004	Impedir que se agenden dos citas a la misma hora
HU18	T0001	Crear el calendario del mes con las citas marcadas
HU18	T0002	Crear el detalle del día elegido
HU19	T0001	Crear la ruta de proyectos y su resumen de pagos
HU19	T0002	Crear cliente y proyecto desde la misma pantalla
HU20	T0001	Crear diagrama de secuencia
HU20	T0002	Crear la ruta de fotos de sesión
HU20	T0003	Mostrar las fotos junto a cada sesión
HU24	T0001	Crear la ficha del cliente con su historial
HU30	T0001	Crear diagrama de secuencia
HU30	T0002	Calificar cada sesión de 1 a 5
HU30	T0003	Calcular el puntaje del cliente y ordenar la lista
HU30	T0004	Filtrar los clientes por su responsabilidad
HU17–HU20, HU24, HU30	T0009	Creación y testeo de pruebas unitarias
HU17–HU20, HU24, HU30	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. La Figura 4.38 ilustra el agendado de una cita con la revisión del horario, que es la parte de la agenda en la que el orden de los pasos importa. La subida de una foto repite el flujo de la galería de bocetos, y la calificación de una sesión es una actualización más.

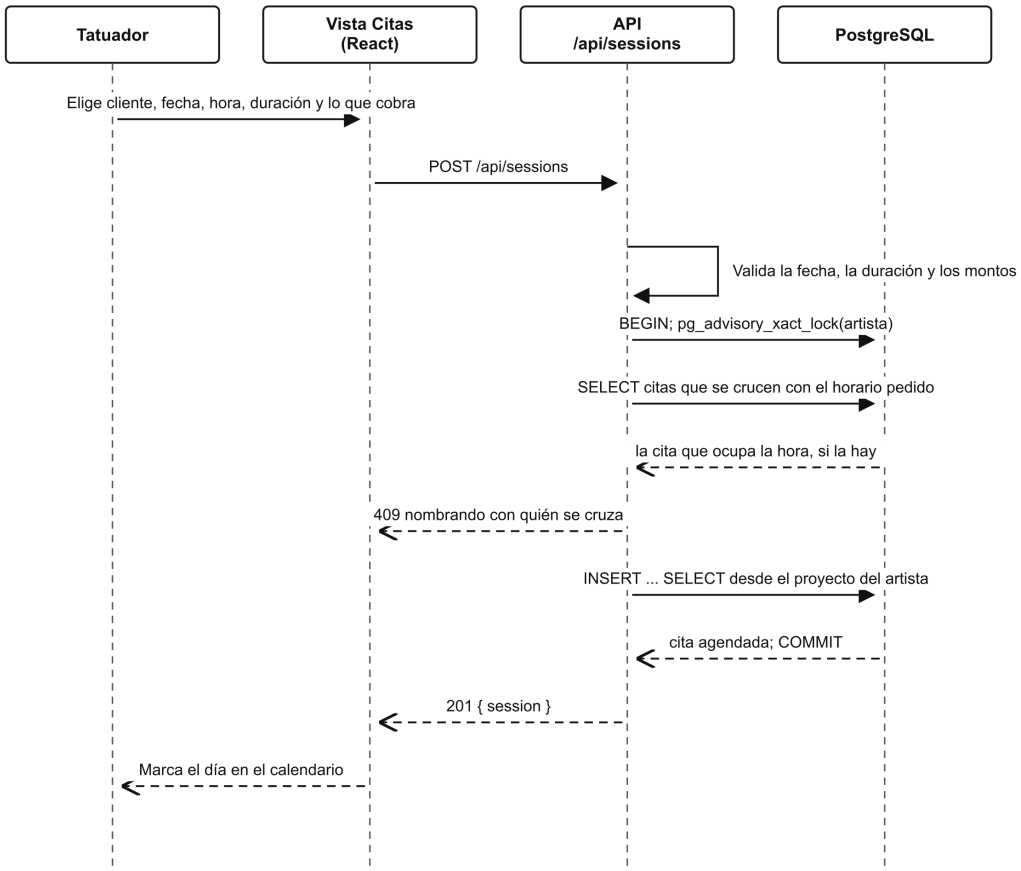


Figura 4.38: Diagrama de secuencia HU17: agendado de una cita con revisión del horario.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. El modelo de datos de la agenda es una cadena de cuatro niveles: un cliente tiene proyectos, un proyecto tiene sesiones y una sesión tiene fotos. La decisión que ordena todo lo demás es que **cada sesión es una cita**: lleva su fecha, su hora, su duración

y lo que se cobra en ella. Un proyecto grande no es una cita larga, sino varias, y por eso el precio total del trabajo y lo que se cobra en cada visita son dos cifras distintas que la plataforma mantiene separadas.

Esa cadena también resuelve la propiedad de los datos. Las consultas filtran siempre por el identificador del tatuador, y las inserciones de los niveles hijos no reciben el identificador del padre a ciegas: lo seleccionan desde la tabla del padre ya filtrada por el dueño, de modo que no hay forma de colgar una sesión del proyecto de otro artista. Un identificador ajeno en la dirección responde 404, para no confirmar que esa fila existe, y uno ajeno en el cuerpo de la petición responde 400.

La parte más delicada de la HU17 es que una hora ocupada no se pueda volver a comprometer. La revisión en sí es una consulta por intersección de intervalos, pero revisar y guardar por separado no basta: dos peticiones simultáneas —un doble clic, dos pestañas abiertas— pasarían la revisión a la vez y quedarían las dos citas. Por eso ambas cosas ocurren dentro de la misma transacción, bajo un candado tomado por artista, como muestra la Figura 4.39. Las reglas del cruce salieron de cómo se trabaja realmente: una cita cancelada o a la que el cliente no llegó dejan la hora libre, y dos citas contiguas —una termina a las 15:00 y la siguiente empieza a las 15:00— no se consideran cruzadas. Cuando sí hay cruce, el mensaje no dice solo que la hora está ocupada: nombra al cliente y el horario de la cita que la ocupa, que es lo que el tatuador necesita para decidir.

backend/src/models/sessionModel.js

```
// La revisión y el guardado van juntos bajo un candado por artista: sin
// él, dos peticiones simultáneas pasarían la revisión a la vez.
await client.query(
  'SELECT pg_advisory_xact_lock(hashtextextended($1::text, 0))', [userId]
);

// Una cita cancelada o un «no asistió» dejan libre la hora, y las
// contiguas (una termina a las 15:00, otra empieza a las 15:00) no chocan.
const { rows } = await db.query(
  `SELECT ps.starts_at, ps.duration_minutes, c.name AS client_name
     FROM project_sessions ps
     JOIN projects p ON p.id = ps.project_id
     JOIN clients c ON c.id = p.client_id
    WHERE ps.user_id = $1
      AND ps.status IN ('agendada', 'completada')
      AND ps.starts_at < $2::timestamptz + make_interval(mins => $3::int)
      AND ps.starts_at + make_interval(mins => ps.duration_minutes) > $2`,
  [userId, startsAt, durationMinutes]
);
if (rows.length) throw new ConflictError(`Ese horario se cruza con la cita
  de ${other.client_name} del ${dateLabel(from)}...`);
```

Figura 4.39: Revisión del horario y candado por artista, dentro de la misma transacción.

Esta revisión es, además, la que ocupó el lugar de la HU27 descartada en la Iteración 5. En vez de pedirle al tatuador que declare por adelantado sus días y horas de trabajo, el sistema protege las horas que de verdad están comprometidas, que es el problema que la historia original intentaba resolver.

La HU20 reutiliza la capa de almacenamiento de la galería de bocetos, apuntando a su propia carpeta. El orden de los pasos importa: antes de subir nada se comprueba que la sesión exista y sea del artista, porque no tiene sentido gastar almacenamiento en una imagen que después se va a rechazar; y si la sesión se borra mientras la imagen se está subiendo, el archivo recién guardado se elimina en lugar de quedar huérfano.

La HU24 se resuelve sin consultas nuevas desde el frontend: la ficha del cliente trae, en la misma respuesta, cuántos proyectos tiene, cuándo es su próxima cita agendada y cómo ha cumplido hasta ahora.

La HU30 es la historia que más cambió respecto de lo que pedía su enunciado. La primera versión guardaba un puntaje fijo en el cliente, que el artista escribía a mano; el problema es que ese número envejece y nadie lo actualiza. La versión definitiva

califica **cada sesión** de 1 a 5 según cómo llegó el cliente, y el puntaje del cliente no se guarda en ninguna parte: es el promedio de sus sesiones calificadas, calculado al consultar, como muestra la Figura 4.40. Así nunca queda desactualizado, y la lista de clientes puede ordenarse del mejor promedio al peor sin mantener ningún dato duplicado. La columna anterior se retiró del esquema. Junto al puntaje viajan las cifras que lo explican —sesiones hechas, inasistencias y cancelaciones—, porque un promedio sin contexto no dice lo mismo que un promedio con tres sesiones detrás. Una regla que se fijó desde el principio: calificar es información interna del artista y nunca le envía ningún aviso al cliente.

backend/src/models/clientModel.js

```
-- El puntaje del cliente no se guarda: es el promedio de las sesiones que
-- el artista calificó, calculado al consultar, así nunca queda viejo.
SELECT c.id, c.name, ...
       count(ps.id) FILTER (WHERE ps.status = 'completada') AS sessions_done,
       count(ps.id) FILTER (WHERE ps.status = 'no_asistio')  AS no_shows,
       count(ps.id) FILTER (WHERE ps.status = 'cancelada')   AS cancellations,
       round(avg(ps.rating), 1)::float8 AS rating_avg,
       count(ps.rating) AS rating_count
  FROM clients c
  LEFT JOIN LATERAL (
    SELECT ... FROM projects p
    LEFT JOIN project_sessions ps ON ps.project_id = p.id
     WHERE p.client_id = c.id
  ) x ON true
 WHERE c.user_id = $1
-- Primero el de mejor promedio; a igualdad, el de más sesiones
-- calificadas; los sin calificar al final, por nombre.
 ORDER BY x.rating_avg DESC NULLS LAST, x.rating_count DESC, lower(c.name)
```

Figura 4.40: Puntaje del cliente calculado al consultar, con las cifras que lo explican.

En el frontend se desarrollaron tres vistas. La de citas reúne el calendario del mes, con un contador en cada día que tiene citas, el detalle del día elegido y un formulario que permite agendar creando al cliente y al proyecto en el mismo paso, que es como llega la mayoría de las solicitudes. La de clientes presenta el marcador con las estrellas, los filtros por responsabilidad y la búsqueda por nombre o contacto. La ficha del cliente muestra su historial de proyectos con lo pagado y lo pendiente.

Resultado del desarrollo

Como resultado de esta iteración, el tatuador tiene su agenda completa dentro de la plataforma: puede ver su mes de un vistazo, agendar sin riesgo de comprometer

dos veces la misma hora, seguir el avance de una pieza grande con fotos de cada sesión y saber con qué clientes puede contar. Las Figuras 4.41, 4.42 y 4.43 presentan las vistas desarrolladas durante esta etapa.

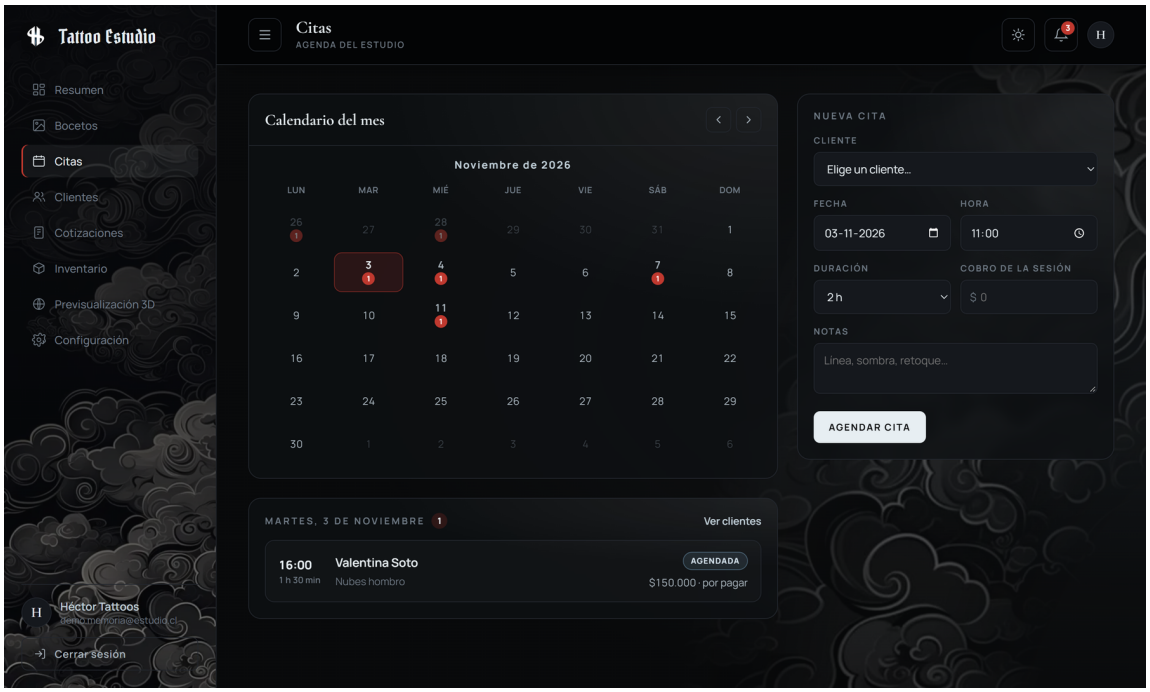


Figura 4.41: Vista “Citas”: calendario del mes con las citas marcadas, detalle del día elegido y formulario de agendado.

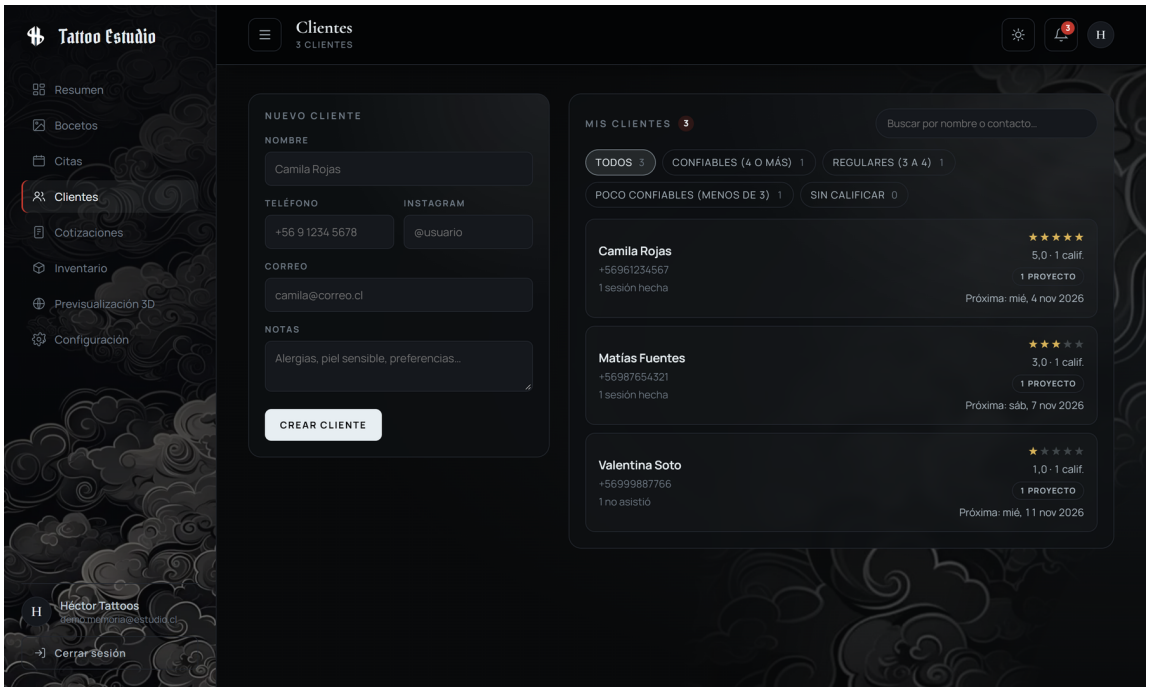


Figura 4.42: Vista “Clientes”: marcador ordenado por puntaje, con los filtros por responsabilidad de llegada.

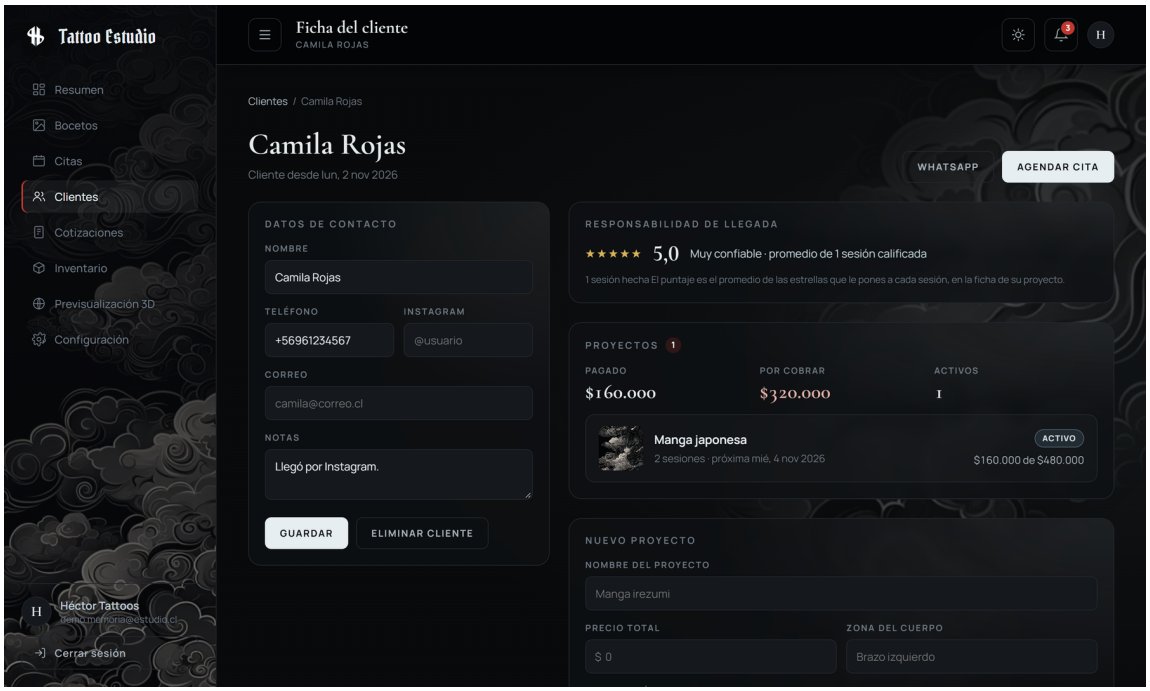


Figura 4.43: Ficha del cliente: responsabilidad de llegada, historial de proyectos y estado de los pagos.

Sistema de pruebas

Después de la implementación se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Agendar una cita creando el cliente y el proyecto en el mismo paso.
- Agendar sobre una hora que ya está ocupada.
- Agendar justo cuando termina otra cita.
- Agendar dos veces seguidas con un doble clic.
- Subir una foto del avance de una sesión y luego borrarla.
- Subir una foto a la sesión de otro artista.
- Revisar el historial de proyectos y sesiones de un cliente.
- Calificar una sesión y ver el puntaje que toma el cliente.

- Quitar la calificación de una sesión.
- Filtrar la lista de clientes por responsabilidad de llegada.

Pruebas unitarias

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, con Jest y Supertest y con la base de datos reemplazada por dobles de prueba. La Tabla 4.17 detalla las pruebas implementadas.

Tabla 4.17: Pruebas unitarias implementadas en la Iteración 6

ID	**Función evaluada**	**Objetivo**	**Tests**
PU-26	`POST /clients`, `GET /clients/:id`	Validar el registro saneado, el rechazo sin nombre o con teléfono inválido y la ficha con su historial.	8
PU-27	`POST /projects`	Comprobar el precio total, el rechazo de un cliente o un boceto ajeno y el detalle con sus sesiones.	7
PU-28	`GET` y `POST /sessions`	Verificar el rango obligatorio del listado, el agendado y la validación de fecha y pago.	6
PU-29	`POST /sessions/:id/photos`	Validar la subida, el rechazo de una sesión ajena sin tocar el archivo y el borrado.	3
PU-30	`PATCH /sessions/:id`	Comprobar que el puntaje solo acepta un entero de 1 a 5 o nulo, y que calificar no avisa al cliente.	11
PU-31	`sessionModel.create()`	Verificar el candado del artista y el rechazo de una hora ya ocupada.	5

En total, durante esta iteración se ejecutaron 40 pruebas unitarias repartidas en dos archivos de prueba. Esos dos archivos suman 64 pruebas, porque cubren también

historias vecinas que viven en las mismas rutas: los ingresos del mes de la Iteración 4 y, de la Iteración 7, los avisos al cliente y el reagendamiento de una cita.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto del agendado, de la subida de fotos y del puntaje. Se comprobó especialmente que el candado del artista se toma antes de revisar el horario y dentro de la misma transacción que guarda la cita; que una cita cancelada deja su hora libre mientras que dos citas contiguas no se consideran cruzadas; y que el puntaje solo acepta un entero de 1 a 5, de modo que un valor con decimales, un texto o un cero se rechazan en lugar de guardarse. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.18 y muestra la Figura 4.44.

Tabla 4.18: Resultado de ejecución de pruebas unitarias de la Iteración 6

Archivos evaluados	**Tests ejecutados**	**Resultado**
2	40	40 aprobados

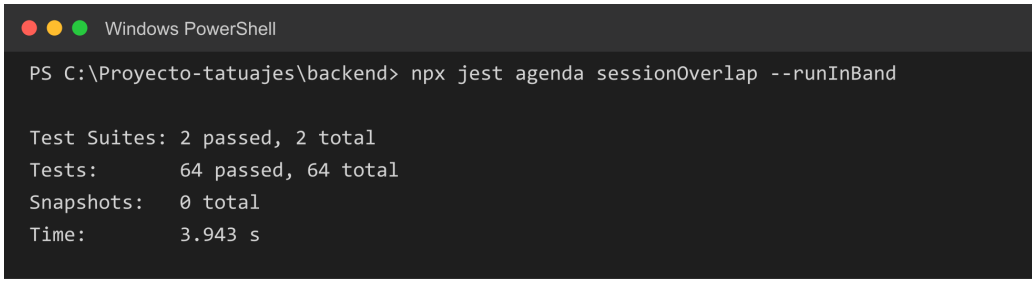


Figura 4.44: Ejecución de los dos archivos de la agenda, que reúnen las 40 pruebas de esta iteración junto con 24 de historias vecinas.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Las seis historias se completaron, aunque el reparto del esfuerzo no fue el previsto. La HU17 costó casi el doble de lo estimado, y no por el formulario ni por el calendario: se fue en la revisión del horario, que parecía una condición más en una consulta y terminó obligando a mover la revisión y el guardado a una misma transacción con candado. La HU18, la

HU19 y la HU24, en cambio, resultaron más baratas de lo previsto, porque las tres leen de la misma cadena de datos y bastó con pedirle a la base los resúmenes ya calculados.

El aprendizaje de la iteración vino de la HU30. Implementada tal como estaba escrita —un puntaje que el artista le pone al cliente— funcionaba, pero quedaba viejo apenas el cliente volvía al estudio. Preguntarle al tatuador cuándo iba a actualizarlo fue lo que dejó en evidencia que el dato útil no era el puntaje del cliente, sino cómo llegó a cada sesión: el puntaje es una consecuencia de eso y conviene calcularlo, no guardarlo. Cambiarlo obligó a retirar una columna que ya estaba en el esquema, que es exactamente el tipo de corrección que sale barata en la iteración en que se detecta y cara más adelante.

4.3.7. Iteración 7

En esta iteración se implementan las historias de usuario HU21, HU22, HU23, HU28, HU29 y HU31. Las cuatro primeras tienen un hilo común: hasta aquí la plataforma le servía al tatuador, y a partir de ahora también le habla al cliente. La HU29 cierra el registro de clientes atendidos y la HU31 abre el panel de administración, que es la primera historia del proyecto que no es para el tatuador sino para quien administra la plataforma.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 21 puntos según la Tabla 3.2.

- **HU21:** como tatuador quiero cambiar la fecha de una cita o cancelarla, para mantener la agenda actualizada frente a imprevistos.
- **HU22:** como tatuador quiero que el sistema envíe automáticamente un mensaje de WhatsApp 24 horas antes de la sesión, para evitar olvidos o retrasos.
- **HU23:** como tatuador quiero que al cancelar o cambiar una hora el sistema avise al cliente por WhatsApp, para mantener una comunicación profesional.
- **HU28:** como tatuador quiero poder editar el texto base del mensaje que se envía por WhatsApp, para darle recomendaciones del trato del tatuaje posteriormente al trabajo realizado.

- **HU29:** como tatuador quiero tener el registro de todos mis clientes atendidos con su información.
- **HU31:** como administrador quiero registrar nuevos estudios de tatuaje en la plataforma indicando nombre, correo y plan contratado, para incorporar a nuevos clientes del sistema.

La HU29 no requirió desarrollo nuevo. El registro de clientes con su información, su historial y su buscador quedó construido en la Iteración 6 junto con la HU24, así que en esta iteración solo se revisó contra su criterio de aceptación y se dio por cumplida. El esfuerzo efectivo fue entonces de 18 puntos.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.19 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.19: Tareas asociadas a la Iteración 7

Historia	**Tarea**	**Descripción**
HU21	T0001	Permitir reagendar y cancelar una cita
HU21	T0002	Revisar el horario solo cuando el cambio puede chocar
HU22	T0001	Crear diagrama de secuencia
HU22	T0002	Crear el registro de mensajes y su clave única
HU22	T0003	Buscar las citas que necesitan recordatorio
HU22	T0004	Crear el programador interno y la ruta para uno externo
HU22	T0005	Enviar el mensaje por WhatsApp o dejar el enlace manual
HU23	T0001	Crear diagrama de secuencia
HU23	T0002	Decidir qué aviso corresponde a cada cambio de la cita
HU23	T0003	Contar en pantalla qué pasó con el aviso
HU28	T0001	Crear la configuración de avisos del artista
HU28	T0002	Crear la vista con los interruptores y la vista previa
HU29	T0001	Verificar el registro de clientes contra su criterio
HU31	T0001	Crear diagrama de secuencia
HU31	T0002	Crear las tablas de estudio y el rol de administrador
HU31	T0003	Crear la ruta de registro de estudios
HU31	T0004	Crear el panel de administración
HU21–HU23, HU28, HU31	T0009	Creación y testeo de pruebas unitarias
HU21–HU23, HU28, HU31	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. La Figura 4.45 ilustra el envío automático del recordatorio de 24 horas, que es el único flujo de la plataforma que no empieza por una acción del tatuador. El aviso al cliente cuando su cita cambia usa el mismo mecanismo de entrega, disparado al guardar la cita.

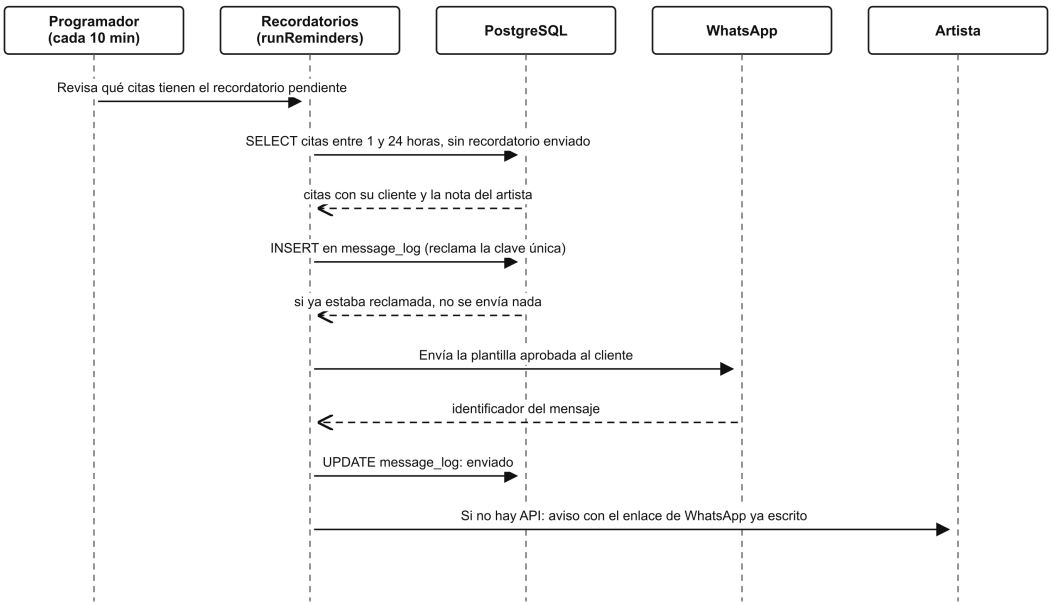


Figura 4.45: Diagrama de secuencia HU22: recordatorio automático 24 horas antes de la cita.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. La HU21 se apoya en la agenda de la iteración anterior: reagendar y cancelar son cambios de estado y de hora sobre una cita existente. Lo que hubo que afinar es cuándo corresponde volver a revisar el horario. Revisarlo en cada cambio bloqueaba operaciones legítimas —corregir el monto o las notas de una cita que, por datos antiguos, ya se solapaba con otra—, así que la revisión ocurre solo cuando el cambio puede crear un cruce: mover la hora, alargar la duración o reactivar una cita cancelada. Editar el monto, las notas o el estado de pago nunca se bloquea.

El resto de la iteración es el mensaje al cliente. Cada aviso se arma **una sola vez,**

con las mismas frases para todas las vías, y se entrega por la que esté disponible: la biblioteca que vincula el WhatsApp del propio artista, que automatiza el envío; los parámetros de la plantilla de la API oficial, si el estudio la tiene configurada; o el texto equivalente en un enlace de WhatsApp que el artista abre con un toque. El cliente recibe lo mismo por cualquiera de ellas, y el enlace manual —siempre disponible— es el que funciona sin configurar nada.

El envío se reclama antes de mandarse, como muestra la Figura 4.46. El registro de mensajes tiene una restricción de unicidad por artista y clave de deduplicación, de modo que si dos corridas del programador coinciden, la segunda no escribe al cliente de nuevo: la base rechaza el reclamo y el envío no ocurre. Es la misma idea del sello de la Iteración 5, aplicada a un mensaje en lugar de a un descuento de stock, y responde a que escribirle dos veces al mismo cliente es un error visible y molesto para quien lo recibe.

backend/src/services/clientMessages.js

```
// El envío se RECLAMA antes de mandarlo: message_log tiene una clave única
// por artista y clave de deduplicación, así que dos corridas simultáneas
// del programador no pueden escribirle dos veces al mismo cliente.
const claimed = await messageLogModel.claim({
  userId, sessionId, kind, channel, toPhone: phone, dedupeKey
});
if (!claimed) return { status: 'duplicado', channel: 'ninguno' };

// Sin teléfono utilizable no hay a quién escribirle por ningún canal: se
// deja constancia y se le avisa al artista que lo corrija en la ficha.
if (!phone) {
  await messageLogModel.finish(claimed.id, { status: 'omitido', ... });
  await notify(userId, { kind: 'mensaje_omitido', ... });
  return { status: 'omitido', channel: 'manual', reason: 'sin_telefono' };
}

// Camino manual: gratis y siempre disponible. El artista recibe un aviso
// con el enlace de WhatsApp y el mensaje ya escrito.
const link = waLink(phone, text);
```

Figura 4.46: El envío se reclama antes de mandarse, y sin teléfono válido se avisa al artista.

La HU22 tiene además una restricción del entorno: el servidor gratuito en el que está publicada la plataforma se duerme tras unos minutos sin visitas, de modo que un programador interno no basta por sí solo. La solución fue tener los dos caminos: un programador que corre mientras el servidor está despierto y una ruta que un servicio

externo llama cada diez minutos, que de paso lo mantiene despierto. Las dos vías pueden coincidir sin consecuencias, precisamente por el reclamo previo del envío.

La consulta que decide a qué citas les toca recordatorio está en la Figura 4.47, y sus condiciones salieron de pensar qué mensajes serían absurdos de recibir. Una cita entra en la ventana cuando faltan entre 1 y 24 horas: menos de una hora antes ya no sirve de recordatorio. Tampoco se le escribe a alguien que acaba de reservar hace unos minutos, ni a los clientes de un artista que apagó los recordatorios. Y la clave de deduplicación incluye la hora de la cita, así que reagendar genera un recordatorio nuevo para la hora nueva en lugar de quedar silenciado por el anterior.

backend/src/models/reminderModel.js

```
-- Una cita entra cuando falta entre 1 y 24 horas. Tres filtros evitan
-- mensajes absurdos: que falte menos de una hora, que se acabe de agendar,
-- o que el artista haya apagado los recordatorios.
SELECT ps.id, ps.user_id, ps.starts_at, c.name, c.phone, u.name,
       COALESCE(ns.reminder_note, $2) AS reminder_note
  FROM project_sessions ps
  JOIN projects p ON p.id = ps.project_id
  JOIN clients  c ON c.id = p.client_id
  JOIN users    u ON u.id = ps.user_id
  LEFT JOIN notification_settings ns ON ns.user_id = ps.user_id
 WHERE ps.status = 'agendada'
   AND ps.starts_at >  $1::timestamptz + interval '1 hour'
   AND ps.starts_at <= $1::timestamptz + interval '24 hours'
   AND ps.created_at <= $1::timestamptz - interval '1 hour'
   AND COALESCE(ns.reminders_enabled, true)
   -- La clave lleva la hora de la cita: si se reagenda, sale un
   -- recordatorio nuevo para la hora nueva.
   AND NOT EXISTS (
     SELECT 1 FROM message_log m
      WHERE m.user_id = ps.user_id
        AND m.dedupe_key = 'recordatorio:' || ps.id || ':' ||
                           floor(extract(epoch FROM ps.starts_at))::bigint)
```

Figura 4.47: Ventana de 24 horas y clave de deduplicación del recordatorio.

La HU23 no agrega una pantalla nueva: el aviso sale del mismo cambio que el tatuador ya hace en la agenda. Al guardar, el sistema compara cómo estaba la cita con cómo quedó y deduce qué corresponde —cancelación, cambio de hora o cuidados posteriores—, con una regla que vale para los tres: solo se avisa de citas futuras, porque escribirle a alguien sobre algo que ya ocurrió no tiene sentido. La respuesta del servidor incluye qué pasó con el aviso, de manera que la pantalla puede decirlo en lugar de dejar al artista con la duda de si el cliente se enteró.

La HU28 convierte en configuración lo que de otro modo habría quedado fijo en el código. El artista decide qué mensajes quiere, y edita con sus palabras la nota del recordatorio y las indicaciones de cuidados posteriores, que son justamente las dos partes del mensaje donde cada estudio tiene su criterio. La pantalla muestra una vista previa de cómo le llegará el texto al cliente, con las frases fijas y las suyas ya combinadas.

La automatización del envío se apoya en la biblioteca de vinculación: el artista enlaza su propio WhatsApp al servidor como un dispositivo más y los mensajes salen de su número. La capa de envío respeta un ritmo humano —una fila por artista, pausas de algunos segundos, el aviso de "escribiendo…" y un tope de mensajes por hora— y solo escribe a números que WhatsApp confirma. No es una vía oficial, de modo que viene apagada y exige que el artista acepte explícitamente el riesgo de que su número sea bloqueado, y donde el estudio tiene contratada la API oficial es esa la que tiene prioridad.

La HU31 inaugura la administración de la plataforma. Aparece la noción de estudio, que agrupa cuentas de tatuadores, y el rol de administrador. Dos decisiones vale la pena dejar escritas. La primera es que el rol se lee de la base de datos en cada petición y no del token, de modo que degradar a alguien tiene efecto inmediato y no cuando expire su sesión. La segunda es que la contraseña del primer tatuador se genera en el servidor, se devuelve una sola vez en la respuesta y de ella solo se guarda el hash: el administrador se la entrega al tatuador, que está obligado a cambiarla al entrar. El estudio y su primer tatuador se crean en una misma transacción, para que no pueda quedar un estudio sin nadie dentro.

Resultado del desarrollo

Como resultado de esta iteración, el cliente recibe su recordatorio sin que el tatuador tenga que acordarse, se entera cuando su cita cambia, y el tatuador puede escribir con sus propias palabras lo que se le dice. Además, la plataforma deja de ser de un solo estudio. Las Figuras 4.48 y 4.49 presentan las vistas desarrolladas durante esta etapa.

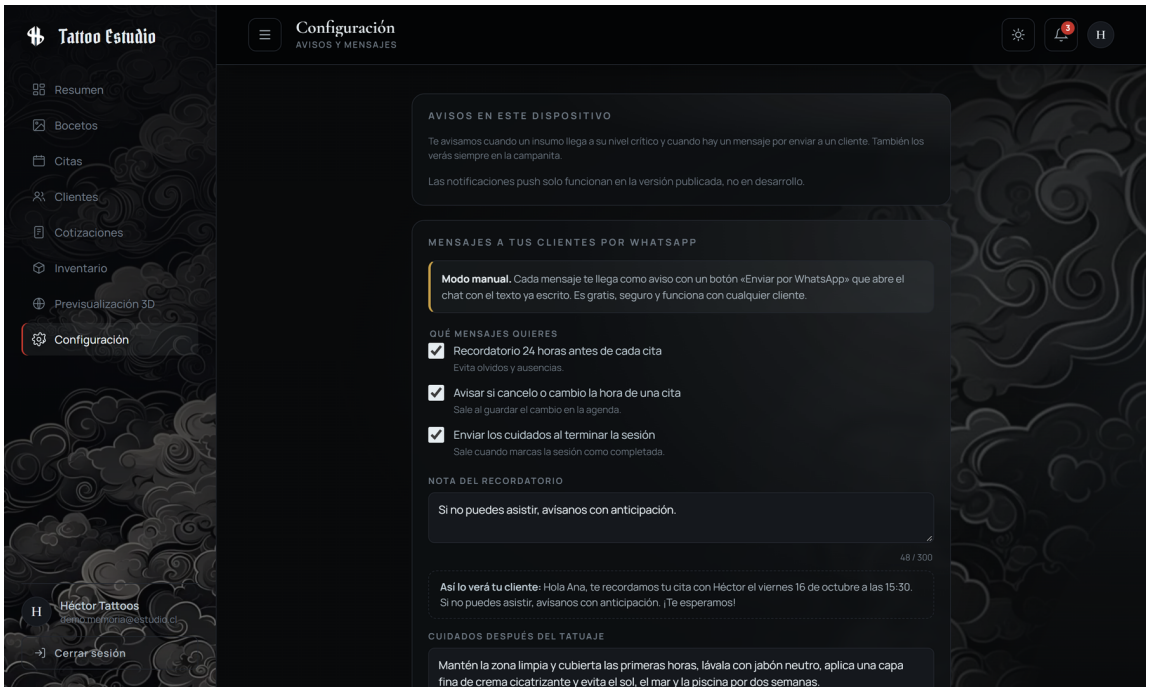


Figura 4.48: Vista “Configuración”: interruptores de cada mensaje, textos editables y vista previa de lo que recibirá el cliente.

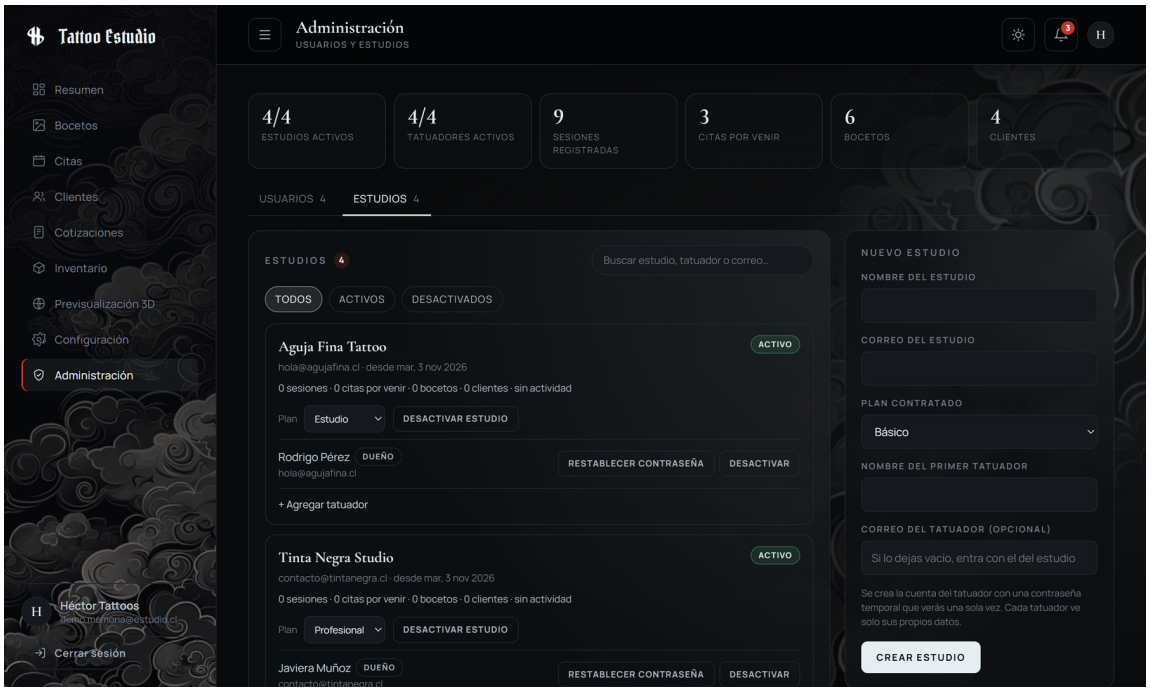


Figura 4.49: Vista "Administración": registro de un estudio nuevo con su plan y su primer tatuador.

Sistema de pruebas

Después de la implementación se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Cambiar la hora de una cita a un horario libre y a uno ocupado.
- Corregir el monto de una cita sin tocar su horario.
- Cancelar una cita futura y una que ya pasó.
- Esperar el recordatorio de una cita agendada para el día siguiente.
- Reagendar una cita que ya recibió su recordatorio.
- Hacer correr dos veces seguidas el envío de recordatorios.
- Escribirle a un cliente cuyo teléfono no es un celular válido.
- Apagar los recordatorios y volver a correr el envío.

- Editar la nota del recordatorio y revisar la vista previa.
- Registrar un estudio nuevo con un correo que ya tiene cuenta.

Pruebas unitarias

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, con Jest y Supertest y con la base de datos reemplazada por dobles de prueba. La Tabla 4.20 detalla las pruebas implementadas.

Tabla 4.20: Pruebas unitarias implementadas en la Iteración 7

ID	Función evaluada	Objetivo	Tests
PU-32	`deliver()`	Validar el reclamo previo del envío, la elección de canal y el aviso cuando falta el teléfono.	19
PU-33	`decideKind()`, `afterUpdate()`	Comprobar qué aviso corresponde a cada cambio y que una cita pasada no avisa.	24
PU-34	`dueFor Reminder()`, `runReminders()`	Verificar la ventana de 24 horas, los interruptores del artista y que no se repite el mensaje.	14
PU-35	`sendTemplate()`, `normalize Phone()`	Validar los parámetros de la plantilla, el rechazo de números que no son celulares y el enlace manual.	35
PU-36	`/notifications`, `/notification-settings`	Comprobar la campanita, los interruptores y que los valores por defecto coinciden con el esquema.	40
PU-37	`notify()`, `/push`	Verificar que el aviso nunca interrumpe la operación y que un endpoint muerto se borra solo.	19
PU-38	`PATCH /sessions/:id`	Validar el reagendamiento y la cancelación, y que un cambio inocuo no revisa el horario.	10
PU-39	`POST /admin/ studios`	Comprobar el registro del estudio con su primer tatuador, el correo repetido y el acceso de quien no es administrador.	10

En total, durante esta iteración se ejecutaron 171 pruebas unitarias repartidas en trece archivos de prueba. Diez de esos archivos son propios de los mensajes y

los avisos y reúnen 151; las otras 20 viven en los archivos de la agenda y de la administración, porque evalúan rutas que esta iteración comparte con las vecinas.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto del reagendamiento, de los recordatorios y de los avisos al cliente. Se comprobó especialmente que un mensaje reclamado no se vuelve a enviar aunque la corrida se repita; que un cliente sin teléfono válido no rompe el envío, sino que deja al artista un aviso para corregir su ficha; que reagendar una cita genera un recordatorio nuevo en lugar de quedar silenciado por el anterior; y que un fallo al avisar nunca deshace el cambio en la agenda. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.21 y muestra la Figura 4.50.

Tabla 4.21: Resultado de ejecución de pruebas unitarias de la Iteración 7

Archivos evaluados	Tests ejecutados	Resultado
13	171	171 aprobados

Figura 4.50: Ejecución de los diez archivos propios de los mensajes y los avisos, que reúnen 151 de las 171 pruebas de la iteración.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Fue la iteración más larga del proyecto y la que más se desvió de lo estimado, aunque no por la programación. La HU22 estaba escrita como “enviar un WhatsApp 24 horas antes”, y recién al implementarla apareció que el envío no se resolvía con una sola integración: la vía

oficial exige plantillas aprobadas por Meta y tarda días en aprobarlas, así que la automatización se montó sobre la biblioteca de vinculación y el enlace manual quedó de respaldo. Armar el mensaje una sola vez para que pudiera salir por cualquiera de las tres vías costó más que escribir el envío, pero es lo que permite que la historia funcione en el estudio real del proyecto.

Quedaron dos aprendizajes. El primero es que una historia que depende de un servicio externo hay que investigarla antes de estimarla: lo que se estimó en cinco puntos era en realidad un problema de integración ajeno al desarrollo. El segundo tiene que ver con la responsabilidad frente al usuario final: el canal que vincula el WhatsApp del propio artista es el que automatiza el envío y el que más se usaría, y aun así se dejó apagado y marcado como riesgoso, porque el costo de un bloqueo del número lo pagaría el tatuador y no el sistema.

4.3.8. Iteración 8

En esta iteración se implementan las historias de usuario HU08, HU09 y HU10, que son la previsualización del tatuaje sobre un modelo anatómico en tres dimensiones. Es la funcionalidad que distingue a la plataforma de una agenda con inventario, y también la más cara del proyecto: 21 de los puntos de esfuerzo estimados, repartidos en solo tres historias. Se dejó para esta altura del desarrollo a propósito, porque necesita el boceto de la Iteración 3 y porque la medida que de aquí sale es la que alimenta la cotización de la Iteración 5.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 21 puntos según la Tabla 3.2.

- **HU08:** como tatuador quiero elegir entre distintos modelos anatómicos 3D ya integrados, para utilizarlos como lienzo de previsualización.
- **HU09:** como tatuador quiero proyectar un boceto 2D (PNG) sobre el modelo 3D, para evaluar cómo se adapta a la curvatura de la piel.
- **HU10:** como tatuador quiero escalar, mover y rotar el diseño sobre el modelo 3D, para definir la posición exacta.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.22 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.22: Tareas asociadas a la Iteración 8

Historia	**Tarea**	**Descripción**
HU08	T0001	Crear diagrama de secuencia
HU08	T0002	Preparar los catorce modelos y publicarlos optimizados
HU08	T0003	Cargar el modelo y escalarlo a la altura humana real
HU08	T0004	Crear el selector de cuerpo, pieza y encuadre
HU09	T0001	Proyectar el boceto sobre la piel donde se toque
HU09	T0002	Descartar los triángulos que no miran al proyector
HU09	T0003	Envolver el dibujo alrededor del miembro
HU09	T0004	Medir la curvatura de la piel bajo la calca
HU10	T0001	Crear diagrama de secuencia
HU10	T0002	Arrastrar, escalar y girar la calca
HU10	T0003	Recortar la vecindad de la malla para el arrastre
HU10	T0004	Manejar los gestos de dos dedos
HU10	T0005	Guardar la escena y copiar la medida al proyecto
HU08–HU10	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. La Figura 4.51 ilustra la carga de un modelo y la colocación del boceto sobre la piel, que es donde ocurre todo lo propio de esta iteración; el guardado de la escena es, en cambio, una escritura corriente contra el servidor.

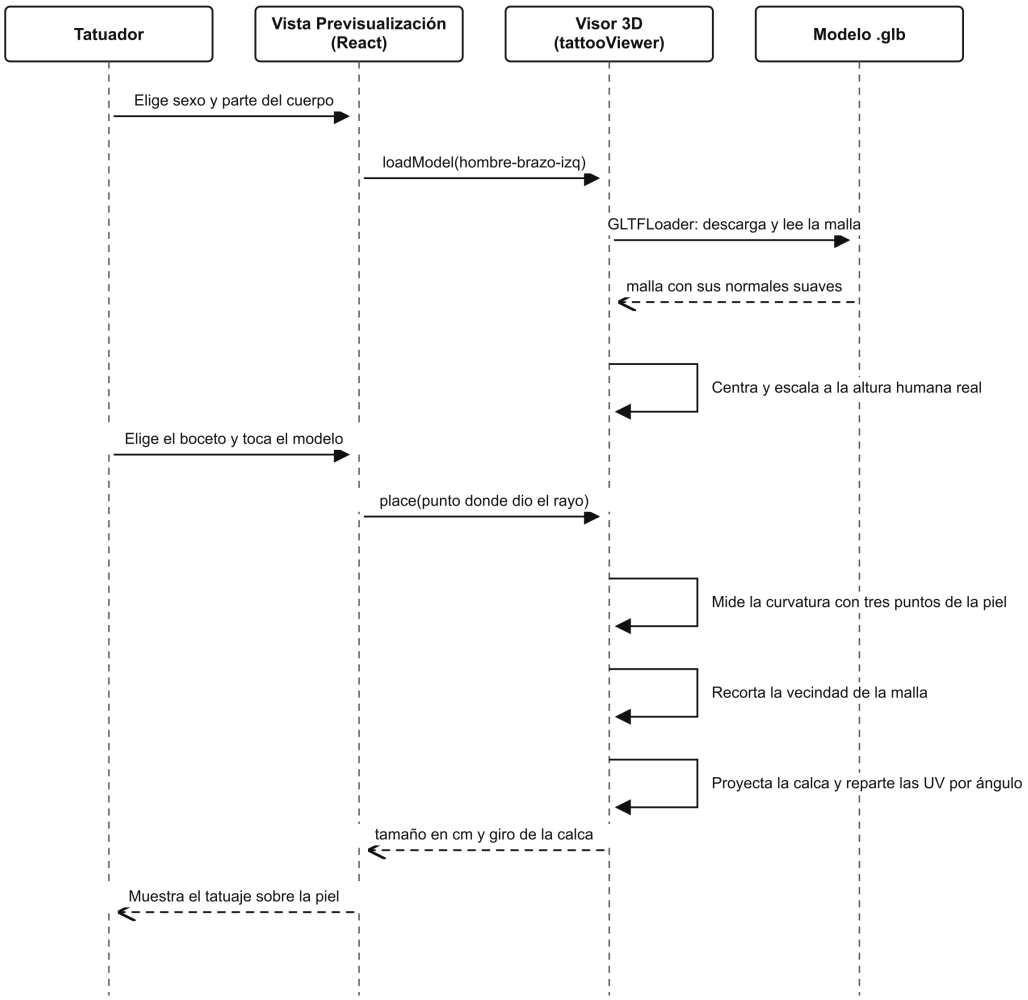


Figura 4.51: Diagrama de secuencia HU08 y HU09: carga del modelo y proyección del boceto sobre la piel.

Implementación

En esta etapa se desarrolla el visor y el guardado de las escenas. A diferencia de las iteraciones anteriores, el peso del trabajo está en el frontend: el servidor solo

guarda la escena, y todo lo que define si la previsualización sirve o no ocurre en el navegador, con three.js.

La HU08 se resolvió con catorce modelos: dos cuerpos por cada uno de siete piezas, que son el cuerpo completo, el torso, los dos brazos, las dos piernas y la cabeza. Cada pieza es un modelo suelto y no un recorte del cuerpo completo, de modo que elegir "brazo izquierdo" muestra ese brazo llenando la vista. Al cargarlo, el visor lo centra y lo escala a una altura humana real, y esa normalización no es solo estética: es lo que hace que el tamaño de la calca esté en centímetros de piel de verdad, que son los que después recibe la cotización de la Iteración 5. Los modelos se publican optimizados y con la versión en el nombre del archivo, porque el servidor los entrega con caché de un año y cambiar uno sin cambiarle el nombre dejaría a los usuarios viendo el anterior.

La HU09 pega el boceto sobre la malla con una calca proyectada, una técnica que recorta la geometría de la piel con una caja y le aplica la imagen. Esa técnica trae dos límites, y los dos se notaban de inmediato sobre un brazo. El primero es que la caja atraviesa el miembro, de modo que también recortaba la cara de atrás y el dibujo aparecía repetido al otro lado del brazo; se resolvió descartando los triángulos que no miran hacia el proyector. El segundo es más de fondo: una proyección plana no pasa de la silueta, porque reparte la imagen según su sombra, que se agolpa al llegar al borde del brazo. El dibujo se cortaba justo donde el tatuaje tendría que empezar a envolver. La solución fue dejar de proyectar en plano cuando la piel se curva lo suficiente: se proyecta una caja que abarca el anillo completo del miembro y las coordenadas de textura se recalculan por ángulo alrededor de su eje, como muestra la Figura 4.52. Así la imagen avanza a paso constante sobre la piel y rodea la muñeca en vez de cortarse, y cuesta una sola proyección, igual que antes.

frontend/src/lib/tattooViewer.ts

```
// Una proyección plana no pasa de la silueta: la textura se agolpa al
// llegar al borde del brazo y el dibujo se corta. Cuando la piel se curva
// lo bastante, se proyecta una caja que abarca el anillo entero y las
// coordenadas de textura se recalculan por ángulo alrededor del miembro.
const curve = this.curvatureFor(entry, forward, right, up)
const arc = curve ? span / curve.radius : 0
const wrapping = curve !== null && arc >= MIN_WRAP_ANGLE

if (!wrapping) return flat()

// El eje del cilindro pasa un radio por dentro de la piel, bajo la calca.
// Se deduce aquí y NO se guarda: así sigue a la calca mientras se arrastra.
// Guardarlo dejaba el anillo clavado donde se midió y el dibujo se iba.
const center = entry.position.clone().addScaledVector(forward, -radius)

// Si el reparto por ángulo no deja ni un triángulo (al borde de una zona)
// se cae a la proyección plana, en vez de congelar la geometría vieja.
return wrapUv(geometry, { center, spin, forward, axis, radius, arc }) ?? flat()
```

Figura 4.52: Decisión de envolver y anillo de proyección, que se deduce en cada reconstrucción.

Para saber cuánto se curva la piel hubo que cambiar de método a mitad de camino. La primera versión miraba cuánto giran las normales de la malla, que es lo directo; el problema es que estos modelos vienen con las normales suavizadas, y esa medida daba 7,4 centímetros de radio en un antebrazo cuya medida real es la mitad. La versión definitiva ajusta una circunferencia que pase por tres puntos de la superficie, es decir, mide la piel en lugar de preguntarle al sombreado. El muestreo además se va cerrando progresivamente, porque en una muñeca delgada un vano ancho cae fuera de la piel y no devuelve nada.

La HU10 es la historia donde más se nota la diferencia entre “funciona” y “se siente bien”. Arrastrar la calca exige rehacer su geometría en cada fotograma, y las primeras versiones hacían tres cosas que lo arruinaban. La primera era volver a medir la curvatura en cada reconstrucción: el radio salía algo distinto cada vez y el dibujo temblaba, así que ahora se congela mientras dura el arrastre y se vuelve a medir al soltar. La segunda era guardar el centro del cilindro junto con el radio; al arrastrar, la calca se iba pero el anillo de proyección se quedaba clavado donde se había medido, y la imagen se movía de forma errática hasta desaparecer. Ahora el centro no se guarda: se deduce en cada reconstrucción de dónde está la calca en ese momento. La tercera era recorrer la malla entera para recortar la calca; ahora se

trabaja sobre un trozo de malla alrededor del tatuaje, como muestra la Figura 4.53, salvo que ese trozo sea tan grande que no compense.

frontend/src/lib/tattooViewer.ts

```
/**
 * Trozo de malla alrededor de la calca.
 *
 * DecalGeometry recorre la malla entera en cada reconstrucción: 41 ms
 * sobre el brazo de 40.000 triángulos (24 fps al arrastrar) frente a
 * 18 ms sobre la vecindad. El recorte se hace con búferes tipados: ir
 * empujando a un array normal costaba 212 ms justo al agarrar la calca.
 *
 * Si se quedaría con más del 60 % de la malla no compensa, y se proyecta
 * sobre el cuerpo entero; esa decisión también se cachea.
 */
function localPatch(mesh, center, radius) {
  const keep = new Uint32Array(count)
  // ... recorre los triángulos y conserva los que caen dentro del radio
  if (kept > count * PATCH_WORTH_IT) return null // no compensa recortar
  return new THREE.Mesh(recorte, mesh.material)
}
```

Figura 4.53: Recorte de la vecindad de la malla, con los tiempos que lo justifican.

El giro tuvo su propio ajuste. Al arrastrar no se rehace la orientación de la calca desde la normal nueva, porque eso perdía el giro que el artista le había dado; en su lugar se aplica la rotación mínima que lleva de la normal anterior a la nueva, que adapta la calca a la piel conservando su inclinación, y ese origen se vuelve a anclar cada cuarto de vuelta para que no haya un punto ambiguo en el que el dibujo pegue un volantazo. La normal que se usa es la interpolada entre vértices y no la de la cara, porque con esta última la calca saltaba de triángulo en triángulo sobre un brazo de pocos polígonos.

En los gestos táctiles la decisión fue hacerse cargo de los dos dedos en lugar de dejárselos a los controles de cámara, que con un pellizco hacen acercamiento y desplazamiento a la vez y terminaban sacando la cámara fuera del cuerpo. Ahora, con dos dedos sobre el tatuaje elegido, separarlos lo agranda y girarlos lo rota, con una pequeña holgura para que no se tuerza al escalar; en cualquier otro caso el gesto es de cámara. El tope del deslizador de tamaño es fijo por zona del cuerpo: medirlo en vivo contra el modelo daba un tope que cambiaba con cada giro, y la escala se movía bajo el dedo. También se eliminó el control de opacidad que traía la primera

versión, sencillamente porque un tatuaje no es translúcido.

Por último, la escena se guarda en el servidor como una lista de colocaciones saneada —solo los campos conocidos, para que nadie pueda inflar el documento con datos arbitrarios— y al guardarla se copia además la medida del tatuaje al proyecto. Esa copia no es redundante: el enlace entre proyecto y escena se borra si la escena desaparece, y sin la copia se perdería el único lugar donde está escrito de qué porte es el tatuaje.

Resultado del desarrollo

Como resultado de esta iteración, el tatuador puede mostrarle al cliente cómo se verá el diseño sobre su cuerpo antes de tatuarlo, ajustarlo hasta dejarlo donde va, y dejar esa decisión guardada junto al proyecto. Las Figuras 4.54 y 4.55 presentan la vista desarrollada durante esta etapa.

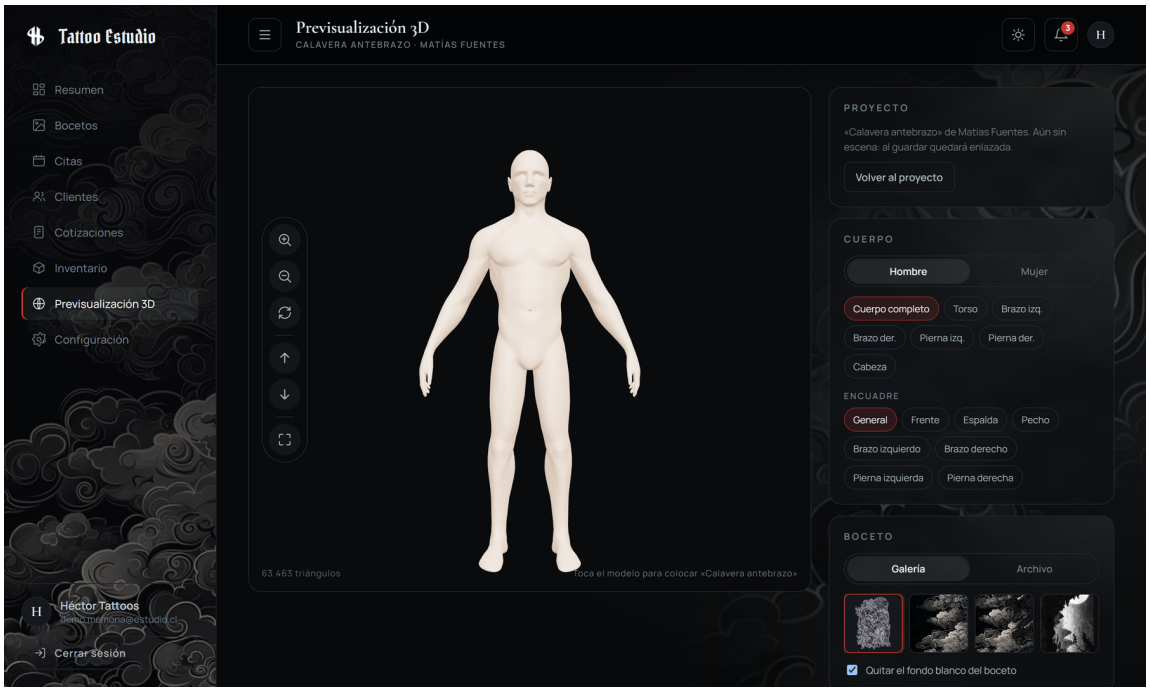


Figura 4.54: Vista “Previsualización 3D”: catálogo de cuerpos, piezas y encuadres, con el boceto listo para colocar.

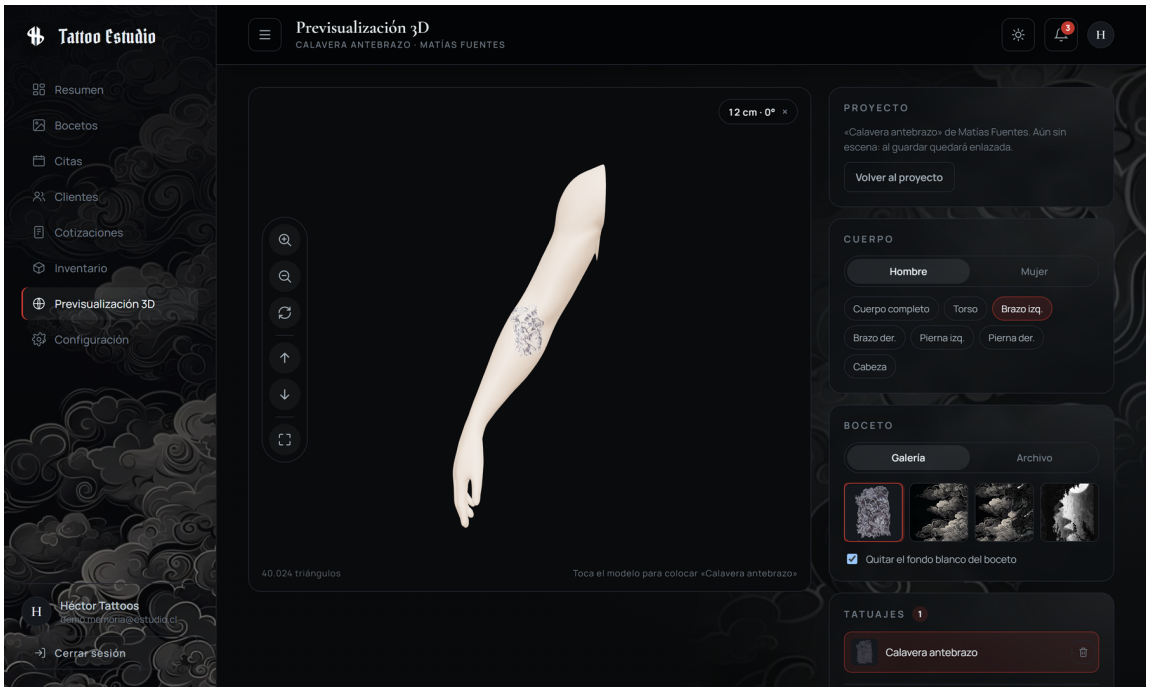


Figura 4.55: El boceto proyectado sobre el antebrazo, con su tamaño en centímetros y su giro sobre el lienzo.

Sistema de pruebas

Esta iteración se apartó del esquema de pruebas de las demás, y conviene dejarlo dicho con claridad: **no se escribieron pruebas unitarias automatizadas para el visor**. La razón es que lo que hay que verificar aquí es si el dibujo se ve bien sobre la piel, y eso no lo responde una aserción sobre un valor devuelto: una proyección que deja el tatuaje duplicado al otro lado del brazo es, para cualquier prueba automatizada razonable, una geometría perfectamente válida. Las pruebas se hicieron entonces de dos maneras: recorriendo a mano una lista de casos sobre los modelos reales, y midiendo el rendimiento en el navegador, que es lo único de esta iteración que sí admite un número objetivo.

Los casos recorridos a mano fueron los siguientes:

- Colocar el boceto en el antebrazo, el torso, la pierna y la cabeza.
- Girar el modelo y comprobar que el dibujo no aparece duplicado atrás.
- Agrandar el tatuaje hasta que tenga que envolver la muñeca.

- Arrastrar la calca desde el codo hasta la muñeca sin soltarla.
- Sacar el cursor del lienzo en medio de un arrastre.
- Girar el tatuaje media vuelta y seguir arrastrándolo.
- Escalar y rotar con dos dedos sobre una pantalla táctil.
- Acercar la cámara con dos dedos fuera del tatuaje.
- Guardar la escena y volver a abrirla desde el proyecto.
- Cotizar el proyecto y comprobar que el tamaño llega desde la escena.

Resultados obtenidos

Los problemas que estos casos dejaron en evidencia son los que se describen en la implementación: el dibujo duplicado en la cara opuesta, el corte en la silueta, el temblor al arrastrar y el salto del tatuaje al volver a entrar al lienzo. Todos se corrigieron dentro de la iteración.

La medición de rendimiento se hizo en el navegador, sobre el modelo de brazo de 40.000 triángulos, y es la que resume la Tabla 4.23. El punto de partida era un arrastre a 24 fotogramas por segundo, con un tirón de más de dos décimas de segundo justo al agarrar el tatuaje; la versión final sostiene 60 fotogramas por segundo de mediana durante todo el arrastre, sin ningún fotograma por encima del presupuesto.

Tabla 4.23: Medición de rendimiento del visor, antes y después

Medición	**Antes**	**Después**
Reconstruir la calca	41 ms (malla entera)	18 ms (vecindad)
Primer recorte al agarrar el tatuaje	212 ms	32 ms
Arrastre sostenido	24 fps	60 fps de mediana

Queda como deuda reconocida que las rutas de guardado de escenas sí podrían cubrirse con pruebas unitarias como el resto del backend —validan datos y filtran por dueño igual que las demás—, y que esta iteración no las dejó escritas.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Las tres historias se completaron, pero fue la iteración con más retrocesos del proyecto: varias piezas se escribieron, se probaron sobre el modelo y hubo que rehacerlas con otro enfoque. La estimación, que ya era la más alta del backlog, igual se quedó corta.

El aprendizaje principal es metodológico. En las iteraciones anteriores, cuando algo no funcionaba, bastaba leer el código para ver por qué; aquí los problemas eran de geometría y solo se entendían mirando el resultado y midiendo. La medida de la curvatura es el mejor ejemplo: el método inicial era el razonable y el código estaba bien escrito, pero daba el doble del radio real, y eso no se descubre revisando el código sino comparando su resultado con el antebrazo que se tiene delante. Lo mismo vale para el rendimiento: el arrastre "se sentía raro" mucho antes de que hubiera un número que lo explicara, y recién al medir los fotogramas quedó claro dónde se iba el tiempo.

El segundo aprendizaje es sobre el alcance de las pruebas automatizadas. Sirven muy bien para reglas de negocio —un stock que no puede quedar negativo, un mensaje que no se envía dos veces— y muy mal para decidir si una imagen se ve bien. Reconocerlo a tiempo evitó gastar la iteración escribiendo pruebas que habrían pasado igual con el dibujo duplicado en la espalda.

4.3.9. Iteración 9

En esta iteración se implementan las historias de usuario HU32, HU33, HU34 y HU38, que completan la administración de la plataforma iniciada en la Iteración 7. Si aquella creó el estudio y el rol de administrador, esta se ocupa de lo que viene después de dar de alta a alguien: dejarlo entrar o no, sumarle compañeros de estudio y sacarlo de un apuro cuando pierde su contraseña. Es la iteración más corta del proyecto y, a diferencia de las anteriores, casi todo su trabajo consistió en decidir qué no hacer.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 9 puntos según la Tabla 3.2.

- **HU32:** como administrador quiero activar o desactivar una cuenta de estudio, para gestionar el acceso a la plataforma ante impagos o solicitudes de baja.
- **HU33:** como administrador quiero agregar tatuadores adicionales dentro de un estudio existente, para dar soporte a estudios con más de un profesional.
- **HU34:** como administrador quiero eliminar o desactivar la cuenta de un tatuador específico dentro de un estudio, para mantener actualizada la nómina de usuarios activos.
- **HU38:** como administrador quiero restablecer la contraseña de cualquier usuario registrado, para brindar soporte ante bloqueos de acceso.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.24 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.24: Tareas asociadas a la Iteración 9

Historia	**Tarea**	**Descripción**
HU32	T0001	Crear diagrama de secuencia
HU32	T0002	Crear la ruta de edición y baja del estudio
HU32	T0003	Cortar el acceso de un estudio desactivado
HU33	T0001	Crear diagrama de secuencia
HU33	T0002	Crear la ruta para sumar tatuadores al estudio
HU33	T0003	Entregar la contraseña temporal una sola vez
HU34	T0001	Crear la ruta de activación y baja de una cuenta
HU34	T0002	Impedir que la plataforma quede sin administrador
HU38	T0001	Crear la ruta de restablecimiento de contraseña
HU38	T0002	Obligar a cambiar la contraseña que puso el administrador
HU32–HU34, HU38	T0009	Creación y testeo de pruebas unitarias
HU32–HU34, HU38	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. La Figura 4.56 ilustra la baja de una cuenta y el corte de su acceso, que ocurre en la petición siguiente y no cuando vence su token.

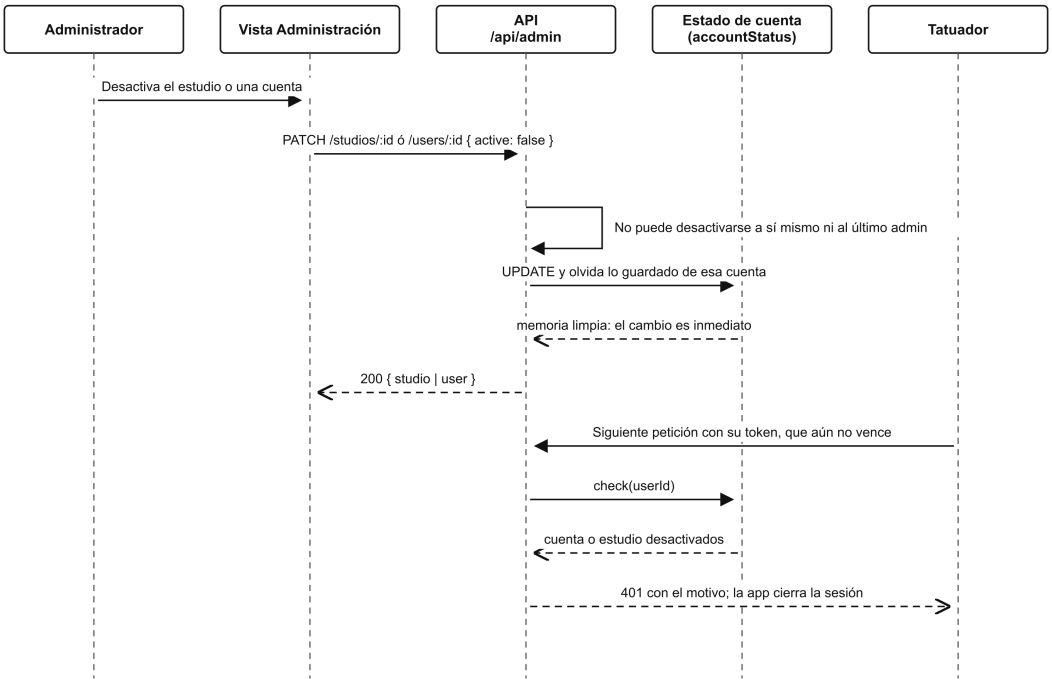


Figura 4.56: Diagrama de secuencia HU32 y HU34: baja de una cuenta y corte de su acceso.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. Las cuatro son operaciones sobre cuentas, y la parte interesante no está en escribirlas sino en lo que implican.

El problema de fondo de la HU32 y la HU34 es que la sesión de la plataforma dura siete días. Desactivar una cuenta en la base de datos no sirve de nada si quien la usa ya tiene un token emitido: seguiría entrando una semana más, que es justamente lo que no se quiere cuando la baja responde a un impago. Por eso la comprobación de si una cuenta puede seguir trabajando ocurre en **cada petición**, como muestra la Figura 4.57. Para no consultar la base cada vez, la respuesta se guarda unos segundos por cuenta, y el administrador limpia esa memoria en el mismo momento en que activa o desactiva, de modo que el corte es inmediato. Cuando la cuenta ya no puede entrar, el servidor responde con el motivo —cuenta desactivada o estudio desactivado— y la aplicación cierra la sesión en lugar de dejar al tatuador frente a errores sueltos.

backend/src/services/accountStatus.js

```
// Lo pregunta requireAuth en CADA petición: un token dura 7 días, así que
// sin esto un tatuador o un estudio desactivado seguiría entrando hasta
// que su token venciera. La respuesta se guarda 30 segundos por cuenta, y
// el administrador limpia esa memoria al activar o desactivar.
function decide(row) {
  if (!row) return { ok: false, reason: 'missing', ... };
  if (!row.active) return { ok: false, reason: 'user_inactive', ... };
  if (row.role !== 'admin' && row.studio_active === false) {
    return { ok: false, reason: 'studio_inactive', ... };
  }
  return { ok: true, role: row.role };
}

// Un administrador no depende de un estudio. Un tatuador sin estudio
// (cuentas anteriores a los estudios) puede entrar: su studio_active es
// null, no false.
```

Figura 4.57: Comprobación del estado de la cuenta y de su estudio, en cada petición.

La decisión de diseño de la iteración está en la HU34, que pide "eliminar o desactivar" la cuenta de un tatuador. Se implementó solo la desactivación y **no existe borrado de cuentas** en la plataforma. La razón es que una cuenta de tatuador no contiene únicamente sus datos: de ella cuelgan sus clientes, su agenda, los pagos registrados y las fotos de las sesiones. Borrarla destruiría el historial de personas que no son usuarios del sistema y que no tienen cómo recuperarlo, y lo haría de forma irreversible para resolver algo —sacar a alguien de la nómina— que la desactivación resuelve igual de bien y se puede deshacer. Desactivar conserva los datos, corta el acceso de inmediato y permite volver atrás si fue un error.

Sobre la misma ruta se pusieron dos puertas, que están en la Figura 4.58: un administrador no puede desactivarse a sí mismo, y tampoco puede desactivar al último administrador activo. Ambas evitan el mismo accidente, que es dejar la plataforma sin nadie que pueda administrarla, con el agravante de que sería un estado del que no se sale desde la propia aplicación.

backend/src/controllers/adminController.js

```
// Dos puertas que impiden dejar la plataforma sin quien la administre.
if (!active && req.params.id === req.user.sub) {
  return res.status(400).json({ message: 'No puedes desactivar tu propia cuenta.' });
}
const target = await userModel.findById(req.params.id);
if (!target) return res.status(404).json(USER_NOT_FOUND);
if (!active && target.role === 'admin' && target.active &&
    (await userModel.countActiveAdmins()) <= 1) {
  return res.status(409).json({
    message: 'Es el único administrador activo: la plataforma quedaría sin ninguno.'
  });
}

// No hay borrado de cuentas: desactivar conserva los datos y es
// reversible. El cambio corta el acceso en la siguiente petición.
const user = await userModel.setActive(req.params.id, active);
accountStatus.invalidate(req.params.id);
```

Figura 4.58: Las dos puertas que impiden dejar la plataforma sin administrador.

La HU33 suma tatuadores a un estudio existente con el mismo mecanismo que creó al primero: el servidor genera una contraseña temporal legible, guarda solo su hash y la devuelve una única vez en la respuesta, para que el administrador se la entregue al tatuador. Conviene dejar dicho qué significa aquí pertenecer a un estudio: agrupa cuentas para la administración y la facturación, pero **no comparte datos**. Cada tatuador sigue viendo solo sus clientes, su agenda y su inventario. Fue una decisión explícita del estudio con el que se trabajó, y además compartir esos datos habría obligado a rehacer las reglas de propiedad en todo el backend, que es la base sobre la que se construyeron las siete iteraciones anteriores.

La HU38 reutiliza ese mismo mecanismo para el soporte: el administrador restablece la contraseña de cualquier cuenta y recibe una temporal nueva. Además invalida los enlaces de "olvidé mi contraseña" que estuvieran pendientes, porque dejar vivo un enlace antiguo después de un restablecimiento sería una puerta trasera. Hay una limitación que conviene anotar: restablecer la contraseña no invalida los tokens ya emitidos, de modo que no sirve para cortarle el acceso a alguien de inmediato; para eso está la desactivación, y así quedó documentado.

En el frontend, la contraseña que pone el administrador deja la cuenta marcada para cambio obligatorio: al entrar, la aplicación lleva al tatuador a una pantalla donde elige una propia. Es un empujón del frontend y no un candado del servidor

—las rutas de la API siguen funcionando con la contraseña temporal—, y se eligió así porque el objetivo es que la contraseña que conoce el administrador deje de ser la del tatuador, no bloquear su trabajo.

Resultado del desarrollo

Como resultado de esta iteración, la plataforma puede administrarse sin tocar la base de datos: dar de alta estudios y tatuadores, cortar accesos y resolver bloqueos. Las Figuras 4.59 y 4.60 presentan las vistas desarrolladas durante esta etapa.

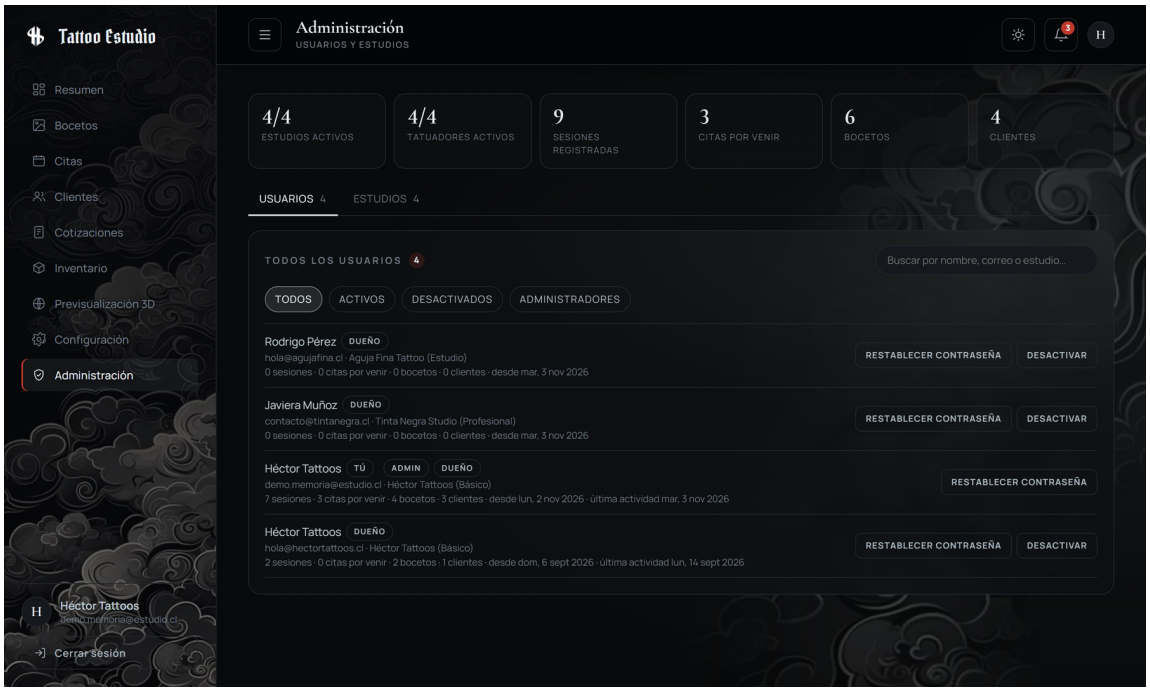


Figura 4.59: Vista “Administración”: cuentas con su estudio y su uso, con las acciones de restablecer contraseña y desactivar.

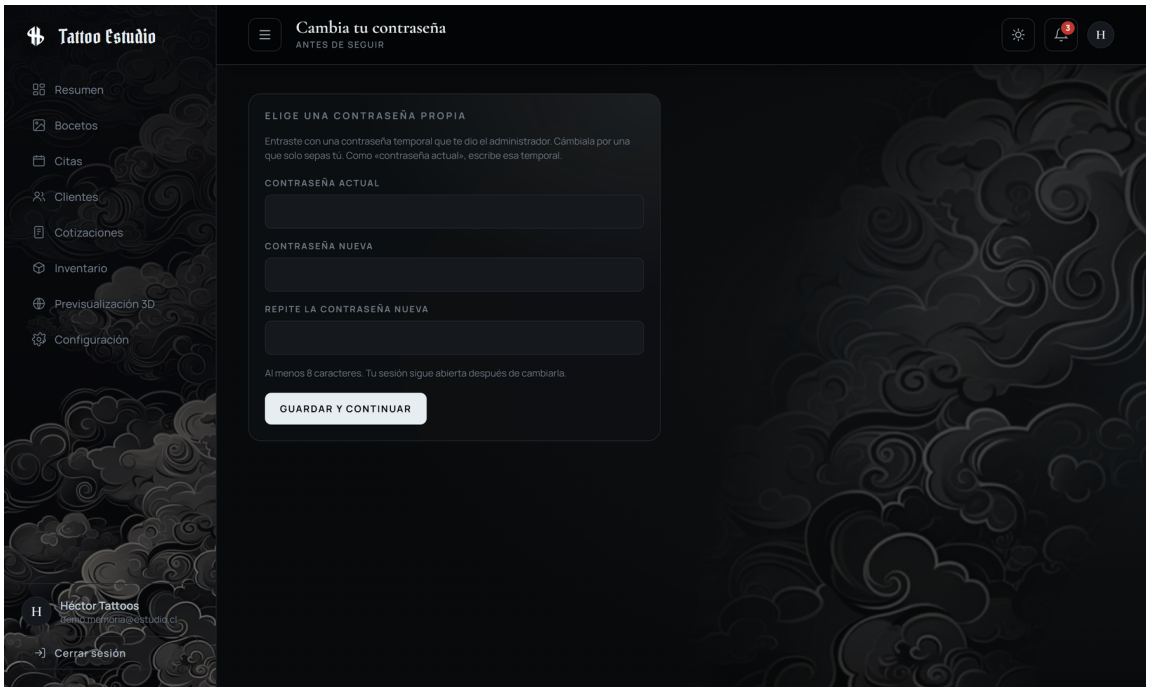


Figura 4.60: Pantalla de cambio obligatorio, a la que llega quien entró con una contraseña puesta por el administrador.

Sistema de pruebas

Después de la implementación se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Desactivar un estudio y comprobar que sus tatuadores quedan fuera.
- Seguir trabajando con la sesión abierta cuando el estudio se desactiva.
- Cambiar el plan de un estudio sin tocar su acceso.
- Agregar un segundo tatuador y entrar con su contraseña temporal.
- Agregar un tatuador con un correo que ya tiene cuenta.
- Desactivar y volver a activar a un tatuador.
- Intentar desactivarse a uno mismo siendo administrador.
- Intentar desactivar al único administrador activo.

- Restablecer una contraseña y entrar con la nueva.
- Pedir un enlace de recuperación y restablecer la contraseña después.

Pruebas unitarias

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, con Jest y Supertest y con la base de datos reemplazada por dobles de prueba. La Tabla 4.25 detalla las pruebas implementadas.

Tabla 4.25: Pruebas unitarias implementadas en la Iteración 9

ID	**Función evaluada**	**Objetivo**	**Tests**
PU-40	`PATCH /admin/studios/:id`	Validar la baja del estudio, el cambio de plan y de dueño, y el estudio inexistente.	5
PU-41	`POST /studios/:id/tattooists`	Comprobar el alta con contraseña temporal y el rechazo de un correo ocupado.	2
PU-42	`PATCH /admin/users/:id`	Verificar la baja y el alta, y las dos puertas que protegen al último administrador.	5
PU-43	`POST /users/:id/reset-password`	Validar la contraseña temporal nueva, que sea distinta cada vez y que anule los enlaces pendientes.	3
PU-44	`accountStatus.decide()`	Comprobar que una cuenta o un estudio desactivados quedan fuera, y que el administrador no depende de un estudio.	15
PU-45	`POST /auth/change-password`	Verificar el cambio obligatorio y que una contraseña actual equivocada no cierra la sesión.	4

En total, durante esta iteración se ejecutaron 34 pruebas unitarias repartidas en tres archivos de prueba. Esos tres archivos suman 60 pruebas, porque incluyen también las del registro de estudios de la Iteración 7, las métricas de la Iteración 10 y el resto del inicio de sesión.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto de las altas, las bajas y el restablecimiento de contraseñas. Se comprobó especialmente que desactivar un estudio deja fuera a todos sus tatuadores en la petición siguiente, sin esperar a que venzan sus tokens; que un administrador no puede desactivarse a sí mismo ni dejar la plataforma sin ninguno activo; que cada contraseña temporal generada es distinta; y que escribir mal la contraseña actual al cambiarla devuelve un error de formulario y no un cierre de sesión. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.26 y muestra la Figura 4.61.

Tabla 4.26: Resultado de ejecución de pruebas unitarias de la Iteración 9

Archivos evaluados	**Tests ejecutados**	**Resultado**
3	34	34 aprobados

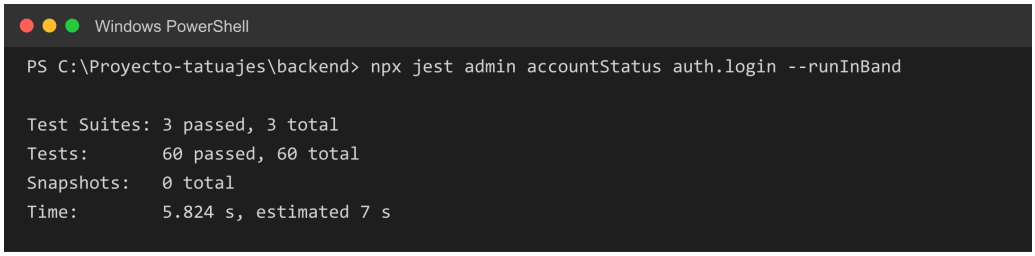


Figura 4.61: Ejecución de los tres archivos de administración y sesión, que reúnen las 34 pruebas de esta iteración junto con 26 de historias vecinas.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. Las cuatro historias se completaron dentro de lo estimado, y fue la iteración más predecible del proyecto: rutas que validan datos, consultan y actualizan, sobre un backend cuyas reglas de propiedad ya estaban firmes desde la Iteración 6.

Lo que sí tomó tiempo fue decidir el alcance de la HU34. La historia ofrecía dos caminos —eliminar o desactivar— y la tentación era implementar los dos, porque técnicamente el borrado es la operación más simple de las dos. Pensar qué se lleva por delante una cuenta borrada fue lo que cerró la discusión: datos de clientes que nunca fueron usuarios del sistema. Quedó como aprendizaje que cuando una historia ofrece una alternativa irreversible y una reversible que cumplen el mismo objetivo, implementar solo la reversible no es dejar la historia a medias.

El segundo aprendizaje es sobre el costo escondido de las sesiones largas. Un token de siete días es cómodo para el usuario, pero obliga a revisar el estado de la cuenta en cada petición si se quiere poder cortar un acceso de verdad; ese costo se paga con una pequeña memoria de unos segundos que el administrador limpia al cambiar un estado. Es una decisión que se tomó aquí pero que afecta a todas las rutas de la plataforma.

4.3.10. Iteración 10

En esta última iteración se implementan las historias de usuario HU35 y HU36, que son las métricas de uso de la plataforma, y se decide no implementar la HU37. Con ella se cierra la fase de iteraciones: el panel de administración de la Iteración 7 pasa de permitir dar de alta a mostrar qué está ocurriendo, y el dueño de un estudio con varios tatuadores recibe su propio resumen.

Historias de usuario asociadas

A continuación se listan las historias de usuario asignadas a esta iteración en la Tabla 3.3, cuya estimación de esfuerzo suma 11 puntos según la Tabla 3.2.

- **HU35:** como administrador quiero visualizar un listado de todos los estudios y tatuadores registrados, para monitorear el uso general de la plataforma.
- **HU36:** como administrador quiero ver métricas globales de uso por estudio (número de sesiones, citas agendadas, bocetos almacenados), para evaluar la adopción de la plataforma.
- **HU37:** como administrador quiero gestionar los modelos anatómicos 3D disponibles en la plataforma (agregar, actualizar o eliminar), para mantener actualizado el catálogo compartido entre estudios.

La HU37 quedó fuera del alcance del proyecto, de modo que el esfuerzo efectivo de la iteración fue de 8 puntos. El motivo está en lo que significa hoy un modelo en la plataforma: un archivo que se prepara fuera del sistema, se optimiza con un script y se publica como contenido estático con caché de un año. Convertir eso en un catálogo administrable desde la aplicación no es agregar un formulario de subida; es validar que cada archivo que llegue cumpla las condiciones que el visor da por hechas —una sola malla, un solo material, sin esqueleto, orientado y con una cantidad de triángulos que el navegador pueda mover—, almacenarlo, versionarlo e invalidar el caché de quienes ya tengan el anterior. Es, en tamaño, un subsistema comparable a la galería de bocetos más una validación de geometría que no existe en ninguna otra parte del sistema. Frente a eso, el beneficio para esta versión es bajo: los catorce modelos integrados cubren las zonas del cuerpo que efectivamente se tatúan, y agregar uno nuevo es una tarea de mantenimiento que ocurre rara vez. Se dejó registrada como trabajo futuro.

Inicio de Iteración

En esta etapa se crean y registran las tareas acordes a las historias de usuario, donde la Tabla 4.27 ilustra el registro de las tareas resultantes de la iteración.

Tabla 4.27: Tareas asociadas a la Iteración 10

Historia	**Tarea**	**Descripción**
HU35	T0001	Crear diagrama de secuencia
HU35	T0002	Listar estudios con sus tatuadores y cuentas
HU35	T0003	Buscar y filtrar por estado en el panel
HU36	T0001	Crear diagrama de secuencia
HU36	T0002	Calcular las métricas de uso de cada estudio
HU36	T0003	Mostrar los totales de la plataforma
HU36	T0004	Crear el resumen del mes para el dueño del estudio
HU36	T0005	Crear la vista ≪Mi estudio≫
HU37	T0001	Descartada: queda como trabajo futuro
HU35, HU36	T0009	Creación y testeo de pruebas unitarias
HU35, HU36	T0010	Creación y uso de sistema de pruebas

Diseño

En esta etapa se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. La Figura 4.62 ilustra el resumen mensual por tatuador, que es el flujo con una regla propia: lo ve solo el dueño del estudio. El listado con las métricas sigue el mismo recorrido que las demás rutas de administración.

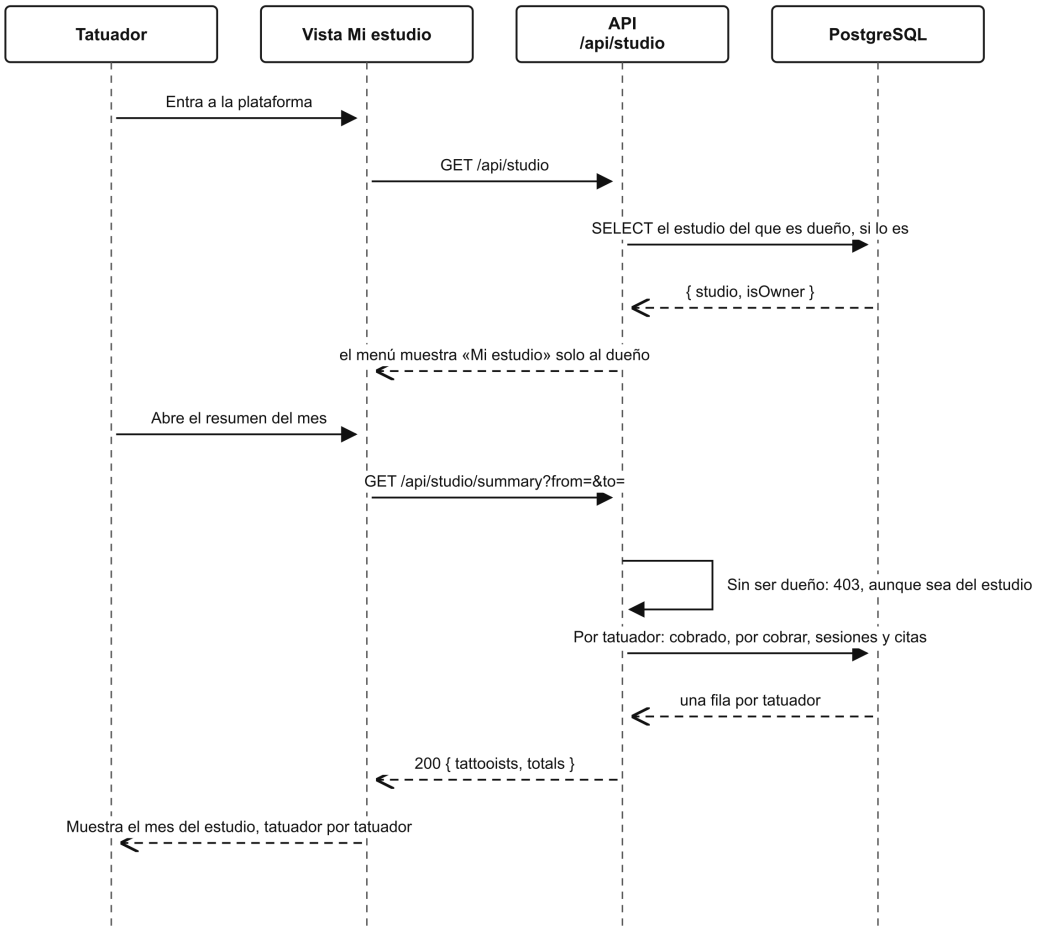


Figura 4.62: Diagrama de secuencia HU36: resumen mensual por tatuador, visible solo para el dueño.

Implementación

En esta etapa se desarrolla el backend y el frontend de las historias asignadas. Las métricas de un estudio son, en el fondo, las de sus tatuadores sumadas: cuántas sesiones se han registrado, cuántas citas tiene por delante, cuántos bocetos guardados y cuántos clientes. La parte que exigió cuidado es cómo se suman. Con varios enlaces directos a las tablas hijas —sesiones, bocetos y clientes de cada tatuador— las filas se multiplican entre sí y los conteos salen inflados sin que nada falle de forma visible: un estudio con tres bocetos y cuatro sesiones aparece con doce de cada cosa. Por eso cada métrica va en su propia subconsulta lateral, como muestra la Figura 4.63, y la base devuelve una sola fila por estudio ya sumada.

backend/src/models/studioModel.js

```
-- Las métricas de un estudio son las de sus tatuadores sumadas. Van en
-- subconsultas laterales y no en JOIN directos: con varios JOIN sobre
-- tablas hijas las filas se multiplican entre sí y los conteos salen
-- inflados.
SELECT s.*,
       COALESCE(m.sessions_total, 0)::int          AS sessions_total,
       COALESCE(m.upcoming_appointments, 0)::int   AS upcoming_appointments,
       COALESCE(k.sketches_total, 0)::int          AS sketches_total,
       COALESCE(c.clients_total, 0)::int           AS clients_total,
       GREATEST(m.last_session_at, k.last_sketch_at) AS last_activity_at
  FROM studios s
  LEFT JOIN LATERAL (
    SELECT count(*) AS sessions_total,
           count(*) FILTER (WHERE ps.status = 'agendada'
                              AND ps.starts_at >= now()) AS upcoming_appointments,
           max(ps.created_at) AS last_session_at
      FROM project_sessions ps JOIN users u ON u.id = ps.user_id
     WHERE u.studio_id = s.id
  ) m ON true
```

Figura 4.63: Métricas de cada estudio, en subconsultas laterales para que los conteos no se multipliquen.

A las cifras de volumen se le sumó una de actividad: la fecha del último movimiento del estudio, que es el máximo entre su última sesión registrada y su último boceto subido. Es la que responde la pregunta que de verdad le interesa a quien administra la plataforma, que no es cuántos estudios hay inscritos sino cuáles siguen usándola.

La HU36 tiene una segunda mitad que no estaba en su enunciado y que apareció al conversar sobre para qué servirían estas métricas: un estudio con varios tatuadores también quiere ver su propio mes. Se construyó entonces “Mi estudio”, un resumen por tatuador con lo cobrado, lo que queda por cobrar, las sesiones hechas y las citas agendadas del mes. Las dos cifras de dinero se calculan con el mismo criterio del panel de control de la Iteración 4 —lo cobrado por la fecha en que se marcó pagada la sesión, lo pendiente por la fecha de la cita—, para que el artista no vea dos números distintos según la pantalla en que mire.

Aquí hubo que tomar una decisión de privacidad, que está en la Figura 4.64: cuánto factura cada tatuador es información sensible dentro del propio estudio, así que el resumen lo ve **solo el dueño**. Pertenecer al estudio no alcanza, y la ruta responde 403 a cualquier otro, incluso a sus compañeros. La vista tampoco aparece en

el menú salvo que la cuenta sea dueña de un estudio con más de un tatuador, aunque esa parte es comodidad para el usuario: quien decide es el servidor. El resumen además es de un mes a la vez, con un rango acotado, porque es una vista de gestión y no una herramienta de exportación.

backend/src/controllers/studioController.js

```
// GET /api/studio/summary?from=&to=  → lo que entró por tatuador
const studio = await studioModel.findOwnedBy(req.user.sub);
// Los montos de cada tatuador son información sensible: solo los ve el
// dueño. Pertenecer al estudio no alcanza.
if (!studio) {
  return res.status(403).json({ message: 'Solo el dueño del estudio ve este resumen.' });
}

const tattooists = await studioModel.summary(studio.id, { from, to });
const sum = (key) => tattooists.reduce((n, t) => n + t[key], 0);
return res.json({
  studio, tattooists,
  totals: {
    collected: sum('collected'), pending: sum('pending'),
    sessionsDone: sum('sessions_done'), appointments: sum('appointments'),
  },
});
```

Figura 4.64: El resumen por tatuador, reservado al dueño del estudio.

Resultado del desarrollo

Como resultado de esta iteración, quien administra la plataforma puede ver qué estudios la están usando y cuánto, y el dueño de un estudio con varios tatuadores tiene su propio resumen del mes. Las Figuras 4.65 y 4.66 presentan las vistas desarrolladas durante esta etapa.

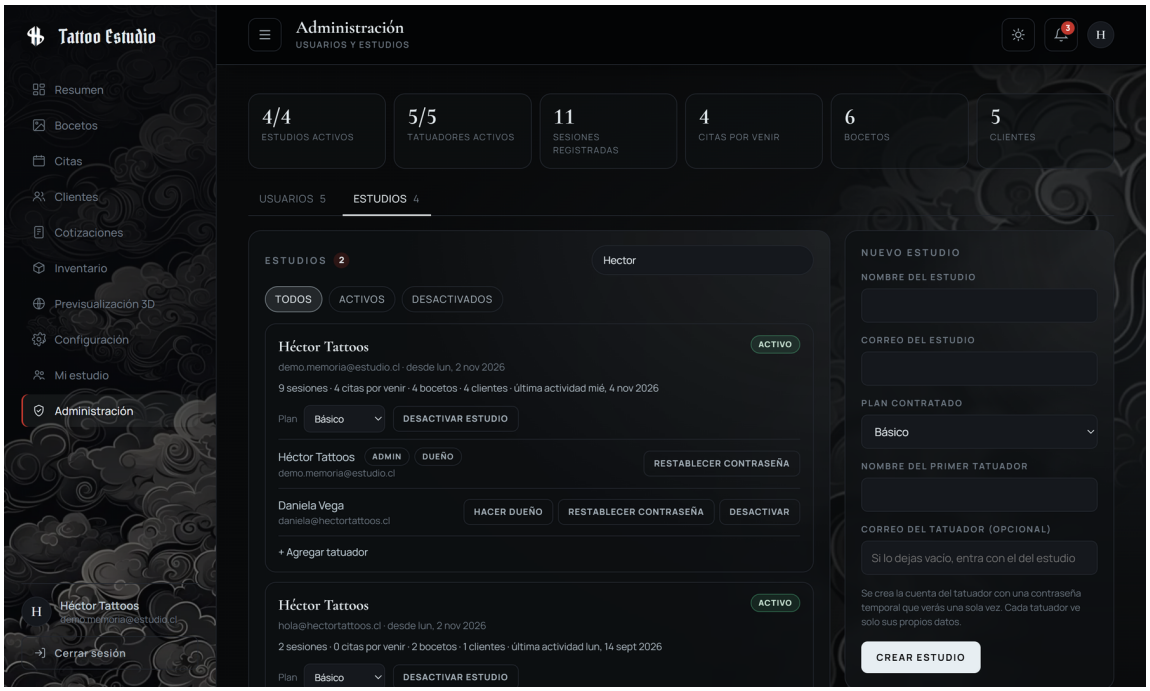


Figura 4.65: Vista “Administración”: totales de la plataforma y uso de cada estudio, con el buscador aplicado.

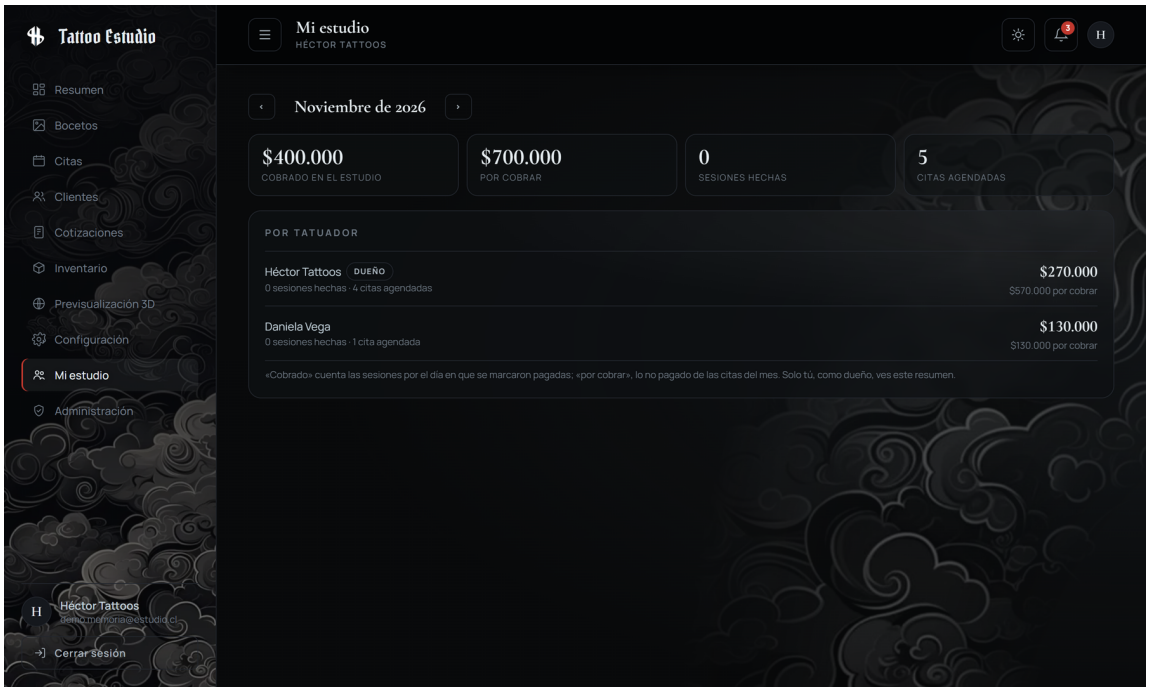


Figura 4.66: Vista “Mi estudio”: resumen del mes del estudio, tatuador por tatuador.

Sistema de pruebas

Después de la implementación se lleva a cabo una fase de pruebas para esta iteración, evaluando los siguientes casos:

- Entrar al panel y revisar los totales de la plataforma.
- Comparar las cifras de un estudio con lo que tiene cargado.
- Buscar un estudio por su nombre y por el correo de un tatuador.
- Filtrar los estudios y las cuentas por estado.
- Entrar a «Mi estudio» siendo dueño de un estudio con dos tatuadores.
- Intentar entrar a «Mi estudio» sin ser dueño.
- Marcar una sesión como pagada y ver el cambio en el resumen.
- Moverse al mes anterior y al siguiente.

Pruebas unitarias

Al finalizar el desarrollo se crean las pruebas unitarias correspondientes a las tareas definidas en el Inicio de Iteración, con Jest y Supertest y con la base de datos reemplazada por dobles de prueba. La Tabla 4.28 detalla las pruebas implementadas.

Tabla 4.28: Pruebas unitarias implementadas en la Iteración 10

ID	**Función evaluada**	**Objetivo**	**Tests**
PU-46	`GET /admin/studios`, `/users`	Validar el listado con tatuadores, métricas y totales, y que un tatuador no entra.	3
PU-47	`GET /api/studio`	Comprobar que dice si la cuenta es dueña y cuántos tatuadores tiene el estudio.	2
PU-48	`GET /api/studio/summary`	Verificar que solo el dueño lo ve, el detalle por tatuador y el rechazo de un rango mayor a un mes.	3

En total, durante esta iteración se ejecutaron 8 pruebas unitarias repartidas en dos archivos de prueba. Esos dos archivos suman 33 pruebas, porque incluyen también las de la administración de cuentas de las Iteraciones 7 y 9.

Resultados obtenidos

La ejecución de las pruebas evidenció un funcionamiento correcto del listado, de las métricas y del resumen del estudio. Se comprobó especialmente que las cifras de cada estudio corresponden a las de sus tatuadores sin multiplicarse entre sí; que un tatuador que no es dueño recibe 403 al pedir el resumen, aunque pertenezca al estudio; y que un rango de más de un mes se rechaza. El resultado general fue satisfactorio, con aprobación completa de los casos evaluados, como resume la Tabla 4.29 y muestra la Figura 4.67.

Tabla 4.29: Resultado de ejecución de pruebas unitarias de la Iteración 10

Archivos evaluados	Tests ejecutados	Resultado
2	8	8 aprobados

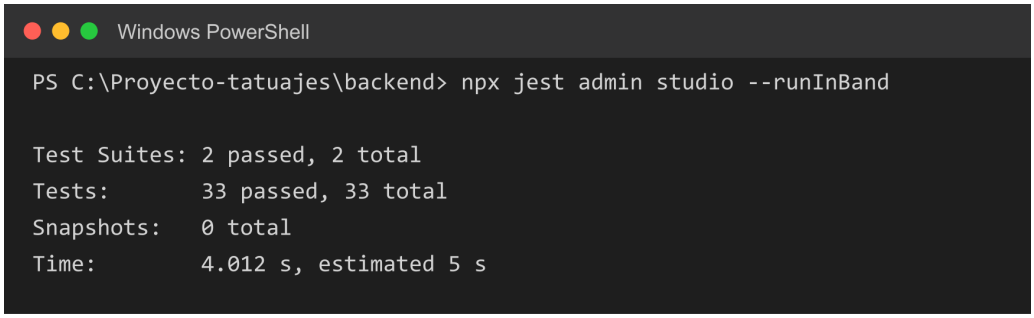


Figura 4.67: Ejecución de los dos archivos de administración y estudio, que reúnen las 8 pruebas de esta iteración junto con 25 de iteraciones anteriores.

Retrospectiva

Al finalizar la iteración se revisa si las estimaciones de tiempo fueron precisas y se identifican posibles mejoras en el proceso de desarrollo. La HU35 y la HU36 se completaron dentro de lo estimado, con la salvedad de que la segunda creció: a su enunciado, que hablaba del administrador de la plataforma, se le sumó el resumen para el dueño del estudio, que no estaba escrito en ninguna historia y que surgió

al preguntar para qué servirían las métricas. Es la única vez en el proyecto en que una iteración entregó más de lo que pedía su historia, y vale anotar que eso solo fue posible porque era la última y no había nada detrás esperando.

Siendo la iteración de cierre, conviene mirar el backlog completo. De las 38 historias de usuario definidas, 36 quedaron implementadas. Las otras dos no se abandonaron por falta de tiempo sino por decisión explícita: la HU27 se descartó porque describía una forma de trabajar que no era la del estudio, y la necesidad detrás de ella quedó cubierta por la revisión de horarios de la Iteración 6; la HU37 quedó como trabajo futuro porque su costo real, al examinarla, resultó desproporcionado frente a lo que aportaba. A ellas se suma la HU29, que no necesitó desarrollo propio porque lo que pedía ya estaba construido. Las tres aparecen documentadas en la iteración que les correspondía, con su razón, en lugar de quedar como casillas sin marcar.

El aprendizaje de cierre es que la estimación inicial acertó en el total pero no en el reparto. Las historias que resultaron caras —la previsualización 3D, el reparto de tinta por color, los recordatorios por WhatsApp— lo fueron por motivos que no se ven al escribir la historia: geometría, costos de un servicio externo, formas de trabajo del oficio. Las que resultaron baratas lo fueron porque se apoyaron en decisiones tomadas en iteraciones anteriores, sobre todo en las reglas de propiedad de los datos que se fijaron temprano y no hubo que tocar nunca más.

5. Pruebas y análisis de resultados

Este capítulo reúne la evaluación de la plataforma construida. Se aplica lo definido en el Capítulo 3: pruebas de caja negra sobre el sistema en funcionamiento, pruebas de caja blanca sobre el código y la prueba de usabilidad SUS con el usuario final. Los resultados se contrastan después con los objetivos específicos del Capítulo 1 y con los resultados esperados del Capítulo 3.

Conviene decir desde el principio qué parte de esta evaluación está hecha y cuál no. Las pruebas de software se ejecutaron, y lo que aquí se reporta es su salida real. La prueba de usabilidad y la encuesta de contraste, en cambio, dependen de las respuestas del tatuador experto y de un periodo de operación del estudio con la plataforma en uso; a la fecha de esta entrega no se han tomado. Por eso este capítulo describe su instrumento y su procedimiento, y deja declarado lo que falta en lugar de presentar cifras que no se midieron.

5.1. Pruebas

La evaluación del software se organizó en tres planos que se complementan. Las **pruebas funcionales** comprueban el sistema desde fuera, por el mismo camino que usa la aplicación, sin mirar cómo está escrito por dentro. Las **pruebas unitarias** verifican la lógica interna de cada módulo y son las que permiten medir cobertura. La **verificación estática** revisa el código sin ejecutarlo. A ellos se suma la **prueba de usabilidad**, que es la única que no puede automatizarse porque su resultado es el juicio de una persona.

5.1.1. Pruebas funcionales

Las pruebas funcionales se diseñaron como pruebas de caja negra, según lo descrito en el Capítulo 3: se define una entrada, se declara la salida esperada y se compara con lo que el sistema devuelve, sin conocimiento de su implementación. Cada caso entra por la interfaz de programación del backend, que es exactamente la puerta que usa la aplicación web, de modo que lo que se prueba es el recorrido completo: validación de la petición, control de acceso, reglas de negocio, escritura en la base de datos y respuesta.

Los casos se derivaron de las historias de usuario. De cada módulo se tomó el flujo principal y, junto a él, los rechazos que el sistema debe producir: once de los treinta y cinco casos esperan que la petición sea rechazada, y el caso se considera conforme solo si el rechazo ocurre con el código y el mensaje correctos. Esto es deliberado, porque buena parte de las reglas del sistema son negativas —una hora ocupada no se vuelve a agendar, una cotización no se descuenta dos veces, un artista no ve los datos de otro— y una prueba que solo recorriera el camino feliz no las tocaría.

Para las reglas de propiedad de los datos la corrida crea **dos cuentas distintas**: la segunda intenta leer, editar y referenciar lo que creó la primera. Son los casos CP-10, CP-12 y CP-33, a los que se suma CP-35, donde una cuenta desactivada por el administrador pierde el acceso en su siguiente petición.

El entorno de ejecución es el mismo que describe el Capítulo 4: el backend sobre Node.js, la base de datos PostgreSQL y el almacenamiento de archivos, los tres levantados en contenedores. Las imágenes que se suben en los casos de boceto y de foto de sesión son archivos reales, de modo que también se ejercita la medición de tinta y el guardado en disco. El registro de cada caso —entrada, salida esperada, salida obtenida y veredicto— queda en un archivo de resultados que es el que alimenta las tablas de la sección siguiente.

Pruebas unitarias y de integración

Las pruebas de caja blanca son las que el Capítulo 4 va presentando iteración por iteración. Están escritas con Jest y Supertest, y se agrupan en dos baterías que se ejecutan en una sola corrida.

La primera son las **pruebas unitarias**: ejercitan funciones puras —el motor de

cotización, el emparejamiento de color, la normalización de teléfonos, el cifrado de la sesión de WhatsApp— y levantan la aplicación Express en memoria para hacerle peticiones, con la base de datos sustituida por dobles. Al no depender de una base levantada corren en segundos, pero dejan sin ejecutar justamente las consultas SQL.

La segunda son las **pruebas de integración**, que cubren ese hueco: corren contra un PostgreSQL de verdad, en una base aparte que la propia corrida crea, a la que aplica el esquema del proyecto y cuyas tablas vacía antes de cada caso. Son las que permiten comprobar lo que solo el motor de base de datos puede demostrar: que dos peticiones simultáneas no agendan la misma hora, que el stock no queda negativo, que una cotización no se descuenta dos veces y que las consultas filtran siempre por el artista dueño de los datos. Una comprobación en el arranque impide que esta batería apunte por error a la base de desarrollo: si el nombre de la base conectada no es el de pruebas, se detiene antes de vaciar nada.

Verificación estática

Antes de cada entrega se revisa el código sin ejecutarlo, con dos herramientas que el proyecto ya usa: el compilador de TypeScript, que comprueba los tipos de todo el frontend, y ESLint, que aplica las reglas de estilo y detecta construcciones sospechosas. No sustituyen a las pruebas, pero atrapan temprano una familia de errores que de otro modo llegaría a la ejecución.

5.1.2. Pruebas de usabilidad (SUS)

La prueba de usabilidad aplica el cuestionario SUS descrito en el Capítulo 3: diez afirmaciones que se responden en una escala de 1 a 5, cinco redactadas en positivo y cinco en negativo, cuyo puntaje final se obtiene con el cálculo allí detallado y se sitúa entre 0 y 100. El proyecto se fijó como meta un puntaje de al menos 85 puntos.

El procedimiento previsto es el siguiente. Antes de responder, el participante recorre la plataforma por su cuenta ejecutando una secuencia de tareas que atraviesa los cinco módulos: subir un boceto a la galería, colocarlo sobre un modelo anatómico y guardar la escena, agendar una cita para un cliente nuevo, cotizar el costo de insumos de ese trabajo y revisar el inventario después del descuento. Las tareas se entregan por escrito y sin acompañamiento, porque una de las afirmaciones del cuestionario pregunta justamente si haría falta ayuda técnica para usar la plataforma. Recién

terminada esa sesión se responde el cuestionario.

El participante principal es el tatuador experto que planteó los requisitos, cuyo juicio es el que más pesa porque conoce el trabajo que la plataforma pretende ordenar. Un único respondiente, sin embargo, no permite hablar de un puntaje representativo, de manera que el instrumento está pensado para aplicarse también a otros tatuadores del mismo entorno.

Esta prueba todavía no se ha aplicado. Lo que falta no es el instrumento ni el procedimiento, que están definidos, sino la sesión con los participantes.

5.2. Resultados

5.2.1. Resultados de pruebas funcionales

Se ejecutaron los 35 casos y los 35 resultaron conformes. Las Tablas 5.1 a 5.4 presentan el detalle agrupado por módulo, con la salida real de la corrida en la última columna; la Figura 5.1 muestra la ejecución completa.

Tabla 5.1: Casos de prueba funcional: autenticación y bocetos

ID	**Caso de prueba y resultado esperado**	**Resultado obtenido**
CP-01	Registrar una cuenta nueva (HU01). 201 con token y datos del usuario	201, token emitido para cp-mwegg7pc-a@pruebas.local
CP-02	Registrar con un correo ya usado (HU01). 409 y ninguna cuenta nueva	409 Ya existe una cuenta con ese correo.
CP-03	Iniciar sesión con credenciales válidas (HU02). 200 con token y rol del usuario	200, rol tatuador
CP-04	Iniciar sesión con contraseña incorrecta (HU02). 401 sin token	401 sin token
CP-05	Entrar a un recurso privado sin sesión (HU02). 401 y ningún dato	401 sin datos
CP-06	Subir un boceto con su imagen (HU04). 201, el boceto queda con enlace y cobertura de tinta medida	201, cobertura 0.7 %
CP-07	Subir un boceto sin título (HU04). 400 y el boceto no se guarda	400 El título es obligatorio.
CP-08	Filtrar la galería por etiqueta (HU06). Solo los bocetos que la tienen	200, 1 boceto(s)
CP-09	Editar los datos de un boceto (HU05). 200 con los datos actualizados	200, título ≪Boceto editado≫
CP-10	Editar el boceto de otro artista (HU06). 404, sin revelar que existe	404, sin cambios

Tabla 5.2: Casos de prueba funcional: previsualización, inventario y cotizaciones

ID	**Caso de prueba y resultado esperado**	**Resultado obtenido**
CP-11	Guardar una escena 3D con el boceto colocado (HU09). 201 y la escena conserva la calca	201, 1 calca(s)
CP-12	Guardar una escena con el boceto de otro artista (HU09). 400, la referencia ajena se rechaza	400 Alguno de los bocetos no existe o no es tuyo.
CP-13	Entrar por primera vez al inventario (HU11). Se cargan los 42 insumos del kit inicial	200, 42 insumos
CP-14	Registrar un insumo nuevo (HU11). 201 con el insumo creado	201, insumo creado
CP-15	Descontar más stock del que hay (HU12). El stock queda en 0, nunca negativo	200, stock 0
CP-16	Listar los insumos en nivel crítico (HU13). Aparece el insumo cuyo stock llegó a 0	200, 1 en nivel crítico
CP-17	Estimar el costo de un tatuaje (HU14). Costo mayor que cero, con tiempo y sesiones calculados	200, $7340, 30 min, 1 sesión(es)
CP-18	Guardar la cotización (HU15). 201 con el detalle por insumo	201, 22 líneas de insumo
CP-19	Descontar del inventario lo cotizado (HU16). El stock de los insumos baja según el detalle	200, ≪Cartuchos magnum 09 (M1)≫ de 20 a 19
CP-20	Descontar dos veces la misma cotización (HU16). 409, el inventario no se descuenta de nuevo	409 Esta cotización ya descontó el stock.

Tabla 5.3: Casos de prueba funcional: agenda y pagos

ID	**Caso de prueba y resultado esperado**	**Resultado obtenido**
CP-21	Crear una ficha de cliente (HU20). 201 con el cliente creado	201, cliente creado
CP-22	Crear un proyecto del cliente (HU19). 201 con el proyecto enlazado al cliente	201, proyecto creado
CP-23	Agendar una cita (HU17). 201 con la cita agendada	201, cita agendada
CP-24	Agendar una cita que se solapa con otra (HU17). 409 nombrando con quién y a qué hora choca	409 Ese horario se cruza con la cita de Cliente de prueba del viernes 20 de noviembre, de 10:00 a 12:00. Elige otra hora.
CP-25	Agendar una cita contigua a otra (HU17). 201, las citas seguidas no se consideran choque	201, cita agendada
CP-26	Cancelar una cita libera su horario (HU21). 201, la hora vuelve a estar disponible	201, hora reutilizada
CP-27	Calificar una sesión con un valor fuera de rango (HU30). 400 y la calificación no se guarda	400 El puntaje debe ser un número entero entre 1 y 5.
CP-28	Calificar una sesión completada (HU30). El cliente queda con promedio 4,0 sobre una sesión calificada	promedio 4 sobre 1 sesión(es)
CP-29	Marcar una sesión como pagada (HU24). Queda sellada la fecha de pago y el monto entra en lo cobrado del mes	fecha de pago sellada, cobrado $90000
CP-30	Subir una foto del avance (HU18). 201 y la foto queda asociada a la sesión	201, foto guardada

Tabla 5.4: Casos de prueba funcional: avisos y administración

ID	Caso de prueba y resultado esperado	Resultado obtenido
CP-31	Cambiar la hora de una cita futura (HU23). La respuesta trae el aviso de reagendamiento para el cliente	200, aviso «reagendamiento»
CP-32	Editar el texto de los mensajes al cliente (HU28). Los textos quedan guardados y se leen igual	200, nota guardada
CP-33	Entrar a la administración sin ser administrador (HU31). 403, la ruta es solo del administrador	403
CP-34	Registrar un estudio con su primer tatuador (HU31). 201 con el estudio, la cuenta y una contraseña temporal	201, estudio y cuenta creados con contraseña temporal
CP-35	Desactivar una cuenta (HU34). Su siguiente petición es rechazada con 401 y el motivo	401 con código «user_inactive»

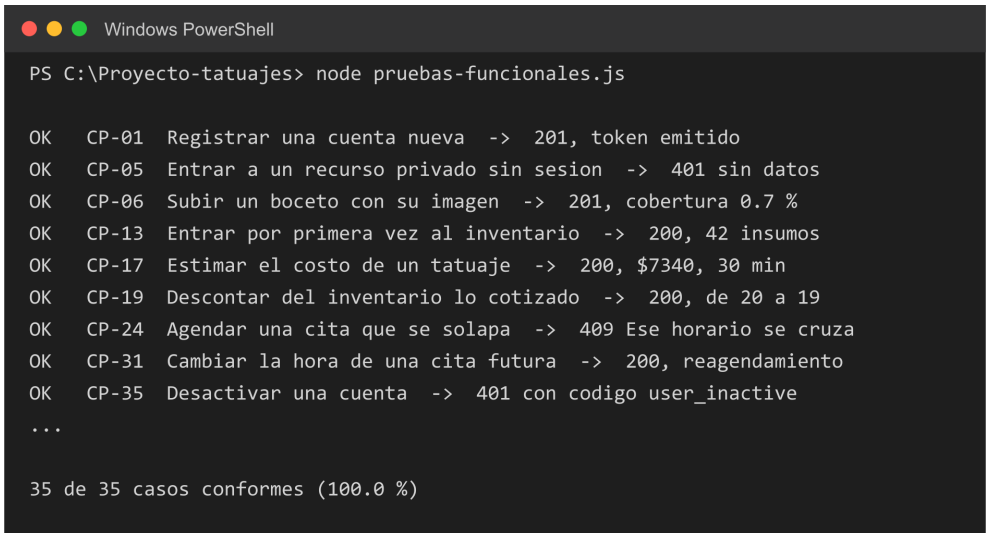


Figura 5.1: Ejecución de los 35 casos de prueba funcional contra el sistema en funcionamiento.

Un resultado de 35 sobre 35 pide una advertencia para leerse bien: estos casos se escribieron sobre un sistema ya terminado y estabilizado, de modo que lo que miden

es que las reglas comprometidas siguen cumpliéndose, no la cantidad de defectos que el proyecto tuvo. Los defectos reales aparecieron durante el desarrollo y están documentados en las retrospectivas del Capítulo 4, con el cambio que cada uno obligó a hacer.

Pruebas unitarias y de integración

La batería completa reúne **783 pruebas en 44 archivos** —672 unitarias y 111 de integración— y se ejecuta entera en algo menos de un minuto. La Tabla 5.5 las agrupa por módulo y la Figura 5.2 muestra la ejecución.

Tabla 5.5: Pruebas automatizadas por módulo

Módulo	**Unitarias**	**Integración**	**Total**
Mensajes al cliente y avisos	240	26	266
Agenda, sesiones y pagos	108	29	137
Cotizaciones y reparto de tinta	96	13	109
Autenticación y control de acceso	66	13	79
Inventario	40	11	51
Administración y estudios	40	8	48
Bocetos	31	6	37
Previsualización 3D	30	5	35
Almacenamiento de archivos	16	—	16
Valores por defecto del esquema	5	—	5
Total	**672**	**111**	**783**

Dos filas merecen una aclaración. La de **previsualización 3D** aparece aquí con 35 pruebas, mientras que la Iteración 8 declaraba no tener ninguna: esas pruebas se escribieron durante esta evaluación y cubren el lado del servidor —qué se acepta como escena válida, que una calca no pueda referenciar el boceto de otro artista, que la escena ajena no se vea ni se edite—. Lo que sigue sin comprobarse con aserciones es la geometría del visor, que vive en el navegador: que la calca se adapte a la curvatura, que no se repita en la cara opuesta del miembro, que el arrastre se sienta

fluido. Eso se verificó midiendo el rendimiento (Tabla 4.23) y mirando el resultado, y sigue siendo la deuda técnica más clara del proyecto.

La otra es la de **almacenamiento de archivos**, que no existía en el capítulo anterior: el driver de disco se prueba escribiendo de verdad en una carpeta temporal, y el de nube con el SDK sustituido por un doble, de modo que se puede verificar sin tener una cuenta contratada.

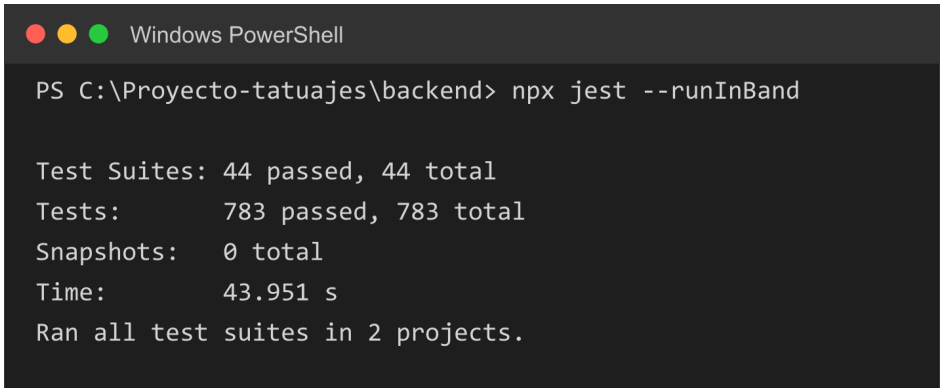


Figura 5.2: Ejecución de las dos baterías del backend: 44 archivos y 783 pruebas, todas aprobadas.

Cobertura de código

La cobertura se midió sobre todo el código del servicio y se resume en la Tabla 5.6. El total es de **95,5 % de las sentencias y 97,2 % de las líneas**, con lo que se alcanza el 95 % que el proyecto se fijó como objetivo. La cobertura de ramas, más exigente porque cuenta cada bifurcación por separado, queda en 85,7 %. Quedan fuera de la medición los scripts de consola (`db:init`, `admin:create` y los demás de mantenimiento), que se ejecutan a mano y no forman parte de la aplicación en marcha; la exclusión está declarada en la configuración de las pruebas.

Tabla 5.6: Cobertura de código por capa

Capa	**Sentencias**	**Ramas**	**Funciones**
Rutas	100 %	100 %	100 %
Datos de catálogo	100 %	100 %	100 %
Almacenamiento de archivos	97,0 %	77,1 %	100 %
Modelos (consultas SQL)	96,7 %	90,7 %	97,2 %
Servicios	96,0 %	84,4 %	92,7 %
Utilidades	96,0 %	87,1 %	100 %
Controladores	95,9 %	85,1 %	98,3 %
Configuración	95,2 %	100 %	66,7 %
Middleware	95,2 %	95,0 %	100 %
Servicios de WhatsApp vinculado	89,6 %	79,2 %	62,3 %
Total	**95,5 %**	**85,7 %**	**91,3 %**

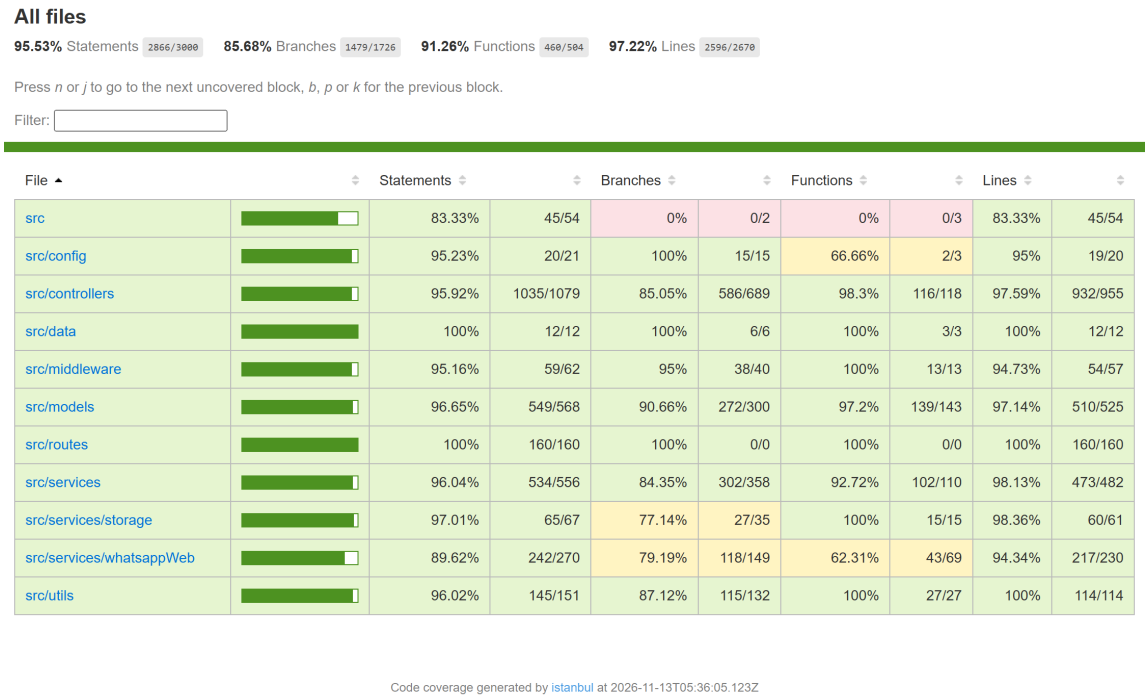
El reparto por capa deja todas las capas propias de la aplicación sobre el 95 %. La que queda abajo es la del **WhatsApp vinculado**, con 89,6 %: lo que no se ejecuta ahí son los caminos que solo ocurren contra un socket de WhatsApp real —reconexiones, códigos de desconexión del proveedor—, y simularlos con un doble no probaría gran cosa. A ellos se suman el lector de imágenes, cuyas ramas dependen de formatos que `sharp` interpreta por dentro, y el arranque del servidor.

Vale contrastarlo con el punto de partida. La primera medición de esta evaluación dio 67,2 % de las sentencias, con la capa de modelos en 22 %: las consultas SQL se sustituían por dobles y nunca llegaban a ejecutarse. Escribir la batería de integración la llevó a 96,7 % y el total a 90,6 %; cubrir después los caminos de error de los controladores, el programador de recordatorios y el envío de correo por SMTP completó el 95,5 % actual.

La cobertura de **ramas** es la que sigue más baja, en 85,7 %, y conviene no disimularlo: mide cada bifurcación por separado, de modo que una condición con tres causas posibles exige tres pruebas aunque la línea ya se haya ejecutado. El objetivo del proyecto se fijó sobre la cobertura de código sin distinguir la métrica, y es la de

sentencias y líneas la que se informa habitualmente.

La Figura 5.3 es el informe que la propia herramienta de medición deja escrito al terminar la corrida. No es un resumen redactado para esta memoria: se genera con `npx jest --coverage` y queda en `coverage/index.html`, con el detalle de cada capa —cuántas sentencias, ramas, funciones y líneas se ejecutaron sobre el total— y la fecha en que se midió.



All files

95.53% Statements 2866/3000 **85.68%** Branches 1479/1726 **91.26%** Functions 460/504 **97.22%** Lines 2596/2670

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	83.33%	45/54	0%	0/2	0%	0/3	83.33%	45/54
src/config	95.23%	20/21	100%	15/15	66.66%	2/3	95%	19/20
src/controllers	95.92%	1035/1079	85.05%	586/689	98.3%	116/118	97.59%	932/955
src/data	100%	12/12	100%	6/6	100%	3/3	100%	12/12
src/middleware	95.16%	59/62	95%	38/40	100%	13/13	94.73%	54/57
src/models	96.65%	549/568	90.66%	272/300	97.2%	139/143	97.14%	510/525
src/routes	100%	160/160	100%	0/0	100%	0/0	100%	160/160
src/services	96.04%	534/556	84.35%	302/358	92.72%	102/110	98.13%	473/482
src/services/storage	97.01%	65/67	77.14%	27/35	100%	15/15	98.36%	60/61
src/services/whatsappWeb	89.62%	242/270	79.19%	118/149	62.31%	43/69	94.34%	217/230
src/utils	96.02%	145/151	87.12%	115/132	100%	27/27	100%	114/114

Code coverage generated by istanbul at 2026-11-13T05:36:05.123Z

Figura 5.3: Informe de cobertura generado por la herramienta tras la corrida completa: 2.866 de 3.000 sentencias y 2.596 de 2.670 líneas ejecutadas.

Verificación estática

El compilador de TypeScript recorre el frontend completo sin reportar un solo error de tipos, y ESLint no levanta advertencias sobre el código del proyecto. Las dos revisiones están limpias.

5.2.2. Resultados de usabilidad (SUS)

La prueba SUS no se ha aplicado todavía, de modo que no hay un puntaje que informar. Cuando se aplique, lo que corresponde reportar aquí es el puntaje de cada

participante, el promedio del grupo y su clasificación según la escala de la Tabla 5.7, junto con el contraste contra la meta de 85 puntos que fijó el proyecto.

Tabla 5.7: Escala de interpretación del puntaje SUS

Puntaje	**Interpretación**
0 a 49	Inaceptable
50 a 68	Mediocre
69 a 100	Aceptable
85	Meta declarada para este proyecto

Lo mismo ocurre con la encuesta de contraste del Capítulo 3, la que compara el estado inicial y el final de los tiempos del tatuador. Sus preguntas están definidas y el estado inicial corresponde a lo que el tatuador experto describió al plantear el problema, pero el estado final exige que el estudio opere con la plataforma durante un periodo suficiente para que las cifras signifiquen algo. Ese periodo no ha ocurrido.

5.3. Análisis de los resultados

Puestos los resultados junto a los objetivos específicos del Capítulo 1, el balance es el que sigue.

El objetivo de **implementar la gestión de bocetos en la nube** está cumplido y verificado: cinco casos funcionales recorren la subida con medición de tinta, el filtrado de la galería, la edición y el rechazo del acceso ajeno, y el módulo tiene además sus pruebas unitarias.

El objetivo de **evaluar la calidad mediante pruebas unitarias, de caja negra y de caja blanca con cobertura de al menos 95 %** está cumplido. Hay 783 pruebas unitarias y de integración, y 35 casos de caja negra ejecutados sobre el sistema en funcionamiento, todos conformes; la cobertura llega al 95,5 % de las sentencias y al 97,2 % de las líneas.

Corresponde precisar dos cosas. La primera es que la cobertura de ramas queda en 85,7 %, por debajo de esa cifra: el objetivo no nombraba una métrica y se informa la de sentencias y líneas, que es la habitual, pero el dato completo está en la Tabla

5.6. La segunda es que llegar aquí no fue escribir más aserciones sobre lo mismo. La medición inicial, 67,2 %, dejaba fuera toda la capa de consultas SQL, que las pruebas sustituían por dobles; cerrarla exigió montar una segunda batería contra una base de datos real, y esa batería es la que hoy comprueba que dos peticiones simultáneas no agendan la misma hora o que una cotización no se descuenta dos veces. Lo que la cobertura no mide, y conviene recordar, es si las pruebas comprueban lo correcto: eso lo sostienen los 35 casos funcionales y las decisiones de diseño que cada iteración documenta.

Los objetivos de **reducción de tiempos y de ausentismo** —50 % en inasistencia, 40 % en cotización, 30 % en previsualización— no pueden darse por cumplidos ni por incumplidos con la evidencia disponible, porque son medidas sobre el trabajo de una persona y exigen la encuesta de contraste. Lo que sí está verificado es que los mecanismos que deberían producir esas reducciones funcionan: el recordatorio sale solo 24 horas antes de la cita y no se repite aunque el programador corra dos veces, el cambio de hora genera el aviso al cliente, la cotización calcula el costo de insumos a partir del boceto medido y descuenta el inventario al cerrar la sesión, y la escena 3D guarda la posición y el tamaño del diseño sobre el modelo. Que esos mecanismos funcionen es condición necesaria de las reducciones esperadas, pero no es la medición de las reducciones.

El objetivo de **validación con el usuario** —aceptación sobre el 90 % y SUS sobre 85 puntos— está pendiente por completo.

Más allá del cumplimiento formal, hay un resultado que conviene destacar porque atraviesa todo el sistema. Cuatro de los casos funcionales comprueban que un artista no alcanza los datos de otro, por distintas vías: pidiendo un recurso ajeno, referenciándolo dentro de otro recurso, entrando a una ruta de administración sin el rol, y operando con una cuenta recién desactivada. Los cuatro se rechazan. Esa es la regla sobre la que descansa que la plataforma pueda alojar a varios estudios a la vez, y es también la decisión que el Capítulo 4 identifica como la más barata de todo el proyecto, precisamente porque se tomó temprano y no hubo que volver sobre ella.

5.4. Análisis de accesibilidad

La accesibilidad se evaluó tomando como referencia las pautas WCAG 2.1 en su nivel AA, a las que se suma el criterio de tamaño de objetivo que incorporó la versión 2.2. La revisión se hizo con un guion propio que recorre el árbol del documento de las vistas ya cargadas en el navegador y comprueba criterios verificables de forma automática —texto alternativo, nombre accesible de los controles, etiquetas de formulario, idioma declarado, relación de contraste y tamaño de los objetivos táctiles—, complementada con una revisión visual de las mismas pantallas. Se revisaron el resumen, la agenda y el inventario, que entre las tres reúnen los patrones de interfaz que el resto de las pantallas repite.

Tabla 5.8: Resultados de la revisión de accesibilidad

Criterio	**Resultado de la revisión**	**Estado**
Texto alternativo (1.1.1)	Todas las imágenes de las vistas revisadas declaran su alternativa textual	Cumple
Nombre accesible de los controles (4.1.2)	Ninguno de los 193 controles del inventario queda sin nombre; el código declara 67 etiquetas `aria-label` y 34 `aria-labelledby`	Cumple
Etiquetas de formulario (3.3.2)	Los 11 campos del inventario y los 6 de la agenda tienen etiqueta asociada	Cumple
Idioma de la página (3.1.1)	El documento declara español	Cumple
Contraste mínimo (1.4.3)	El color del texto secundario sobre las tarjetas da 4,42:1, bajo el 4,5:1 exigido; afecta a 144 textos de 12 px en una sola vista	No cumple
Estructura de encabezados (1.3.1)	El resumen salta del encabezado de primer nivel al de tercero	Parcial
Región principal y salto de bloques (2.4.1)	Las vistas declaran cabecera y navegación, pero no una región de contenido principal	No cumple
Tamaño del objetivo (2.5.8, WCAG 2.2)	Entre 42 y 46 controles por vista miden menos de 24 px en una de sus dimensiones, sobre todo las celdas del calendario y los enlaces en línea	No cumple

Lo que la revisión deja claro es que la base está bien puesta y lo que falla es de ajuste. Los puntos difíciles de corregir después —que cada control tenga nombre, que cada campo tenga etiqueta, que las imágenes tengan alternativa— están

resueltos desde el código, y eso se debe a que la interfaz se escribió con etiquetas de accesibilidad desde el principio y no como un parche final.

Los tres incumplimientos son, en cambio, baratos de cerrar. El contraste falla por ocho centésimas: basta aclarar un paso el color del texto secundario para pasar el umbral en todas las pantallas a la vez, porque es un único valor del sistema de diseño. La región principal se resuelve envolviendo el contenido de la plantilla común. El tamaño de los objetivos exige algo más de trabajo, porque implica agrandar las celdas del calendario del mes, que es donde se concentra el problema.

Queda una limitación declarada: la medición de contraste del **tema claro** no pudo hacerse de forma sistemática con el guion, porque cada pantalla redefine sus propios colores para ese tema y el guion no siempre resuelve cuál es el fondo real de un texto. Se revisó visualmente sin encontrar texto ilegible, pero esa comprobación no tiene el mismo valor que una medición y queda como trabajo pendiente.

5.5. Análisis de sostenibilidad

Sostenibilidad económica

La plataforma se opera hoy sin costo mensual, como detalla el análisis de factibilidad del Capítulo 4: cada pieza vive en el plan gratuito de un servicio distinto y no hay licencias que pagar. Para el estudio pequeño al que apunta el proyecto, esto es lo que la vuelve adoptable: no hay una suscripción que justificar antes de probar si sirve. El límite de ese modelo también está dicho: los planes gratuitos imponen cuotas y pueden cambiar sus condiciones, y por eso la arquitectura mantiene cada pieza reemplazable por separado.

Sostenibilidad técnica

Lo que permite que el sistema siga vivo después de esta memoria es, sobre todo, que tiene 783 pruebas que se ejecutan en menos de un minuto: quien venga después puede cambiar algo y saber en el acto si rompió otra cosa. A eso se suman decisiones tomadas para no quedar atados a un proveedor —el almacenamiento de archivos pasa por una capa con controladores intercambiables, el envío de mensajes al cliente tiene tres vías posibles— y una sola base de código para todos los dispositivos, que es lo que hace sostenible el mantenimiento para una persona sola. La deuda conocida

que queda es la geometría del visor 3D, que se verifica mirando y midiendo en el navegador y no con aserciones.

Sostenibilidad ambiental

El consumo de un sistema web se juega en los bytes que viajan y en el cómputo que queda encendido. En lo primero, la aplicación se construye en piezas que se descargan solo cuando hacen falta: la primera carga son 244 kB de código, 79 kB una vez comprimido, más unos 100 kB de estilos, y cada pantalla agrega entre 7 y 20 kB al entrar en ella. Lo pesado del proyecto —el visor tridimensional, de 687 kB, y los modelos anatómicos, de entre 0,47 y 0,76 MB cada uno— no se descarga nunca si el tatuador no abre la previsualización, y aun entonces baja un solo modelo de los catorce. La aplicación guarda su armazón en el dispositivo, de modo que las visitas siguientes no vuelven a pedirlo.

En lo segundo, el servidor gratuito se suspende tras unos minutos sin uso y no consume cómputo mientras nadie lo necesita; la contrapartida es la demora de la primera petición, que el Capítulo 4 ya discute. Hay además un ahorro material que el propio problema planteaba: el papel hectográfico del stencil es desechable y se pierde entero cuando la posición no convence, de manera que cada prueba de ubicación que se hace en pantalla en lugar de en papel es papel que no se bota. Este trabajo no midió cuánto se ahorra, y no corresponde afirmarlo sin medirlo.

Sostenibilidad social

La plataforma guarda los datos de cada artista separados de los del resto, incluso dentro de un mismo estudio, y el detalle de lo que cada tatuador cobra solo lo ve el dueño. No obliga a contratar un servicio de mensajería de pago para usar los recordatorios, ni a comprar equipamiento: basta un teléfono con navegador. Son decisiones que apuntan a que el costo de usar el sistema no recaiga en quien menos puede asumirlo, que es el tatuador independiente.

5.6. Resumen del capítulo

En este capítulo se evaluó la plataforma en tres planos. Las pruebas funcionales de caja negra, 35 casos ejecutados sobre el sistema en funcionamiento, resultaron todas conformes, incluidos los once casos que esperan un rechazo y los cuatro que

comprueban que un artista no alcanza los datos de otro. Las pruebas unitarias y de integración suman 783 en 44 archivos, todas aprobadas, con una cobertura de 95,5 % de las sentencias y 97,2 % de las líneas, que cumple el 95 % comprometido; la de ramas queda en 85,7 %. La capa de consultas SQL, que las pruebas antes sustituían por dobles y nunca llegaban a ejecutar, hoy está cubierta al 96,7 % por una batería que corre contra una base de datos real. La verificación estática del frontend está limpia.

La evaluación de usabilidad con el cuestionario SUS y la encuesta de contraste de tiempos quedaron pendientes: su instrumento y su procedimiento están definidos en este capítulo, pero no se han aplicado, y por lo tanto los objetivos de reducción de tiempos y de validación con el usuario no pueden darse por cumplidos. El análisis de accesibilidad encontró resueltos los criterios estructurales y tres incumplimientos de ajuste —contraste, región principal y tamaño de los objetivos—, todos de corrección acotada. El análisis de sostenibilidad mostró un sistema que opera sin costo mensual, que descarga lo pesado solo cuando se usa y que deja dos deudas técnicas declaradas.

En el siguiente capítulo se revisa el cumplimiento de los objetivos del proyecto, se reflexiona sobre el trabajo realizado y se propone el trabajo futuro.

Glosario

Framework: Un framework es un conjunto de herramientas, bibliotecas y convenciones que proporcionan una estructura predefinida para el desarrollo de aplicaciones. Facilita la organización del código, la reutilización de componentes y la implementación de funcionalidades comunes, permitiendo a los desarrolladores centrarse en la lógica específica de su aplicación en lugar de preocuparse por detalles de bajo nivel.

JWT (JSON Web Token): Es un estándar abierto (RFC 7519) que define un formato compacto y seguro para la transmisión de información entre partes como un objeto JSON. Se utiliza comúnmente para la autenticación y autorización en aplicaciones web, permitiendo a los servidores generar tokens que contienen información del usuario y sus permisos, los cuales pueden ser verificados y decodificados por el cliente para acceder a recursos protegidos.

Nodemailer: Es una biblioteca de Node.js que facilita el envío de correos electrónicos desde aplicaciones Node.js. Proporciona una interfaz sencilla para configurar y enviar correos electrónicos utilizando diversos servicios de correo, como SMTP, Gmail, Outlook, entre otros

Supertest: Es una biblioteca de pruebas para Node.js que permite realizar pruebas de integración en aplicaciones web. Proporciona una API fácil de usar para simular solicitudes HTTP y verificar las respuestas del servidor, lo que facilita la validación de rutas, controladores y middleware en aplicaciones Express u otros frameworks web.

Jest: Es un framework de pruebas de JavaScript desarrollado por Facebook, diseñado para trabajar con proyectos que utilizan React, aunque también es

compatible con otros entornos JavaScript. Jest proporciona una experiencia de prueba completa con características como pruebas unitarias, pruebas de integración, pruebas de extremo a extremo, y una interfaz de usuario amigable.

API: Una API (Interfaz de Programación de Aplicaciones) es un conjunto de reglas y protocolos que permiten la comunicación entre diferentes software. Las APIs definen cómo los componentes de software deben interactuar, permitiendo a los desarrolladores acceder a funcionalidades específicas de una aplicación o servicio sin necesidad de conocer su implementación interna.

SMTP (Simple Mail Transfer Protocol): Es un protocolo de comunicación utilizado para el envío de correos electrónicos a través de Internet. SMTP define las reglas y procedimientos para la transferencia de mensajes de correo electrónico entre servidores de correo, permitiendo a los usuarios enviar correos electrónicos desde sus clientes de correo electrónico a los servidores de correo y entre servidores de correo.

GitHub: Es una plataforma de desarrollo colaborativo que utiliza el sistema de control de versiones Git. GitHub permite a los desarrolladores alojar, compartir y colaborar en proyectos de software, facilitando la gestión de código fuente, el seguimiento de cambios, la revisión de código y la integración continua.

Bibliografía

[1] Julián Esteban Ballén Valderrama y Javier Antonio Castillo López. «La práctica del tatuaje y la imagen corporal». En: *Revista Iberoamericana de psicología* 8.1 (2015), págs. 103-109.

[2] F. Poulsen Valenzuela. «El tatuaje: más allá de la tinta». Memoria de Título, Escuela de Diseño. Pontificia Universidad Católica de Chile, 2021.

[3] Informes de Expertos (IDE). *Mercado de Tatuajes: Tamaño, Análisis, Pronóstico 2026-2035*. 2026. URL: https://www.informesdeexpertos.com/informes/mercado-de-tatuajes.

[4] P. Sepúlveda. «El 17 % de los chilenos mayores de 15 años tiene un tatuaje». En: *La Tercera* (oct. de 2017). URL: https://www.latercera.com/noticia/17-los-chilenos-mayores-15-anos-tatuaje/.

[5] Stephen A White. «Introduction to BPMN». En: *Ibm Cooperation* (2004).

[6] WhatsApp LLC. *WhatsApp Messenger*. Aplicación de mensajería multiplataforma y servicio de voz sobre IP. 2026. URL: https://www.whatsapp.com/.

[7] Aamir Qutub. *What is a native app? Pros and cons explained clearly*. s.f. URL: https://clutch.co/resources/the-pros-and-cons-of-native-apps.

[8] ZDNet. *Web app vs native app: Which one should you build?* s.f. URL: https://www.zdnet.com/article/the-benefits-of-native-versus-web-apps/.

[9] Natively. *Web apps vs. native apps: What is the difference?* s.f. URL: https://natively.dev/blog/web-apps-vs-native-apps.

[10] Gisela Fernandes, Filipe Portela y Manuel Filipe Santos. «PWA and pervasive information system–a new era». En: *World Conference on Information Systems and Technologies*. Springer. 2020, págs. 334-343.

[11] Delia Crovi Druetta et al. «Página Web: una propuesta para su análisis». En: *Revista Mexicana de Ciencias Políticas y Sociales* 45.185 (2002), págs. 167-185.

[12] Meta Platforms, Inc. *WhatsApp Cloud API: Automated and transacted messaging for businesses*. Documentación oficial para desarrolladores de Meta. 2026. URL: `https://developers.facebook.com/docs/whatsapp`.

[13] Mariano Báguena Bueso. «El boceto digital. De la idea a la creación». En: *Diálogos urbanos: Confluencias entre arte y ciudad*. Centro de Investigación Arte y Entorno. 2008, págs. 101-110.

[14] C. R. Jordan. *Basic Fundamentals of Modern Tattoo*. Capítulo referente al equipo y aplicación de esténciles. Tattoo Books Online LLC, 2009. ISBN: 978-0615281476.

[15] Tomas Akenine-Möller et al. *Real-Time Rendering*. 4th. El Capítulo 6 y sus subsecciones abordan en profundidad el mapeo de texturas, proyecciones espaciales y técnicas aplicables a decals sobre mallas 3D. A K Peters/CRC Press, 2018. ISBN: 978-1138627000.

[16] Rudd H Canaday et al. «A back-end computer for data base management». En: *Communications of the ACM* 17.10 (1974), págs. 575-582.

[17] Susana Graciela Pérez Ibarra et al. «Herramientas y tecnologías para el desarrollo web desde el FrontEnd al BackEnd». En: *Xxiii workshop de investigadores en ciencias de la computación (wicc 2021, chilecito, la rioja)*. 2021.

[18] Meta Platforms, Inc. *React: A JavaScript library for building user interfaces*. Documentación oficial para desarrolladores de Meta. 2026. URL: `https://react.org`.

[19] Mozilla Developer Network. *JavaScript: Tecnologías web para desarrolladores*. Documentación oficial y referencia del lenguaje. 2026. URL: `https://developer.mozilla.org/es/docs/Web/JavaScript`.

[20] Ricardo Cabello y Three.js Contributors. *Three.js*. Librería 3D de JavaScript basada en WebGL. 2026. URL: `https://threejs.org/`.

[21] Poimandres. *React Three Fiber: A React renderer for Three.js*. Documentación oficial y repositorio de código abierto. 2026. URL: `https://docs.pmnd.rs/react-three-fiber/`.

[22] Unity Technologies. *Unity: Plataforma de desarrollo 3D en tiempo real*. Sitio web oficial y documentación del motor gráfico. 2026. URL: `https://unity.com/`.

[23] Khronos Group. *WebGL Specification: OpenGL ES for the Web*. Documentación oficial y especificación del estándar. 2026. URL: `https://www.khronos.org/webgl/`.

[24] OpenJS Foundation. *Node.js: Entorno de ejecución para JavaScript basado en el motor V8*. Documentación oficial. 2026. URL: `https://nodejs.org/`.

[25] OpenJS Foundation. *Express: Infraestructura web rápida, minimalista y flexible para Node.js*. Documentación oficial y referencia de la API. 2026. URL: `https://expressjs.com/`.

[26] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Capítulo 5: Representational State Transfer (REST). Tesis doctoral. University of California, Irvine, 2000. URL: `https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm`.

[27] ISO/IEC. *ISO/IEC 9075: Information technology — Database languages — SQL*. Estándar Internacional, Organización Internacional de Normalización. 2023. URL: `https://www.iso.org/standard/76533.html`.

[28] Oracle Corporation. *MySQL: The world's most popular open source database*. Documentación oficial. 2026. URL: `https://www.mysql.com/`.

[29] MariaDB Foundation. *MariaDB: The open source relational database*. Documentación oficial. 2026. URL: `https://mariadb.org/`.

[30] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Documentación oficial y referencia del motor. 2026. URL: `https://www.postgresql.org/`.

[31] Theo Härder y Andreas Reuter. «Principles of Transaction-Oriented Database Recovery». En: *ACM Computing Surveys (CSUR)* 15.4 (1983), págs. 287-317. DOI: `10.1145/289.291`.

[32] The PostgreSQL Global Development Group. *PostgreSQL Documentation: JSON Types*. Documentación oficial del motor de base de datos. 2026. URL: `https://www.postgresql.org/docs/current/datatype-json.html`.

[33] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. Request for Comments 8259. Internet Engineering Task Force (IETF), Diciembre de 2017. URL: `https://datatracker.ietf.org/doc/html/rfc8259`.

[34] WhiskeySockets. *Baileys: biblioteca de WhatsApp Web para Node.js*. Repositorio del proyecto; versión 7.0.0-rc14, utilizada para el envío automatizado desde el número del tatuador. 2026. URL: `https://github.com/WhiskeySockets/Baileys`.

[35] Docker, Inc. *Docker: Accelerated Container Application Development*. Documentación oficial y plataforma de contenedores. 2026. URL: `https://www.docker.com/`.

[36] GitHub, Inc. *GitHub: Where the world builds software*. Plataforma de alojamiento de código y colaboración. 2026. URL: `https://github.com/`.

[37] GitKraken. *GitKraken Client: The legendary Git GUI client for Windows, Mac & Linux*. Sitio web oficial y documentación del software. 2026. URL: `https://www.gitkraken.com/`.

[38] GitHub, Inc. *GitHub Desktop: Simple collaboration from your desktop*. Sitio oficial y documentación de la herramienta. 2026. URL: `https://desktop.github.com/`.

[39] Atlassian Corporation. *Trello: Manage your team's projects from anywhere*. Herramienta de gestión visual de proyectos. 2026. URL: `https://trello.com/`.

[40] Atlassian Corporation. *Jira Software: Issue & Project Tracking Software*. Plataforma ágil de desarrollo de software. 2026. URL: `https://www.atlassian.com/software/jira`.

[41] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, Washington: Blue Hole Press, 2010. ISBN: 978-0984521401.

[42] REV23 Development. *REV23 Desktop*. Software de computadora. 2024. URL: `https://www.rev23.com/`.

[43] Savage Interactive. *Procreate*. Aplicación móvil. 2024. URL: `https://procreate.com/`.

[44] Celsys, Inc. *Clip Studio Paint*. Software de ilustración, cómic y animación digital. 2026. URL: `https://www.clipstudio.net/`.

[45] Adobe Inc. *Adobe Substance 3D Painter*. Software de texturizado de modelos 3D. 2026. URL: `https://www.adobe.com/products/substance3d-painter.html`.

[46] Fresha.com SV Ltd. *Fresha*. Plataforma web de agendamiento para salones y bienestar. 2026. URL: `https://www.fresha.com/`.

[47] Calendly LLC. *Calendly*. Software de automatización de programación de citas. 2026. URL: `https://calendly.com/`.

[48] Apple Inc. *Calendario de iOS*. Aplicación móvil de organización personal. 2026. URL: `https://www.apple.com/ios/`.

[49] Tattoo Studio Pro. *Tattoo Studio Pro*. Software de gestión administrativa de estudios de tatuaje. 2026. URL: `https://www.tattoostudiopro.com/`.

[50] Ink Studio AI. *Ink Studio AI*. Generador de diseños de tatuajes basado en inteligencia artificial. 2026. URL: `https://inkstudioai.com/`.

[51] Meta Platforms, Inc. *Instagram*. Red social y repositorio visual en línea. 2026. URL: `https://www.instagram.com/`.

[52] Google LLC. *Google Photos*. Servicio de almacenamiento y organización de fotografías. 2026. URL: `https://photos.google.com/`.

[53] Adobe Inc. *Behance*. Plataforma de portafolios creativos en línea. 2026. URL: `https://www.behance.net/`.

[54] Watts S Humphrey. «The personal software process». En: *CMU/SEI: Addison Wesley* (2000).

[55] Yani Dzhurov, Iva Krasteva y Sylvia Ilieva. «Personal Extreme Programming– An Agile Process for Autonomous Developers». En: (2009).

[56] José Joskowicz. «Reglas y prácticas en eXtreme Programming». En: *Universidad de Vigo* 22 (2008).

[57] Mandy R Drew, Brooke Falcone y Wendy L Baccus. «What does the system usability scale (SUS) measure? validation using think aloud verbalization and behavioral metrics». En: *International conference of design, user experience, and usability*. Springer. 2018, págs. 356-366.

[58] José PONCE GONZÁLEZ et al. «Pruebas de aceptación orientadas al usuario.» En: ().

[59] Planning Poker. «Planning poker». En: *línea]. Disponible: www. planningpoker.* (2016).

[60] María Isabel Viggiani Rocha. «La sucesión de Fibonacci». En: *Revista de Educación Matemática (RevEM)* 21.3 (2006), pág. 3.

[61] Kent Beck. *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley Professional, 2002. ISBN: 978-0321146533.

[62] Jest Core Team. *Jest. Delightful JavaScript Testing*. Ver. 29.7.0. 2026. URL: `https://jestjs.io/`.